DOCKER PRACTICE WORKBOOK

880 Hands-on Exercises with Step-by-Step Solutions

Zero to advanced practice

CLI · Images · Containers · Volumes · Networks · Builds · Compose · Contexts · Swarm · Security · Buildx · Troubleshooting

Designed for active practice: read the task, try it without looking, compare your command, then verify the result.

Source alignment: organized around the main technical themes in the supplied *Docker Deep Dive* (2020 edition), including the Engine, images, containers, networking, Swarm, stacks, and security. Modern CLI workflows are added where useful. Docker changes over time, so verify version-specific flags with your installed CLI and official documentation.

Safety first

Use a disposable development machine. Some exercises require resources to exist first (for example, named containers, a Dockerfile, compose.yaml, stack.yml, a Swarm manager, or a registry account). Replace placeholders such as REGISTRY/USERNAME, NODE_NAME, and PLUGIN_NAME with your own lab values. Do not use real credentials or production data. Commands that remove containers, volumes, networks, stacks, or build cache can be destructive; inspect the target and practice only with disposable lab resources.

How to use this workbook

• **1. Try first.** Read the task and write your command before checking the solution.

• **2. Run and verify.** Follow the verification step; command success alone does not always mean the application is healthy.

• **3. Reset safely.** Use only the named lab resources you created. Avoid broad prune or force-removal commands on machines containing important workloads.

• **4. Check your version.** Run `docker version` and `docker help`; Compose, Buildx, Swarm, and platform-specific options can vary.

• **5. Prepare prerequisites.** If a task references an object that does not exist, create it first or adapt the name to an object in your lab.

Workbook map

01. Docker Engine & CLI foundations — 20 exercises

02. System diagnostics & events — 20 exercises

03. Images: pulling, listing & filtering — 20 exercises

04. Image metadata & layers — 20 exercises

05. Tagging & image archives — 20 exercises

06. Registries & publishing — 20 exercises

07. Container launch & naming — 20 exercises

08. Container runtime options & isolation — 20 exercises

09. Container discovery & startup — 20 exercises

10. Container stop, restart & pause — 20 exercises

11. Container removal & safe cleanup — 20 exercises

12. Interactive access & file copy — 20 exercises

13. Logs, resource use & processes — 20 exercises

14. Container inspection & port mapping — 20 exercises

15. Resource limits, restart policies & health — 20 exercises

16. Volumes: create & inspect — 20 exercises

17. Volumes: persistence & backup — 20 exercises

18. Networks: create & inspect — 20 exercises

19. Networks: connect & disconnect — 20 exercises

20. Image builds: build & cache — 20 exercises

21. Build arguments, targets & BuildKit — 20 exercises

22. Docker Compose: validate & launch — 20 exercises

23. Docker Compose: observe services — 20 exercises

24. Docker Compose: exec & lifecycle — 20 exercises

25. Contexts & daemon endpoints — 20 exercises

26. Docker Swarm: cluster basics — 20 exercises

27. Docker Swarm: services & scaling — 20 exercises

28. Swarm secrets, configs & stacks — 20 exercises

29. Runtime security & permissions — 20 exercises

30. Health checks, multi-platform builds & final workflow — 20 exercises

31. Container signals & exit status — 20 exercises
32. Container identity & terminal attachment — 20 exercises
33. Image/container filesystem changes — 20 exercises
34. Image transfer & import — 20 exercises
35. Image transfer & import — 20 exercises
36. Volume lifecycle & safe removal — 20 exercises
37. Network lifecycle & cleanup — 20 exercises
38. Compose build & image management — 20 exercises
39. Compose one-off jobs & profiles — 20 exercises
40. Swarm service updates & rollback — 20 exercises
41. Swarm node operations — 20 exercises
42. Swarm stack operations — 20 exercises
43. Docker plugins & extension discovery — 20 exercises
44. Build cache & system cleanup — 20 exercises

Total: 44 modules × 20 exercises = 880 exercises.

Module 01: Docker Engine & CLI foundations

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 001 · Check client/server versions

Task: Compare the Docker CLI and Engine versions for lab case 1.

Solution command

```
docker version
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker version`
4. Interpret the result: Read the Client and Server sections separately; if Server is missing, check that the daemon is reachable.

Why it matters: Version output tells you which features and flags your installation supports.

Exercise 002 · Check client/server versions

Task: Compare the Docker CLI and Engine versions for lab case 2.

Solution command

```
docker version
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker version`
4. Interpret the result: Read the Client and Server sections separately; if Server is missing, check that the daemon is reachable.

Why it matters: Version output tells you which features and flags your installation supports.

Exercise 003 · Check client/server versions

Task: Compare the Docker CLI and Engine versions for lab case 3.

Solution command

```
docker version
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker version`
4. Interpret the result: Read the Client and Server sections separately; if Server is missing, check that the daemon is reachable.

Why it matters: Version output tells you which features and flags your installation supports.

Exercise 004 · Check client/server versions

Task: Compare the Docker CLI and Engine versions for lab case 4.

Solution command

```
docker version
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker version`
4. Interpret the result: Read the Client and Server sections separately; if Server is missing, check that the daemon is reachable.

Why it matters: Version output tells you which features and flags your installation supports.

Exercise 005 · Check client/server versions

Task: Compare the Docker CLI and Engine versions for lab case 5.

Solution command

```
docker version
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker version`
4. Interpret the result: Read the Client and Server sections separately; if Server is missing, check that the daemon is reachable.

Why it matters: Version output tells you which features and flags your installation supports.

Exercise 006 · Check client/server versions

Task: Compare the Docker CLI and Engine versions for lab case 6.

Solution command

```
docker version
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker version`
4. Interpret the result: Read the Client and Server sections separately; if Server is missing, check that the daemon is reachable.

Why it matters: Version output tells you which features and flags your installation supports.

Exercise 007 · Check client/server versions

Task: Compare the Docker CLI and Engine versions for lab case 7.

Solution command

```
docker version
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker version`
4. Interpret the result: Read the Client and Server sections separately; if Server is missing, check that the daemon is reachable.

Why it matters: Version output tells you which features and flags your installation supports.

Exercise 008 · Check client/server versions

Task: Compare the Docker CLI and Engine versions for lab case 8.

Solution command

```
docker version
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker version`
4. Interpret the result: Read the Client and Server sections separately; if Server is missing, check that the daemon is reachable.

Why it matters: Version output tells you which features and flags your installation supports.

Exercise 009 · Check client/server versions

Task: Compare the Docker CLI and Engine versions for lab case 9.

Solution command

```
docker version
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker version`
4. Interpret the result: Read the Client and Server sections separately; if Server is missing, check that the daemon is reachable.

Why it matters: Version output tells you which features and flags your installation supports.

Exercise 010 · Check client/server versions

Task: Compare the Docker CLI and Engine versions for lab case 10.

Solution command

```
docker version
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker version`
4. Interpret the result: Read the Client and Server sections separately; if Server is missing, check that the daemon is reachable.

Why it matters: Version output tells you which features and flags your installation supports.

Exercise 011 · Inspect Engine configuration

Task: Display daemon details before starting lab 1.

Solution command

```
docker info
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Review server version, storage driver, runtimes, operating system, and security options.

Why it matters: Use this before diagnosing daemon, storage, or runtime behavior.

Exercise 012 · Inspect Engine configuration

Task: Display daemon details before starting lab 2.

Solution command

```
docker info
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Review server version, storage driver, runtimes, operating system, and security options.

Why it matters: Use this before diagnosing daemon, storage, or runtime behavior.

Exercise 013 · Inspect Engine configuration

Task: Display daemon details before starting lab 3.

Solution command

```
docker info
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Review server version, storage driver, runtimes, operating system, and security options.

Why it matters: Use this before diagnosing daemon, storage, or runtime behavior.

Exercise 014 · Inspect Engine configuration

Task: Display daemon details before starting lab 4.

Solution command

```
docker info
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Review server version, storage driver, runtimes, operating system, and security options.

Why it matters: Use this before diagnosing daemon, storage, or runtime behavior.

Exercise 015 · Inspect Engine configuration

Task: Display daemon details before starting lab 5.

Solution command

```
docker info
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Review server version, storage driver, runtimes, operating system, and security options.

Why it matters: Use this before diagnosing daemon, storage, or runtime behavior.

Exercise 016 · Inspect Engine configuration

Task: Display daemon details before starting lab 6.

Solution command

```
docker info
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Review server version, storage driver, runtimes, operating system, and security options.

Why it matters: Use this before diagnosing daemon, storage, or runtime behavior.

Exercise 017 · Inspect Engine configuration

Task: Display daemon details before starting lab 7.

Solution command

```
docker info
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Review server version, storage driver, runtimes, operating system, and security options.

Why it matters: Use this before diagnosing daemon, storage, or runtime behavior.

Exercise 018 · Inspect Engine configuration

Task: Display daemon details before starting lab 8.

Solution command

```
docker info
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Review server version, storage driver, runtimes, operating system, and security options.

Why it matters: Use this before diagnosing daemon, storage, or runtime behavior.

Exercise 019 · Inspect Engine configuration

Task: Display daemon details before starting lab 9.

Solution command

```
docker info
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Review server version, storage driver, runtimes, operating system, and security options.

Why it matters: Use this before diagnosing daemon, storage, or runtime behavior.

Exercise 020 · Inspect Engine configuration

Task: Display daemon details before starting lab 10.

Solution command

```
docker info
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Review server version, storage driver, runtimes, operating system, and security options.

Why it matters: Use this before diagnosing daemon, storage, or runtime behavior.

Module 02: System diagnostics & events

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 021 · Measure Docker disk usage

Task: Audit images, containers, volumes, and build cache before lab 1.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Run the verbose form to see individual objects; identify what is reclaimable before deleting anything.

Why it matters: This is diagnostic only; it does not remove data.

Exercise 022 · Measure Docker disk usage

Task: Audit images, containers, volumes, and build cache before lab 2.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Run the verbose form to see individual objects; identify what is reclaimable before deleting anything.

Why it matters: This is diagnostic only; it does not remove data.

Exercise 023 · Measure Docker disk usage

Task: Audit images, containers, volumes, and build cache before lab 3.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Run the verbose form to see individual objects; identify what is reclaimable before deleting anything.

Why it matters: This is diagnostic only; it does not remove data.

Exercise 024 · Measure Docker disk usage

Task: Audit images, containers, volumes, and build cache before lab 4.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Run the verbose form to see individual objects; identify what is reclaimable before deleting anything.

Why it matters: This is diagnostic only; it does not remove data.

Exercise 025 · Measure Docker disk usage

Task: Audit images, containers, volumes, and build cache before lab 5.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Run the verbose form to see individual objects; identify what is reclaimable before deleting anything.

Why it matters: This is diagnostic only; it does not remove data.

Exercise 026 · Measure Docker disk usage

Task: Audit images, containers, volumes, and build cache before lab 6.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Run the verbose form to see individual objects; identify what is reclaimable before deleting anything.

Why it matters: This is diagnostic only; it does not remove data.

Exercise 027 · Measure Docker disk usage

Task: Audit images, containers, volumes, and build cache before lab 7.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Run the verbose form to see individual objects; identify what is reclaimable before deleting anything.

Why it matters: This is diagnostic only; it does not remove data.

Exercise 028 · Measure Docker disk usage

Task: Audit images, containers, volumes, and build cache before lab 8.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Run the verbose form to see individual objects; identify what is reclaimable before deleting anything.

Why it matters: This is diagnostic only; it does not remove data.

Exercise 029 · Measure Docker disk usage

Task: Audit images, containers, volumes, and build cache before lab 9.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Run the verbose form to see individual objects; identify what is reclaimable before deleting anything.

Why it matters: This is diagnostic only; it does not remove data.

Exercise 030 · Measure Docker disk usage

Task: Audit images, containers, volumes, and build cache before lab 10.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Run the verbose form to see individual objects; identify what is reclaimable before deleting anything.

Why it matters: This is diagnostic only; it does not remove data.

Exercise 031 · Watch recent daemon events

Task: Review recent Docker events while testing service 1.

Solution command

```
docker events --since 10m
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker events --since 10m`
4. Interpret the result: Run in a terminal while starting or stopping a test container in another terminal.

Why it matters: Events help correlate lifecycle changes with failures.

Exercise 032 · Watch recent daemon events

Task: Review recent Docker events while testing service 2.

Solution command

```
docker events --since 10m
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker events --since 10m`
4. Interpret the result: Run in a terminal while starting or stopping a test container in another terminal.

Why it matters: Events help correlate lifecycle changes with failures.

Exercise 033 · Watch recent daemon events

Task: Review recent Docker events while testing service 3.

Solution command

```
docker events --since 10m
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker events --since 10m`
4. Interpret the result: Run in a terminal while starting or stopping a test container in another terminal.

Why it matters: Events help correlate lifecycle changes with failures.

Exercise 034 · Watch recent daemon events

Task: Review recent Docker events while testing service 4.

Solution command

```
docker events --since 10m
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker events --since 10m`
4. Interpret the result: Run in a terminal while starting or stopping a test container in another terminal.

Why it matters: Events help correlate lifecycle changes with failures.

Exercise 035 · Watch recent daemon events

Task: Review recent Docker events while testing service 5.

Solution command

```
docker events --since 10m
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker events --since 10m`
4. Interpret the result: Run in a terminal while starting or stopping a test container in another terminal.

Why it matters: Events help correlate lifecycle changes with failures.

Exercise 036 · Watch recent daemon events

Task: Review recent Docker events while testing service 6.

Solution command

```
docker events --since 10m
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker events --since 10m`
4. Interpret the result: Run in a terminal while starting or stopping a test container in another terminal.

Why it matters: Events help correlate lifecycle changes with failures.

Exercise 037 · Watch recent daemon events

Task: Review recent Docker events while testing service 7.

Solution command

```
docker events --since 10m
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker events --since 10m`
4. Interpret the result: Run in a terminal while starting or stopping a test container in another terminal.

Why it matters: Events help correlate lifecycle changes with failures.

Exercise 038 · Watch recent daemon events

Task: Review recent Docker events while testing service 8.

Solution command

```
docker events --since 10m
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker events --since 10m`
4. Interpret the result: Run in a terminal while starting or stopping a test container in another terminal.

Why it matters: Events help correlate lifecycle changes with failures.

Exercise 039 · Watch recent daemon events

Task: Review recent Docker events while testing service 9.

Solution command

```
docker events --since 10m
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker events --since 10m`
4. Interpret the result: Run in a terminal while starting or stopping a test container in another terminal.

Why it matters: Events help correlate lifecycle changes with failures.

Exercise 040 · Watch recent daemon events

Task: Review recent Docker events while testing service 10.

Solution command

```
docker events --since 10m
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker events --since 10m`
4. Interpret the result: Run in a terminal while starting or stopping a test container in another terminal.

Why it matters: Events help correlate lifecycle changes with failures.

Module 03: Images: pulling, listing & filtering

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 041 · Pull an image

Task: Download the Alpine image needed for exercise 1.

Solution command

```
docker pull alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls alpine`
4. Interpret the result: Pull the tag, then confirm it appears in the local image list.

Why it matters: A tag is a human-readable reference; production workflows often pin digests.

Exercise 042 · Pull an image

Task: Download the Alpine image needed for exercise 2.

Solution command

```
docker pull alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls alpine`
4. Interpret the result: Pull the tag, then confirm it appears in the local image list.

Why it matters: A tag is a human-readable reference; production workflows often pin digests.

Exercise 043 · Pull an image

Task: Download the Alpine image needed for exercise 3.

Solution command

```
docker pull alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls alpine`
4. Interpret the result: Pull the tag, then confirm it appears in the local image list.

Why it matters: A tag is a human-readable reference; production workflows often pin digests.

Exercise 044 · Pull an image

Task: Download the Alpine image needed for exercise 4.

Solution command

```
docker pull alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls alpine`
4. Interpret the result: Pull the tag, then confirm it appears in the local image list.

Why it matters: A tag is a human-readable reference; production workflows often pin digests.

Exercise 045 · Pull an image

Task: Download the Alpine image needed for exercise 5.

Solution command

```
docker pull alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls alpine`
4. Interpret the result: Pull the tag, then confirm it appears in the local image list.

Why it matters: A tag is a human-readable reference; production workflows often pin digests.

Exercise 046 · Pull an image

Task: Download the Alpine image needed for exercise 6.

Solution command

```
docker pull alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls alpine`
4. Interpret the result: Pull the tag, then confirm it appears in the local image list.

Why it matters: A tag is a human-readable reference; production workflows often pin digests.

Exercise 047 · Pull an image

Task: Download the Alpine image needed for exercise 7.

Solution command

```
docker pull alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls alpine`
4. Interpret the result: Pull the tag, then confirm it appears in the local image list.

Why it matters: A tag is a human-readable reference; production workflows often pin digests.

Exercise 048 · Pull an image

Task: Download the Alpine image needed for exercise 8.

Solution command

```
docker pull alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls alpine`
4. Interpret the result: Pull the tag, then confirm it appears in the local image list.

Why it matters: A tag is a human-readable reference; production workflows often pin digests.

Exercise 049 · Pull an image

Task: Download the Alpine image needed for exercise 9.

Solution command

```
docker pull alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls alpine`
4. Interpret the result: Pull the tag, then confirm it appears in the local image list.

Why it matters: A tag is a human-readable reference; production workflows often pin digests.

Exercise 050 · Pull an image

Task: Download the Alpine image needed for exercise 10.

Solution command

```
docker pull alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls alpine`
4. Interpret the result: Pull the tag, then confirm it appears in the local image list.

Why it matters: A tag is a human-readable reference; production workflows often pin digests.

Exercise 051 · Filter local images

Task: List local Alpine image references for inventory case 1.

Solution command

```
docker image ls --filter reference='alpine*'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter reference='alpine*'`
4. Interpret the result: Compare repository, tag, image ID, creation time, and size.

Why it matters: Filters narrow the result set without modifying images.

Exercise 052 · Filter local images

Task: List local Alpine image references for inventory case 2.

Solution command

```
docker image ls --filter reference='alpine*'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter reference='alpine*'`
4. Interpret the result: Compare repository, tag, image ID, creation time, and size.

Why it matters: Filters narrow the result set without modifying images.

Exercise 053 · Filter local images

Task: List local Alpine image references for inventory case 3.

Solution command

```
docker image ls --filter reference='alpine*'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter reference='alpine*'`
4. Interpret the result: Compare repository, tag, image ID, creation time, and size.

Why it matters: Filters narrow the result set without modifying images.

Exercise 054 · Filter local images

Task: List local Alpine image references for inventory case 4.

Solution command

```
docker image ls --filter reference='alpine*'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter reference='alpine*'`
4. Interpret the result: Compare repository, tag, image ID, creation time, and size.

Why it matters: Filters narrow the result set without modifying images.

Exercise 055 · Filter local images

Task: List local Alpine image references for inventory case 5.

Solution command

```
docker image ls --filter reference='alpine*'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter reference='alpine*'`
4. Interpret the result: Compare repository, tag, image ID, creation time, and size.

Why it matters: Filters narrow the result set without modifying images.

Exercise 056 · Filter local images

Task: List local Alpine image references for inventory case 6.

Solution command

```
docker image ls --filter reference='alpine*'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter reference='alpine*'`
4. Interpret the result: Compare repository, tag, image ID, creation time, and size.

Why it matters: Filters narrow the result set without modifying images.

Exercise 057 · Filter local images

Task: List local Alpine image references for inventory case 7.

Solution command

```
docker image ls --filter reference='alpine*'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter reference='alpine*'`
4. Interpret the result: Compare repository, tag, image ID, creation time, and size.

Why it matters: Filters narrow the result set without modifying images.

Exercise 058 · Filter local images

Task: List local Alpine image references for inventory case 8.

Solution command

```
docker image ls --filter reference='alpine*'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter reference='alpine*'`
4. Interpret the result: Compare repository, tag, image ID, creation time, and size.

Why it matters: Filters narrow the result set without modifying images.

Exercise 059 · Filter local images

Task: List local Alpine image references for inventory case 9.

Solution command

```
docker image ls --filter reference='alpine*'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter reference='alpine*'`
4. Interpret the result: Compare repository, tag, image ID, creation time, and size.

Why it matters: Filters narrow the result set without modifying images.

Exercise 060 · Filter local images

Task: List local Alpine image references for inventory case 10.

Solution command

```
docker image ls --filter reference='alpine*'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter reference='alpine*'`
4. Interpret the result: Compare repository, tag, image ID, creation time, and size.

Why it matters: Filters narrow the result set without modifying images.

Module 04: Image metadata & layers

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 061 · Inspect image metadata

Task: Find the default command, entrypoint, environment, and architecture for Alpine in case 1.

Solution command

```
docker image inspect alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect --format '{.Os}/{.Architecture} {json .Config.Cmd} {json .Config.Entrypoint}' alpine:3.20`
4. Interpret the result: Inspect the JSON fields or use the format expression to print only the fields you need.

Why it matters: Image metadata describes defaults; container runtime options can override some of them.

Exercise 062 · Inspect image metadata

Task: Find the default command, entrypoint, environment, and architecture for Alpine in case 2.

Solution command

```
docker image inspect alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect --format '{.Os}/{.Architecture} {json .Config.Cmd} {json .Config.Entrypoint}' alpine:3.20`
4. Interpret the result: Inspect the JSON fields or use the format expression to print only the fields you need.

Why it matters: Image metadata describes defaults; container runtime options can override some of them.

Exercise 063 · Inspect image metadata

Task: Find the default command, entrypoint, environment, and architecture for Alpine in case 3.

Solution command

```
docker image inspect alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect --format '{.Os}/{.Architecture} {json .Config.Cmd} {json .Config.Entrypoint}' alpine:3.20`
4. Interpret the result: Inspect the JSON fields or use the format expression to print only the fields you need.

Why it matters: Image metadata describes defaults; container runtime options can override some of them.

Exercise 064 · Inspect image metadata

Task: Find the default command, entrypoint, environment, and architecture for Alpine in case 4.

Solution command

```
docker image inspect alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect --format '{.Os}/{.Architecture} {json .Config.Cmd} {json .Config.Entrypoint}' alpine:3.20`
4. Interpret the result: Inspect the JSON fields or use the format expression to print only the fields you need.

Why it matters: Image metadata describes defaults; container runtime options can override some of them.

Exercise 065 · Inspect image metadata

Task: Find the default command, entrypoint, environment, and architecture for Alpine in case 5.

Solution command

```
docker image inspect alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect --format '{.Os}/{.Architecture} {json .Config.Cmd} {json .Config.Entrypoint}' alpine:3.20`
4. Interpret the result: Inspect the JSON fields or use the format expression to print only the fields you need.

Why it matters: Image metadata describes defaults; container runtime options can override some of them.

Exercise 066 · Inspect image metadata

Task: Find the default command, entrypoint, environment, and architecture for Alpine in case 6.

Solution command

```
docker image inspect alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect --format '{.Os}/{.Architecture} {json .Config.Cmd} {json .Config.Entrypoint}' alpine:3.20`
4. Interpret the result: Inspect the JSON fields or use the format expression to print only the fields you need.

Why it matters: Image metadata describes defaults; container runtime options can override some of them.

Exercise 067 · Inspect image metadata

Task: Find the default command, entrypoint, environment, and architecture for Alpine in case 7.

Solution command

```
docker image inspect alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect --format '{.Os}/{.Architecture} {json .Config.Cmd} {json .Config.Entrypoint}' alpine:3.20`
4. Interpret the result: Inspect the JSON fields or use the format expression to print only the fields you need.

Why it matters: Image metadata describes defaults; container runtime options can override some of them.

Exercise 068 · Inspect image metadata

Task: Find the default command, entrypoint, environment, and architecture for Alpine in case 8.

Solution command

```
docker image inspect alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect --format '{.Os}/{.Architecture} {json .Config.Cmd} {json .Config.Entrypoint}' alpine:3.20`
4. Interpret the result: Inspect the JSON fields or use the format expression to print only the fields you need.

Why it matters: Image metadata describes defaults; container runtime options can override some of them.

Exercise 069 · Inspect image metadata

Task: Find the default command, entrypoint, environment, and architecture for Alpine in case 9.

Solution command

```
docker image inspect alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect --format '{.Os}/{.Architecture} {json .Config.Cmd} {json .Config.Entrypoint}' alpine:3.20`
4. Interpret the result: Inspect the JSON fields or use the format expression to print only the fields you need.

Why it matters: Image metadata describes defaults; container runtime options can override some of them.

Exercise 070 · Inspect image metadata

Task: Find the default command, entrypoint, environment, and architecture for Alpine in case 10.

Solution command

```
docker image inspect alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect --format '{.Os}/{.Architecture} {json .Config.Cmd} {json .Config.Entrypoint}' alpine:3.20`
4. Interpret the result: Inspect the JSON fields or use the format expression to print only the fields you need.

Why it matters: Image metadata describes defaults; container runtime options can override some of them.

Exercise 071 · Read image layer history

Task: Review the layer history for the image audit numbered 1.

Solution command

```
docker image history alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image history --no-trunc alpine:3.20`
4. Interpret the result: Read the CREATED BY column and compare layer sizes; remember that history is not a complete source listing.

Why it matters: Layer history helps explain image construction and size.

Exercise 072 · Read image layer history

Task: Review the layer history for the image audit numbered 2.

Solution command

```
docker image history alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image history --no-trunc alpine:3.20`
4. Interpret the result: Read the CREATED BY column and compare layer sizes; remember that history is not a complete source listing.

Why it matters: Layer history helps explain image construction and size.

Exercise 073 · Read image layer history

Task: Review the layer history for the image audit numbered 3.

Solution command

```
docker image history alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image history --no-trunc alpine:3.20`
4. Interpret the result: Read the CREATED BY column and compare layer sizes; remember that history is not a complete source listing.

Why it matters: Layer history helps explain image construction and size.

Exercise 074 · Read image layer history

Task: Review the layer history for the image audit numbered 4.

Solution command

```
docker image history alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image history --no-trunc alpine:3.20`
4. Interpret the result: Read the CREATED BY column and compare layer sizes; remember that history is not a complete source listing.

Why it matters: Layer history helps explain image construction and size.

Exercise 075 · Read image layer history

Task: Review the layer history for the image audit numbered 5.

Solution command

```
docker image history alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image history --no-trunc alpine:3.20`
4. Interpret the result: Read the CREATED BY column and compare layer sizes; remember that history is not a complete source listing.

Why it matters: Layer history helps explain image construction and size.

Exercise 076 · Read image layer history

Task: Review the layer history for the image audit numbered 6.

Solution command

```
docker image history alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image history --no-trunc alpine:3.20`
4. Interpret the result: Read the CREATED BY column and compare layer sizes; remember that history is not a complete source listing.

Why it matters: Layer history helps explain image construction and size.

Exercise 077 · Read image layer history

Task: Review the layer history for the image audit numbered 7.

Solution command

```
docker image history alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image history --no-trunc alpine:3.20`
4. Interpret the result: Read the CREATED BY column and compare layer sizes; remember that history is not a complete source listing.

Why it matters: Layer history helps explain image construction and size.

Exercise 078 · Read image layer history

Task: Review the layer history for the image audit numbered 8.

Solution command

```
docker image history alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image history --no-trunc alpine:3.20`
4. Interpret the result: Read the CREATED BY column and compare layer sizes; remember that history is not a complete source listing.

Why it matters: Layer history helps explain image construction and size.

Exercise 079 · Read image layer history

Task: Review the layer history for the image audit numbered 9.

Solution command

```
docker image history alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image history --no-trunc alpine:3.20`
4. Interpret the result: Read the CREATED BY column and compare layer sizes; remember that history is not a complete source listing.

Why it matters: Layer history helps explain image construction and size.

Exercise 080 · Read image layer history

Task: Review the layer history for the image audit numbered 10.

Solution command

```
docker image history alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image history --no-trunc alpine:3.20`
4. Interpret the result: Read the CREATED BY column and compare layer sizes; remember that history is not a complete source listing.

Why it matters: Layer history helps explain image construction and size.

Module 05: Tagging & image archives

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 081 · Create a local tag

Task: Tag Alpine as local/alpine-lab:case1.

Solution command

```
docker image tag alpine:3.20 local/alpine-lab:case1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Tagging adds another reference to the same image content; it does not copy the layers.

Why it matters: Use a registry-qualified name when preparing to push to a registry.

Exercise 082 · Create a local tag

Task: Tag Alpine as local/alpine-lab:case2.

Solution command

```
docker image tag alpine:3.20 local/alpine-lab:case2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Tagging adds another reference to the same image content; it does not copy the layers.

Why it matters: Use a registry-qualified name when preparing to push to a registry.

Exercise 083 · Create a local tag

Task: Tag Alpine as local/alpine-lab:case3.

Solution command

```
docker image tag alpine:3.20 local/alpine-lab:case3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Tagging adds another reference to the same image content; it does not copy the layers.

Why it matters: Use a registry-qualified name when preparing to push to a registry.

Exercise 084 · Create a local tag

Task: Tag Alpine as local/alpine-lab:case4.

Solution command

```
docker image tag alpine:3.20 local/alpine-lab:case4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Tagging adds another reference to the same image content; it does not copy the layers.

Why it matters: Use a registry-qualified name when preparing to push to a registry.

Exercise 085 · Create a local tag

Task: Tag Alpine as local/alpine-lab:case5.

Solution command

```
docker image tag alpine:3.20 local/alpine-lab:case5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Tagging adds another reference to the same image content; it does not copy the layers.

Why it matters: Use a registry-qualified name when preparing to push to a registry.

Exercise 086 · Create a local tag

Task: Tag Alpine as local/alpine-lab:case6.

Solution command

```
docker image tag alpine:3.20 local/alpine-lab:case6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Tagging adds another reference to the same image content; it does not copy the layers.

Why it matters: Use a registry-qualified name when preparing to push to a registry.

Exercise 087 · Create a local tag

Task: Tag Alpine as local/alpine-lab:case7.

Solution command

```
docker image tag alpine:3.20 local/alpine-lab:case7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Tagging adds another reference to the same image content; it does not copy the layers.

Why it matters: Use a registry-qualified name when preparing to push to a registry.

Exercise 088 · Create a local tag

Task: Tag Alpine as local/alpine-lab:case8.

Solution command

```
docker image tag alpine:3.20 local/alpine-lab:case8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Tagging adds another reference to the same image content; it does not copy the layers.

Why it matters: Use a registry-qualified name when preparing to push to a registry.

Exercise 089 · Create a local tag

Task: Tag Alpine as local/alpine-lab:case9.

Solution command

```
docker image tag alpine:3.20 local/alpine-lab:case9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Tagging adds another reference to the same image content; it does not copy the layers.

Why it matters: Use a registry-qualified name when preparing to push to a registry.

Exercise 090 · Create a local tag

Task: Tag Alpine as local/alpine-lab:case10.

Solution command

```
docker image tag alpine:3.20 local/alpine-lab:case10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Tagging adds another reference to the same image content; it does not copy the layers.

Why it matters: Use a registry-qualified name when preparing to push to a registry.

Exercise 091 · Export an image archive

Task: Save Alpine to alpine-case1.tar for transfer to an offline lab.

Solution command

```
docker image save -o alpine-case1.tar alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh alpine-case1.tar`
4. Interpret the result: Check that the archive exists and has a plausible size.

Why it matters: Save exports image content and tags; it is different from exporting a container filesystem.

Exercise 092 · Export an image archive

Task: Save Alpine to alpine-case2.tar for transfer to an offline lab.

Solution command

```
docker image save -o alpine-case2.tar alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh alpine-case2.tar`
4. Interpret the result: Check that the archive exists and has a plausible size.

Why it matters: Save exports image content and tags; it is different from exporting a container filesystem.

Exercise 093 · Export an image archive

Task: Save Alpine to alpine-case3.tar for transfer to an offline lab.

Solution command

```
docker image save -o alpine-case3.tar alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh alpine-case3.tar`
4. Interpret the result: Check that the archive exists and has a plausible size.

Why it matters: Save exports image content and tags; it is different from exporting a container filesystem.

Exercise 094 · Export an image archive

Task: Save Alpine to alpine-case4.tar for transfer to an offline lab.

Solution command

```
docker image save -o alpine-case4.tar alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh alpine-case4.tar`
4. Interpret the result: Check that the archive exists and has a plausible size.

Why it matters: Save exports image content and tags; it is different from exporting a container filesystem.

Exercise 095 · Export an image archive

Task: Save Alpine to alpine-case5.tar for transfer to an offline lab.

Solution command

```
docker image save -o alpine-case5.tar alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh alpine-case5.tar`
4. Interpret the result: Check that the archive exists and has a plausible size.

Why it matters: Save exports image content and tags; it is different from exporting a container filesystem.

Exercise 096 · Export an image archive

Task: Save Alpine to alpine-case6.tar for transfer to an offline lab.

Solution command

```
docker image save -o alpine-case6.tar alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh alpine-case6.tar`
4. Interpret the result: Check that the archive exists and has a plausible size.

Why it matters: Save exports image content and tags; it is different from exporting a container filesystem.

Exercise 097 · Export an image archive

Task: Save Alpine to alpine-case7.tar for transfer to an offline lab.

Solution command

```
docker image save -o alpine-case7.tar alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh alpine-case7.tar`
4. Interpret the result: Check that the archive exists and has a plausible size.

Why it matters: Save exports image content and tags; it is different from exporting a container filesystem.

Exercise 098 · Export an image archive

Task: Save Alpine to alpine-case8.tar for transfer to an offline lab.

Solution command

```
docker image save -o alpine-case8.tar alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh alpine-case8.tar`
4. Interpret the result: Check that the archive exists and has a plausible size.

Why it matters: Save exports image content and tags; it is different from exporting a container filesystem.

Exercise 099 · Export an image archive

Task: Save Alpine to alpine-case9.tar for transfer to an offline lab.

Solution command

```
docker image save -o alpine-case9.tar alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh alpine-case9.tar`
4. Interpret the result: Check that the archive exists and has a plausible size.

Why it matters: Save exports image content and tags; it is different from exporting a container filesystem.

Exercise 100 · Export an image archive

Task: Save Alpine to alpine-case10.tar for transfer to an offline lab.

Solution command

```
docker image save -o alpine-case10.tar alpine:3.20
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh alpine-case10.tar`
4. Interpret the result: Check that the archive exists and has a plausible size.

Why it matters: Save exports image content and tags; it is different from exporting a container filesystem.

Module 06: Registries & publishing

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 101 · Authenticate to a registry

Task: Prepare to publish image case1 to the registry configured for your organization.

Solution command

```
docker login
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Run login interactively and follow the registry's authentication flow; never put passwords directly in shell history.

Why it matters: For automation, prefer a credential helper or short-lived token.

Exercise 102 · Authenticate to a registry

Task: Prepare to publish image case2 to the registry configured for your organization.

Solution command

```
docker login
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Run login interactively and follow the registry's authentication flow; never put passwords directly in shell history.

Why it matters: For automation, prefer a credential helper or short-lived token.

Exercise 103 · Authenticate to a registry

Task: Prepare to publish image case3 to the registry configured for your organization.

Solution command

```
docker login
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Run login interactively and follow the registry's authentication flow; never put passwords directly in shell history.

Why it matters: For automation, prefer a credential helper or short-lived token.

Exercise 104 · Authenticate to a registry

Task: Prepare to publish image case4 to the registry configured for your organization.

Solution command

```
docker login
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Run login interactively and follow the registry's authentication flow; never put passwords directly in shell history.

Why it matters: For automation, prefer a credential helper or short-lived token.

Exercise 105 · Authenticate to a registry

Task: Prepare to publish image case5 to the registry configured for your organization.

Solution command

```
docker login
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Run login interactively and follow the registry's authentication flow; never put passwords directly in shell history.

Why it matters: For automation, prefer a credential helper or short-lived token.

Exercise 106 · Authenticate to a registry

Task: Prepare to publish image case6 to the registry configured for your organization.

Solution command

```
docker login
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Run login interactively and follow the registry's authentication flow; never put passwords directly in shell history.

Why it matters: For automation, prefer a credential helper or short-lived token.

Exercise 107 · Authenticate to a registry

Task: Prepare to publish image case7 to the registry configured for your organization.

Solution command

```
docker login
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Run login interactively and follow the registry's authentication flow; never put passwords directly in shell history.

Why it matters: For automation, prefer a credential helper or short-lived token.

Exercise 108 · Authenticate to a registry

Task: Prepare to publish image case8 to the registry configured for your organization.

Solution command

```
docker login
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Run login interactively and follow the registry's authentication flow; never put passwords directly in shell history.

Why it matters: For automation, prefer a credential helper or short-lived token.

Exercise 109 · Authenticate to a registry

Task: Prepare to publish image case9 to the registry configured for your organization.

Solution command

```
docker login
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Run login interactively and follow the registry's authentication flow; never put passwords directly in shell history.

Why it matters: For automation, prefer a credential helper or short-lived token.

Exercise 110 · Authenticate to a registry

Task: Prepare to publish image case10 to the registry configured for your organization.

Solution command

```
docker login
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info`
4. Interpret the result: Run login interactively and follow the registry's authentication flow; never put passwords directly in shell history.

Why it matters: For automation, prefer a credential helper or short-lived token.

Exercise 111 · Push a tagged image

Task: Publish local/example:case1 after tagging it to a registry you control.

Solution command

```
docker image push REGISTRY/USERNAME/example:case1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls REGISTRY/USERNAME/example`
4. Interpret the result: Replace REGISTRY/USERNAME with your own namespace, authenticate, then push the exact tag.

Why it matters: This template cannot succeed until you replace the registry and account placeholders.

Exercise 112 · Push a tagged image

Task: Publish local/example:case2 after tagging it to a registry you control.

Solution command

```
docker image push REGISTRY/USERNAME/example:case2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls REGISTRY/USERNAME/example`
4. Interpret the result: Replace REGISTRY/USERNAME with your own namespace, authenticate, then push the exact tag.

Why it matters: This template cannot succeed until you replace the registry and account placeholders.

Exercise 113 · Push a tagged image

Task: Publish local/example:case3 after tagging it to a registry you control.

Solution command

```
docker image push REGISTRY/USERNAME/example:case3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls REGISTRY/USERNAME/example`
4. Interpret the result: Replace REGISTRY/USERNAME with your own namespace, authenticate, then push the exact tag.

Why it matters: This template cannot succeed until you replace the registry and account placeholders.

Exercise 114 · Push a tagged image

Task: Publish local/example:case4 after tagging it to a registry you control.

Solution command

```
docker image push REGISTRY/USERNAME/example:case4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls REGISTRY/USERNAME/example`
4. Interpret the result: Replace REGISTRY/USERNAME with your own namespace, authenticate, then push the exact tag.

Why it matters: This template cannot succeed until you replace the registry and account placeholders.

Exercise 115 · Push a tagged image

Task: Publish local/example:case5 after tagging it to a registry you control.

Solution command

```
docker image push REGISTRY/USERNAME/example:case5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls REGISTRY/USERNAME/example`
4. Interpret the result: Replace REGISTRY/USERNAME with your own namespace, authenticate, then push the exact tag.

Why it matters: This template cannot succeed until you replace the registry and account placeholders.

Exercise 116 · Push a tagged image

Task: Publish local/example:case6 after tagging it to a registry you control.

Solution command

```
docker image push REGISTRY/USERNAME/example:case6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls REGISTRY/USERNAME/example`
4. Interpret the result: Replace REGISTRY/USERNAME with your own namespace, authenticate, then push the exact tag.

Why it matters: This template cannot succeed until you replace the registry and account placeholders.

Exercise 117 · Push a tagged image

Task: Publish local/example:case7 after tagging it to a registry you control.

Solution command

```
docker image push REGISTRY/USERNAME/example:case7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls REGISTRY/USERNAME/example`
4. Interpret the result: Replace REGISTRY/USERNAME with your own namespace, authenticate, then push the exact tag.

Why it matters: This template cannot succeed until you replace the registry and account placeholders.

Exercise 118 · Push a tagged image

Task: Publish local/example:case8 after tagging it to a registry you control.

Solution command

```
docker image push REGISTRY/USERNAME/example:case8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls REGISTRY/USERNAME/example`
4. Interpret the result: Replace REGISTRY/USERNAME with your own namespace, authenticate, then push the exact tag.

Why it matters: This template cannot succeed until you replace the registry and account placeholders.

Exercise 119 · Push a tagged image

Task: Publish local/example:case9 after tagging it to a registry you control.

Solution command

```
docker image push REGISTRY/USERNAME/example:case9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls REGISTRY/USERNAME/example`
4. Interpret the result: Replace REGISTRY/USERNAME with your own namespace, authenticate, then push the exact tag.

Why it matters: This template cannot succeed until you replace the registry and account placeholders.

Exercise 120 · Push a tagged image

Task: Publish local/example:case10 after tagging it to a registry you control.

Solution command

```
docker image push REGISTRY/USERNAME/example:case10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls REGISTRY/USERNAME/example`
4. Interpret the result: Replace REGISTRY/USERNAME with your own namespace, authenticate, then push the exact tag.

Why it matters: This template cannot succeed until you replace the registry and account placeholders.

Module 07: Container launch & naming

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 121 · Run a one-shot container

Task: Print a message from a disposable Alpine container for case 1.

Solution command

```
docker run --rm alpine:3.20 echo 'hello from case 1'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter ancestor=alpine:3.20`
4. Interpret the result: Run the command and observe the output; --rm removes the stopped container automatically.

Why it matters: Use --rm for disposable tasks where you do not need the stopped container record.

Exercise 122 · Run a one-shot container

Task: Print a message from a disposable Alpine container for case 2.

Solution command

```
docker run --rm alpine:3.20 echo 'hello from case 2'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter ancestor=alpine:3.20`
4. Interpret the result: Run the command and observe the output; --rm removes the stopped container automatically.

Why it matters: Use --rm for disposable tasks where you do not need the stopped container record.

Exercise 123 · Run a one-shot container

Task: Print a message from a disposable Alpine container for case 3.

Solution command

```
docker run --rm alpine:3.20 echo 'hello from case 3'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter ancestor=alpine:3.20`
4. Interpret the result: Run the command and observe the output; --rm removes the stopped container automatically.

Why it matters: Use --rm for disposable tasks where you do not need the stopped container record.

Exercise 124 · Run a one-shot container

Task: Print a message from a disposable Alpine container for case 4.

Solution command

```
docker run --rm alpine:3.20 echo 'hello from case 4'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter ancestor=alpine:3.20`
4. Interpret the result: Run the command and observe the output; --rm removes the stopped container automatically.

Why it matters: Use --rm for disposable tasks where you do not need the stopped container record.

Exercise 125 · Run a one-shot container

Task: Print a message from a disposable Alpine container for case 5.

Solution command

```
docker run --rm alpine:3.20 echo 'hello from case 5'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter ancestor=alpine:3.20`
4. Interpret the result: Run the command and observe the output; --rm removes the stopped container automatically.

Why it matters: Use --rm for disposable tasks where you do not need the stopped container record.

Exercise 126 · Run a one-shot container

Task: Print a message from a disposable Alpine container for case 6.

Solution command

```
docker run --rm alpine:3.20 echo 'hello from case 6'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter ancestor=alpine:3.20`
4. Interpret the result: Run the command and observe the output; --rm removes the stopped container automatically.

Why it matters: Use --rm for disposable tasks where you do not need the stopped container record.

Exercise 127 · Run a one-shot container

Task: Print a message from a disposable Alpine container for case 7.

Solution command

```
docker run --rm alpine:3.20 echo 'hello from case 7'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter ancestor=alpine:3.20`
4. Interpret the result: Run the command and observe the output; --rm removes the stopped container automatically.

Why it matters: Use --rm for disposable tasks where you do not need the stopped container record.

Exercise 128 · Run a one-shot container

Task: Print a message from a disposable Alpine container for case 8.

Solution command

```
docker run --rm alpine:3.20 echo 'hello from case 8'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter ancestor=alpine:3.20`
4. Interpret the result: Run the command and observe the output; --rm removes the stopped container automatically.

Why it matters: Use --rm for disposable tasks where you do not need the stopped container record.

Exercise 129 · Run a one-shot container

Task: Print a message from a disposable Alpine container for case 9.

Solution command

```
docker run --rm alpine:3.20 echo 'hello from case 9'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter ancestor=alpine:3.20`
4. Interpret the result: Run the command and observe the output; --rm removes the stopped container automatically.

Why it matters: Use --rm for disposable tasks where you do not need the stopped container record.

Exercise 130 · Run a one-shot container

Task: Print a message from a disposable Alpine container for case 10.

Solution command

```
docker run --rm alpine:3.20 echo 'hello from case 10'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter ancestor=alpine:3.20`
4. Interpret the result: Run the command and observe the output; --rm removes the stopped container automatically.

Why it matters: Use --rm for disposable tasks where you do not need the stopped container record.

Exercise 131 · Run a named web server

Task: Start a named Nginx container on host port 8081.

Solution command

```
docker run -d --name web1 -p 8081:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web1`
4. Interpret the result: Confirm the container is running and the host port mapping is shown.

Why it matters: Use a free host port; on completion remove only the lab container you created.

Exercise 132 · Run a named web server

Task: Start a named Nginx container on host port 8082.

Solution command

```
docker run -d --name web2 -p 8082:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web2`
4. Interpret the result: Confirm the container is running and the host port mapping is shown.

Why it matters: Use a free host port; on completion remove only the lab container you created.

Exercise 133 · Run a named web server

Task: Start a named Nginx container on host port 8083.

Solution command

```
docker run -d --name web3 -p 8083:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web3`
4. Interpret the result: Confirm the container is running and the host port mapping is shown.

Why it matters: Use a free host port; on completion remove only the lab container you created.

Exercise 134 · Run a named web server

Task: Start a named Nginx container on host port 8084.

Solution command

```
docker run -d --name web4 -p 8084:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web4`
4. Interpret the result: Confirm the container is running and the host port mapping is shown.

Why it matters: Use a free host port; on completion remove only the lab container you created.

Exercise 135 · Run a named web server

Task: Start a named Nginx container on host port 8085.

Solution command

```
docker run -d --name web5 -p 8085:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web5`
4. Interpret the result: Confirm the container is running and the host port mapping is shown.

Why it matters: Use a free host port; on completion remove only the lab container you created.

Exercise 136 · Run a named web server

Task: Start a named Nginx container on host port 8086.

Solution command

```
docker run -d --name web6 -p 8086:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web6`
4. Interpret the result: Confirm the container is running and the host port mapping is shown.

Why it matters: Use a free host port; on completion remove only the lab container you created.

Exercise 137 · Run a named web server

Task: Start a named Nginx container on host port 8087.

Solution command

```
docker run -d --name web7 -p 8087:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web7`
4. Interpret the result: Confirm the container is running and the host port mapping is shown.

Why it matters: Use a free host port; on completion remove only the lab container you created.

Exercise 138 · Run a named web server

Task: Start a named Nginx container on host port 8088.

Solution command

```
docker run -d --name web8 -p 8088:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web8`
4. Interpret the result: Confirm the container is running and the host port mapping is shown.

Why it matters: Use a free host port; on completion remove only the lab container you created.

Exercise 139 · Run a named web server

Task: Start a named Nginx container on host port 8089.

Solution command

```
docker run -d --name web9 -p 8089:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web9`
4. Interpret the result: Confirm the container is running and the host port mapping is shown.

Why it matters: Use a free host port; on completion remove only the lab container you created.

Exercise 140 · Run a named web server

Task: Start a named Nginx container on host port 8090.

Solution command

```
docker run -d --name web10 -p 8090:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web10`
4. Interpret the result: Confirm the container is running and the host port mapping is shown.

Why it matters: Use a free host port; on completion remove only the lab container you created.

Module 08: Container runtime options & isolation

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 141 · Pass an environment variable

Task: Run Alpine with CASE_ID set to case1 and print it.

Solution command

```
docker run --rm -e CASE_ID=case1 alpine:3.20 sh -c 'echo $CASE_ID'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -e CASE_ID=case1 alpine:3.20 sh -c 'echo $CASE_ID'`
4. Interpret the result: The -e flag injects an environment variable into the container process.

Why it matters: Avoid passing secrets as ordinary environment variables in shared logs or command history.

Exercise 142 · Pass an environment variable

Task: Run Alpine with CASE_ID set to case2 and print it.

Solution command

```
docker run --rm -e CASE_ID=case2 alpine:3.20 sh -c 'echo $CASE_ID'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -e CASE_ID=case2 alpine:3.20 sh -c 'echo $CASE_ID'`
4. Interpret the result: The -e flag injects an environment variable into the container process.

Why it matters: Avoid passing secrets as ordinary environment variables in shared logs or command history.

Exercise 143 · Pass an environment variable

Task: Run Alpine with CASE_ID set to case3 and print it.

Solution command

```
docker run --rm -e CASE_ID=case3 alpine:3.20 sh -c 'echo $CASE_ID'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -e CASE_ID=case3 alpine:3.20 sh -c 'echo $CASE_ID'`
4. Interpret the result: The -e flag injects an environment variable into the container process.

Why it matters: Avoid passing secrets as ordinary environment variables in shared logs or command history.

Exercise 144 · Pass an environment variable

Task: Run Alpine with CASE_ID set to case4 and print it.

Solution command

```
docker run --rm -e CASE_ID=case4 alpine:3.20 sh -c 'echo $CASE_ID'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -e CASE_ID=case4 alpine:3.20 sh -c 'echo $CASE_ID'`
4. Interpret the result: The -e flag injects an environment variable into the container process.

Why it matters: Avoid passing secrets as ordinary environment variables in shared logs or command history.

Exercise 145 · Pass an environment variable

Task: Run Alpine with CASE_ID set to case5 and print it.

Solution command

```
docker run --rm -e CASE_ID=case5 alpine:3.20 sh -c 'echo $CASE_ID'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -e CASE_ID=case5 alpine:3.20 sh -c 'echo $CASE_ID'`
4. Interpret the result: The -e flag injects an environment variable into the container process.

Why it matters: Avoid passing secrets as ordinary environment variables in shared logs or command history.

Exercise 146 · Pass an environment variable

Task: Run Alpine with CASE_ID set to case6 and print it.

Solution command

```
docker run --rm -e CASE_ID=case6 alpine:3.20 sh -c 'echo $CASE_ID'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -e CASE_ID=case6 alpine:3.20 sh -c 'echo $CASE_ID'`
4. Interpret the result: The -e flag injects an environment variable into the container process.

Why it matters: Avoid passing secrets as ordinary environment variables in shared logs or command history.

Exercise 147 · Pass an environment variable

Task: Run Alpine with CASE_ID set to case7 and print it.

Solution command

```
docker run --rm -e CASE_ID=case7 alpine:3.20 sh -c 'echo $CASE_ID'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -e CASE_ID=case7 alpine:3.20 sh -c 'echo $CASE_ID'`
4. Interpret the result: The -e flag injects an environment variable into the container process.

Why it matters: Avoid passing secrets as ordinary environment variables in shared logs or command history.

Exercise 148 · Pass an environment variable

Task: Run Alpine with CASE_ID set to case8 and print it.

Solution command

```
docker run --rm -e CASE_ID=case8 alpine:3.20 sh -c 'echo $CASE_ID'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -e CASE_ID=case8 alpine:3.20 sh -c 'echo $CASE_ID'`
4. Interpret the result: The -e flag injects an environment variable into the container process.

Why it matters: Avoid passing secrets as ordinary environment variables in shared logs or command history.

Exercise 149 · Pass an environment variable

Task: Run Alpine with CASE_ID set to case9 and print it.

Solution command

```
docker run --rm -e CASE_ID=case9 alpine:3.20 sh -c 'echo $CASE_ID'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -e CASE_ID=case9 alpine:3.20 sh -c 'echo $CASE_ID'`
4. Interpret the result: The -e flag injects an environment variable into the container process.

Why it matters: Avoid passing secrets as ordinary environment variables in shared logs or command history.

Exercise 150 · Pass an environment variable

Task: Run Alpine with CASE_ID set to case10 and print it.

Solution command

```
docker run --rm -e CASE_ID=case10 alpine:3.20 sh -c 'echo $CASE_ID'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -e CASE_ID=case10 alpine:3.20 sh -c 'echo $CASE_ID'`
4. Interpret the result: The -e flag injects an environment variable into the container process.

Why it matters: Avoid passing secrets as ordinary environment variables in shared logs or command history.

Exercise 151 · Run with a read-only root filesystem

Task: Launch a test container with its root filesystem read-only for case 1.

Solution command

```
docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'`
4. Interpret the result: Read-only mode permits reads but blocks writes to the container root filesystem.

Why it matters: Applications that need temporary writes may require an explicit tmpfs or writable volume.

Exercise 152 · Run with a read-only root filesystem

Task: Launch a test container with its root filesystem read-only for case 2.

Solution command

```
docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'`
4. Interpret the result: Read-only mode permits reads but blocks writes to the container root filesystem.

Why it matters: Applications that need temporary writes may require an explicit tmpfs or writable volume.

Exercise 153 · Run with a read-only root filesystem

Task: Launch a test container with its root filesystem read-only for case 3.

Solution command

```
docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'`
4. Interpret the result: Read-only mode permits reads but blocks writes to the container root filesystem.

Why it matters: Applications that need temporary writes may require an explicit tmpfs or writable volume.

Exercise 154 · Run with a read-only root filesystem

Task: Launch a test container with its root filesystem read-only for case 4.

Solution command

```
docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'`
4. Interpret the result: Read-only mode permits reads but blocks writes to the container root filesystem.

Why it matters: Applications that need temporary writes may require an explicit tmpfs or writable volume.

Exercise 155 · Run with a read-only root filesystem

Task: Launch a test container with its root filesystem read-only for case 5.

Solution command

```
docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'`
4. Interpret the result: Read-only mode permits reads but blocks writes to the container root filesystem.

Why it matters: Applications that need temporary writes may require an explicit tmpfs or writable volume.

Exercise 156 · Run with a read-only root filesystem

Task: Launch a test container with its root filesystem read-only for case 6.

Solution command

```
docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'`
4. Interpret the result: Read-only mode permits reads but blocks writes to the container root filesystem.

Why it matters: Applications that need temporary writes may require an explicit tmpfs or writable volume.

Exercise 157 · Run with a read-only root filesystem

Task: Launch a test container with its root filesystem read-only for case 7.

Solution command

```
docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'`
4. Interpret the result: Read-only mode permits reads but blocks writes to the container root filesystem.

Why it matters: Applications that need temporary writes may require an explicit tmpfs or writable volume.

Exercise 158 · Run with a read-only root filesystem

Task: Launch a test container with its root filesystem read-only for case 8.

Solution command

```
docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'`
4. Interpret the result: Read-only mode permits reads but blocks writes to the container root filesystem.

Why it matters: Applications that need temporary writes may require an explicit tmpfs or writable volume.

Exercise 159 · Run with a read-only root filesystem

Task: Launch a test container with its root filesystem read-only for case 9.

Solution command

```
docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'`
4. Interpret the result: Read-only mode permits reads but blocks writes to the container root filesystem.

Why it matters: Applications that need temporary writes may require an explicit tmpfs or writable volume.

Exercise 160 · Run with a read-only root filesystem

Task: Launch a test container with its root filesystem read-only for case 10.

Solution command

```
docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --read-only alpine:3.20 sh -c 'cat /etc/alpine-release'`
4. Interpret the result: Read-only mode permits reads but blocks writes to the container root filesystem.

Why it matters: Applications that need temporary writes may require an explicit tmpfs or writable volume.

Module 09: Container discovery & startup

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 161 · List all containers

Task: Find running and stopped containers created during lab 1.

Solution command

```
docker ps -a
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --no-trunc`
4. Interpret the result: Compare status, names, image, and command; use IDs to target the exact container.

Why it matters: The default docker ps shows only running containers.

Exercise 162 · List all containers

Task: Find running and stopped containers created during lab 2.

Solution command

```
docker ps -a
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --no-trunc`
4. Interpret the result: Compare status, names, image, and command; use IDs to target the exact container.

Why it matters: The default docker ps shows only running containers.

Exercise 163 · List all containers

Task: Find running and stopped containers created during lab 3.

Solution command

```
docker ps -a
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --no-trunc`
4. Interpret the result: Compare status, names, image, and command; use IDs to target the exact container.

Why it matters: The default docker ps shows only running containers.

Exercise 164 · List all containers

Task: Find running and stopped containers created during lab 4.

Solution command

```
docker ps -a
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --no-trunc`
4. Interpret the result: Compare status, names, image, and command; use IDs to target the exact container.

Why it matters: The default docker ps shows only running containers.

Exercise 165 · List all containers

Task: Find running and stopped containers created during lab 5.

Solution command

```
docker ps -a
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --no-trunc`
4. Interpret the result: Compare status, names, image, and command; use IDs to target the exact container.

Why it matters: The default docker ps shows only running containers.

Exercise 166 · List all containers

Task: Find running and stopped containers created during lab 6.

Solution command

```
docker ps -a
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --no-trunc`
4. Interpret the result: Compare status, names, image, and command; use IDs to target the exact container.

Why it matters: The default docker ps shows only running containers.

Exercise 167 · List all containers

Task: Find running and stopped containers created during lab 7.

Solution command

```
docker ps -a
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --no-trunc`
4. Interpret the result: Compare status, names, image, and command; use IDs to target the exact container.

Why it matters: The default docker ps shows only running containers.

Exercise 168 · List all containers

Task: Find running and stopped containers created during lab 8.

Solution command

```
docker ps -a
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --no-trunc`
4. Interpret the result: Compare status, names, image, and command; use IDs to target the exact container.

Why it matters: The default docker ps shows only running containers.

Exercise 169 · List all containers

Task: Find running and stopped containers created during lab 9.

Solution command

```
docker ps -a
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --no-trunc`
4. Interpret the result: Compare status, names, image, and command; use IDs to target the exact container.

Why it matters: The default docker ps shows only running containers.

Exercise 170 · List all containers

Task: Find running and stopped containers created during lab 10.

Solution command

```
docker ps -a
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --no-trunc`
4. Interpret the result: Compare status, names, image, and command; use IDs to target the exact container.

Why it matters: The default docker ps shows only running containers.

Exercise 171 · Start an existing stopped container

Task: Start the existing container named worker1.

Solution command

```
docker start worker1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=worker1`
4. Interpret the result: Start works only if that container already exists; create it first if needed.

Why it matters: Start reuses the container's writable layer and configuration.

Exercise 172 · Start an existing stopped container

Task: Start the existing container named worker2.

Solution command

```
docker start worker2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=worker2`
4. Interpret the result: Start works only if that container already exists; create it first if needed.

Why it matters: Start reuses the container's writable layer and configuration.

Exercise 173 · Start an existing stopped container

Task: Start the existing container named worker3.

Solution command

```
docker start worker3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=worker3`
4. Interpret the result: Start works only if that container already exists; create it first if needed.

Why it matters: Start reuses the container's writable layer and configuration.

Exercise 174 · Start an existing stopped container

Task: Start the existing container named worker4.

Solution command

```
docker start worker4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=worker4`
4. Interpret the result: Start works only if that container already exists; create it first if needed.

Why it matters: Start reuses the container's writable layer and configuration.

Exercise 175 · Start an existing stopped container

Task: Start the existing container named worker5.

Solution command

```
docker start worker5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=worker5`
4. Interpret the result: Start works only if that container already exists; create it first if needed.

Why it matters: Start reuses the container's writable layer and configuration.

Exercise 176 · Start an existing stopped container

Task: Start the existing container named worker6.

Solution command

```
docker start worker6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=worker6`
4. Interpret the result: Start works only if that container already exists; create it first if needed.

Why it matters: Start reuses the container's writable layer and configuration.

Exercise 177 · Start an existing stopped container

Task: Start the existing container named worker7.

Solution command

```
docker start worker7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=worker7`
4. Interpret the result: Start works only if that container already exists; create it first if needed.

Why it matters: Start reuses the container's writable layer and configuration.

Exercise 178 · Start an existing stopped container

Task: Start the existing container named worker8.

Solution command

```
docker start worker8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=worker8`
4. Interpret the result: Start works only if that container already exists; create it first if needed.

Why it matters: Start reuses the container's writable layer and configuration.

Exercise 179 · Start an existing stopped container

Task: Start the existing container named worker9.

Solution command

```
docker start worker9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=worker9`
4. Interpret the result: Start works only if that container already exists; create it first if needed.

Why it matters: Start reuses the container's writable layer and configuration.

Exercise 180 · Start an existing stopped container

Task: Start the existing container named worker10.

Solution command

```
docker start worker10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=worker10`
4. Interpret the result: Start works only if that container already exists; create it first if needed.

Why it matters: Start reuses the container's writable layer and configuration.

Module 10: Container stop, restart & pause

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 181 · Stop gracefully

Task: Gracefully stop web1 and allow its configured stop timeout.

Solution command

```
docker stop --time 10 web1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=web1`
4. Interpret the result: Docker sends the configured stop signal and waits up to the timeout before forcing termination.

Why it matters: Prefer stop over kill for normal shutdown.

Exercise 182 · Stop gracefully

Task: Gracefully stop web2 and allow its configured stop timeout.

Solution command

```
docker stop --time 10 web2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=web2`
4. Interpret the result: Docker sends the configured stop signal and waits up to the timeout before forcing termination.

Why it matters: Prefer stop over kill for normal shutdown.

Exercise 183 · Stop gracefully

Task: Gracefully stop web3 and allow its configured stop timeout.

Solution command

```
docker stop --time 10 web3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=web3`
4. Interpret the result: Docker sends the configured stop signal and waits up to the timeout before forcing termination.

Why it matters: Prefer stop over kill for normal shutdown.

Exercise 184 · Stop gracefully

Task: Gracefully stop web4 and allow its configured stop timeout.

Solution command

```
docker stop --time 10 web4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=web4`
4. Interpret the result: Docker sends the configured stop signal and waits up to the timeout before forcing termination.

Why it matters: Prefer stop over kill for normal shutdown.

Exercise 185 · Stop gracefully

Task: Gracefully stop web5 and allow its configured stop timeout.

Solution command

```
docker stop --time 10 web5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=web5`
4. Interpret the result: Docker sends the configured stop signal and waits up to the timeout before forcing termination.

Why it matters: Prefer stop over kill for normal shutdown.

Exercise 186 · Stop gracefully

Task: Gracefully stop web6 and allow its configured stop timeout.

Solution command

```
docker stop --time 10 web6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=web6`
4. Interpret the result: Docker sends the configured stop signal and waits up to the timeout before forcing termination.

Why it matters: Prefer stop over kill for normal shutdown.

Exercise 187 · Stop gracefully

Task: Gracefully stop web7 and allow its configured stop timeout.

Solution command

```
docker stop --time 10 web7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=web7`
4. Interpret the result: Docker sends the configured stop signal and waits up to the timeout before forcing termination.

Why it matters: Prefer stop over kill for normal shutdown.

Exercise 188 · Stop gracefully

Task: Gracefully stop web8 and allow its configured stop timeout.

Solution command

```
docker stop --time 10 web8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=web8`
4. Interpret the result: Docker sends the configured stop signal and waits up to the timeout before forcing termination.

Why it matters: Prefer stop over kill for normal shutdown.

Exercise 189 · Stop gracefully

Task: Gracefully stop web9 and allow its configured stop timeout.

Solution command

```
docker stop --time 10 web9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=web9`
4. Interpret the result: Docker sends the configured stop signal and waits up to the timeout before forcing termination.

Why it matters: Prefer stop over kill for normal shutdown.

Exercise 190 · Stop gracefully

Task: Gracefully stop web10 and allow its configured stop timeout.

Solution command

```
docker stop --time 10 web10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=web10`
4. Interpret the result: Docker sends the configured stop signal and waits up to the timeout before forcing termination.

Why it matters: Prefer stop over kill for normal shutdown.

Exercise 191 · Pause and unpause

Task: Pause worker1, inspect its status, then resume it.

Solution command

```
docker pause worker1 && docker unpause worker1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' worker1`
4. Interpret the result: Pause freezes container processes; unpause resumes them.

Why it matters: This is not the same as stopping the container.

Exercise 192 · Pause and unpause

Task: Pause worker2, inspect its status, then resume it.

Solution command

```
docker pause worker2 && docker unpause worker2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' worker2`
4. Interpret the result: Pause freezes container processes; unpause resumes them.

Why it matters: This is not the same as stopping the container.

Exercise 193 · Pause and unpause

Task: Pause worker3, inspect its status, then resume it.

Solution command

```
docker pause worker3 && docker unpause worker3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' worker3`
4. Interpret the result: Pause freezes container processes; unpause resumes them.

Why it matters: This is not the same as stopping the container.

Exercise 194 · Pause and unpause

Task: Pause worker4, inspect its status, then resume it.

Solution command

```
docker pause worker4 && docker unpause worker4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' worker4`
4. Interpret the result: Pause freezes container processes; unpause resumes them.

Why it matters: This is not the same as stopping the container.

Exercise 195 · Pause and unpause

Task: Pause worker5, inspect its status, then resume it.

Solution command

```
docker pause worker5 && docker unpause worker5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' worker5`
4. Interpret the result: Pause freezes container processes; unpause resumes them.

Why it matters: This is not the same as stopping the container.

Exercise 196 · Pause and unpause

Task: Pause worker6, inspect its status, then resume it.

Solution command

```
docker pause worker6 && docker unpause worker6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' worker6`
4. Interpret the result: Pause freezes container processes; unpause resumes them.

Why it matters: This is not the same as stopping the container.

Exercise 197 · Pause and unpause

Task: Pause worker7, inspect its status, then resume it.

Solution command

```
docker pause worker7 && docker unpause worker7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' worker7`
4. Interpret the result: Pause freezes container processes; unpause resumes them.

Why it matters: This is not the same as stopping the container.

Exercise 198 · Pause and unpause

Task: Pause worker8, inspect its status, then resume it.

Solution command

```
docker pause worker8 && docker unpause worker8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' worker8`
4. Interpret the result: Pause freezes container processes; unpause resumes them.

Why it matters: This is not the same as stopping the container.

Exercise 199 · Pause and unpause

Task: Pause worker9, inspect its status, then resume it.

Solution command

```
docker pause worker9 && docker unpause worker9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' worker9`
4. Interpret the result: Pause freezes container processes; unpause resumes them.

Why it matters: This is not the same as stopping the container.

Exercise 200 · Pause and unpause

Task: Pause worker10, inspect its status, then resume it.

Solution command

```
docker pause worker10 && docker unpause worker10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' worker10`
4. Interpret the result: Pause freezes container processes; unpause resumes them.

Why it matters: This is not the same as stopping the container.

Module 11: Container removal & safe cleanup

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 201 · Remove a stopped container

Task: Remove only the stopped lab container named old1.

Solution command

```
docker rm old1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=old1`
4. Interpret the result: Confirm it is stopped, remove it, then verify that the named container is absent.

Why it matters: Avoid -f unless you understand why a running container must be forcibly removed.

Exercise 202 · Remove a stopped container

Task: Remove only the stopped lab container named old2.

Solution command

```
docker rm old2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=old2`
4. Interpret the result: Confirm it is stopped, remove it, then verify that the named container is absent.

Why it matters: Avoid -f unless you understand why a running container must be forcibly removed.

Exercise 203 · Remove a stopped container

Task: Remove only the stopped lab container named old3.

Solution command

```
docker rm old3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=old3`
4. Interpret the result: Confirm it is stopped, remove it, then verify that the named container is absent.

Why it matters: Avoid -f unless you understand why a running container must be forcibly removed.

Exercise 204 · Remove a stopped container

Task: Remove only the stopped lab container named old4.

Solution command

```
docker rm old4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=old4`
4. Interpret the result: Confirm it is stopped, remove it, then verify that the named container is absent.

Why it matters: Avoid -f unless you understand why a running container must be forcibly removed.

Exercise 205 · Remove a stopped container

Task: Remove only the stopped lab container named old5.

Solution command

```
docker rm old5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=old5`
4. Interpret the result: Confirm it is stopped, remove it, then verify that the named container is absent.

Why it matters: Avoid -f unless you understand why a running container must be forcibly removed.

Exercise 206 · Remove a stopped container

Task: Remove only the stopped lab container named old6.

Solution command

```
docker rm old6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=old6`
4. Interpret the result: Confirm it is stopped, remove it, then verify that the named container is absent.

Why it matters: Avoid -f unless you understand why a running container must be forcibly removed.

Exercise 207 · Remove a stopped container

Task: Remove only the stopped lab container named old7.

Solution command

```
docker rm old7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=old7`
4. Interpret the result: Confirm it is stopped, remove it, then verify that the named container is absent.

Why it matters: Avoid -f unless you understand why a running container must be forcibly removed.

Exercise 208 · Remove a stopped container

Task: Remove only the stopped lab container named old8.

Solution command

```
docker rm old8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=old8`
4. Interpret the result: Confirm it is stopped, remove it, then verify that the named container is absent.

Why it matters: Avoid -f unless you understand why a running container must be forcibly removed.

Exercise 209 · Remove a stopped container

Task: Remove only the stopped lab container named old9.

Solution command

```
docker rm old9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=old9`
4. Interpret the result: Confirm it is stopped, remove it, then verify that the named container is absent.

Why it matters: Avoid -f unless you understand why a running container must be forcibly removed.

Exercise 210 · Remove a stopped container

Task: Remove only the stopped lab container named old10.

Solution command

```
docker rm old10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=old10`
4. Interpret the result: Confirm it is stopped, remove it, then verify that the named container is absent.

Why it matters: Avoid -f unless you understand why a running container must be forcibly removed.

Exercise 211 · Remove dangling images safely

Task: Review dangling images and remove only dangling image layers for cleanup case 1.

Solution command

```
docker image prune
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter dangling=true`
4. Interpret the result: Inspect the prompt and reclaimed-space summary; use -f only in a controlled lab where confirmation is intended.

Why it matters: Prune can delete reusable build artifacts; do not run broad prune commands on important hosts casually.

Exercise 212 · Remove dangling images safely

Task: Review dangling images and remove only dangling image layers for cleanup case 2.

Solution command

```
docker image prune
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter dangling=true`
4. Interpret the result: Inspect the prompt and reclaimed-space summary; use -f only in a controlled lab where confirmation is intended.

Why it matters: Prune can delete reusable build artifacts; do not run broad prune commands on important hosts casually.

Exercise 213 · Remove dangling images safely

Task: Review dangling images and remove only dangling image layers for cleanup case 3.

Solution command

```
docker image prune
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter dangling=true`
4. Interpret the result: Inspect the prompt and reclaimed-space summary; use -f only in a controlled lab where confirmation is intended.

Why it matters: Prune can delete reusable build artifacts; do not run broad prune commands on important hosts casually.

Exercise 214 · Remove dangling images safely

Task: Review dangling images and remove only dangling image layers for cleanup case 4.

Solution command

```
docker image prune
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter dangling=true`
4. Interpret the result: Inspect the prompt and reclaimed-space summary; use -f only in a controlled lab where confirmation is intended.

Why it matters: Prune can delete reusable build artifacts; do not run broad prune commands on important hosts casually.

Exercise 215 · Remove dangling images safely

Task: Review dangling images and remove only dangling image layers for cleanup case 5.

Solution command

```
docker image prune
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter dangling=true`
4. Interpret the result: Inspect the prompt and reclaimed-space summary; use -f only in a controlled lab where confirmation is intended.

Why it matters: Prune can delete reusable build artifacts; do not run broad prune commands on important hosts casually.

Exercise 216 · Remove dangling images safely

Task: Review dangling images and remove only dangling image layers for cleanup case 6.

Solution command

```
docker image prune
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter dangling=true`
4. Interpret the result: Inspect the prompt and reclaimed-space summary; use -f only in a controlled lab where confirmation is intended.

Why it matters: Prune can delete reusable build artifacts; do not run broad prune commands on important hosts casually.

Exercise 217 · Remove dangling images safely

Task: Review dangling images and remove only dangling image layers for cleanup case 7.

Solution command

```
docker image prune
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter dangling=true`
4. Interpret the result: Inspect the prompt and reclaimed-space summary; use -f only in a controlled lab where confirmation is intended.

Why it matters: Prune can delete reusable build artifacts; do not run broad prune commands on important hosts casually.

Exercise 218 · Remove dangling images safely

Task: Review dangling images and remove only dangling image layers for cleanup case 8.

Solution command

```
docker image prune
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter dangling=true`
4. Interpret the result: Inspect the prompt and reclaimed-space summary; use -f only in a controlled lab where confirmation is intended.

Why it matters: Prune can delete reusable build artifacts; do not run broad prune commands on important hosts casually.

Exercise 219 · Remove dangling images safely

Task: Review dangling images and remove only dangling image layers for cleanup case 9.

Solution command

```
docker image prune
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter dangling=true`
4. Interpret the result: Inspect the prompt and reclaimed-space summary; use -f only in a controlled lab where confirmation is intended.

Why it matters: Prune can delete reusable build artifacts; do not run broad prune commands on important hosts casually.

Exercise 220 · Remove dangling images safely

Task: Review dangling images and remove only dangling image layers for cleanup case 10.

Solution command

```
docker image prune
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls --filter dangling=true`
4. Interpret the result: Inspect the prompt and reclaimed-space summary; use -f only in a controlled lab where confirmation is intended.

Why it matters: Prune can delete reusable build artifacts; do not run broad prune commands on important hosts casually.

Module 12: Interactive access & file copy

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 221 · Open a shell in a running container

Task: Open a shell in running container app1.

Solution command

```
docker exec -it app1 sh
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker exec app1 sh -c 'id && pwd'`
4. Interpret the result: Use -it for an interactive terminal; use a non-interactive exec command for repeatable checks.

Why it matters: The shell must exist in the image; minimal images may not include bash.

Exercise 222 · Open a shell in a running container

Task: Open a shell in running container app2.

Solution command

```
docker exec -it app2 sh
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker exec app2 sh -c 'id && pwd'`
4. Interpret the result: Use -it for an interactive terminal; use a non-interactive exec command for repeatable checks.

Why it matters: The shell must exist in the image; minimal images may not include bash.

Exercise 223 · Open a shell in a running container

Task: Open a shell in running container app3.

Solution command

```
docker exec -it app3 sh
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker exec app3 sh -c 'id && pwd'`
4. Interpret the result: Use -it for an interactive terminal; use a non-interactive exec command for repeatable checks.

Why it matters: The shell must exist in the image; minimal images may not include bash.

Exercise 224 · Open a shell in a running container

Task: Open a shell in running container app4.

Solution command

```
docker exec -it app4 sh
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker exec app4 sh -c 'id && pwd'`
4. Interpret the result: Use -it for an interactive terminal; use a non-interactive exec command for repeatable checks.

Why it matters: The shell must exist in the image; minimal images may not include bash.

Exercise 225 · Open a shell in a running container

Task: Open a shell in running container app5.

Solution command

```
docker exec -it app5 sh
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker exec app5 sh -c 'id && pwd'`
4. Interpret the result: Use -it for an interactive terminal; use a non-interactive exec command for repeatable checks.

Why it matters: The shell must exist in the image; minimal images may not include bash.

Exercise 226 · Open a shell in a running container

Task: Open a shell in running container app6.

Solution command

```
docker exec -it app6 sh
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker exec app6 sh -c 'id && pwd'`
4. Interpret the result: Use -it for an interactive terminal; use a non-interactive exec command for repeatable checks.

Why it matters: The shell must exist in the image; minimal images may not include bash.

Exercise 227 · Open a shell in a running container

Task: Open a shell in running container app7.

Solution command

```
docker exec -it app7 sh
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker exec app7 sh -c 'id && pwd'`
4. Interpret the result: Use -it for an interactive terminal; use a non-interactive exec command for repeatable checks.

Why it matters: The shell must exist in the image; minimal images may not include bash.

Exercise 228 · Open a shell in a running container

Task: Open a shell in running container app8.

Solution command

```
docker exec -it app8 sh
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker exec app8 sh -c 'id && pwd'`
4. Interpret the result: Use -it for an interactive terminal; use a non-interactive exec command for repeatable checks.

Why it matters: The shell must exist in the image; minimal images may not include bash.

Exercise 229 · Open a shell in a running container

Task: Open a shell in running container app9.

Solution command

```
docker exec -it app9 sh
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker exec app9 sh -c 'id && pwd'`
4. Interpret the result: Use -it for an interactive terminal; use a non-interactive exec command for repeatable checks.

Why it matters: The shell must exist in the image; minimal images may not include bash.

Exercise 230 · Open a shell in a running container

Task: Open a shell in running container app10.

Solution command

```
docker exec -it app10 sh
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker exec app10 sh -c 'id && pwd'`
4. Interpret the result: Use -it for an interactive terminal; use a non-interactive exec command for repeatable checks.

Why it matters: The shell must exist in the image; minimal images may not include bash.

Exercise 231 · Copy a file from a container

Task: Copy /etc/os-release from container app1 to ./os-release-1.txt.

Solution command

```
docker cp app1:/etc/os-release ./os-release-1.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -l ./os-release-1.txt`
4. Interpret the result: docker cp works with running or stopped containers and copies between container and host paths.

Why it matters: Check path permissions and avoid overwriting important files.

Exercise 232 · Copy a file from a container

Task: Copy /etc/os-release from container app2 to ./os-release-2.txt.

Solution command

```
docker cp app2:/etc/os-release ./os-release-2.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -l ./os-release-2.txt`
4. Interpret the result: docker cp works with running or stopped containers and copies between container and host paths.

Why it matters: Check path permissions and avoid overwriting important files.

Exercise 233 · Copy a file from a container

Task: Copy /etc/os-release from container app3 to ./os-release-3.txt.

Solution command

```
docker cp app3:/etc/os-release ./os-release-3.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -l ./os-release-3.txt`
4. Interpret the result: docker cp works with running or stopped containers and copies between container and host paths.

Why it matters: Check path permissions and avoid overwriting important files.

Exercise 234 · Copy a file from a container

Task: Copy /etc/os-release from container app4 to ./os-release-4.txt.

Solution command

```
docker cp app4:/etc/os-release ./os-release-4.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -l ./os-release-4.txt`
4. Interpret the result: docker cp works with running or stopped containers and copies between container and host paths.

Why it matters: Check path permissions and avoid overwriting important files.

Exercise 235 · Copy a file from a container

Task: Copy /etc/os-release from container app5 to ./os-release-5.txt.

Solution command

```
docker cp app5:/etc/os-release ./os-release-5.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -l ./os-release-5.txt`
4. Interpret the result: docker cp works with running or stopped containers and copies between container and host paths.

Why it matters: Check path permissions and avoid overwriting important files.

Exercise 236 · Copy a file from a container

Task: Copy /etc/os-release from container app6 to ./os-release-6.txt.

Solution command

```
docker cp app6:/etc/os-release ./os-release-6.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -l ./os-release-6.txt`
4. Interpret the result: docker cp works with running or stopped containers and copies between container and host paths.

Why it matters: Check path permissions and avoid overwriting important files.

Exercise 237 · Copy a file from a container

Task: Copy /etc/os-release from container app7 to ./os-release-7.txt.

Solution command

```
docker cp app7:/etc/os-release ./os-release-7.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -l ./os-release-7.txt`
4. Interpret the result: docker cp works with running or stopped containers and copies between container and host paths.

Why it matters: Check path permissions and avoid overwriting important files.

Exercise 238 · Copy a file from a container

Task: Copy /etc/os-release from container app8 to ./os-release-8.txt.

Solution command

```
docker cp app8:/etc/os-release ./os-release-8.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -l ./os-release-8.txt`
4. Interpret the result: docker cp works with running or stopped containers and copies between container and host paths.

Why it matters: Check path permissions and avoid overwriting important files.

Exercise 239 · Copy a file from a container

Task: Copy /etc/os-release from container app9 to ./os-release-9.txt.

Solution command

```
docker cp app9:/etc/os-release ./os-release-9.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -l ./os-release-9.txt`
4. Interpret the result: docker cp works with running or stopped containers and copies between container and host paths.

Why it matters: Check path permissions and avoid overwriting important files.

Exercise 240 · Copy a file from a container

Task: Copy /etc/os-release from container app10 to ./os-release-10.txt.

Solution command

```
docker cp app10:/etc/os-release ./os-release-10.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -l ./os-release-10.txt`
4. Interpret the result: docker cp works with running or stopped containers and copies between container and host paths.

Why it matters: Check path permissions and avoid overwriting important files.

Module 13: Logs, resource use & processes

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 241 · Read recent logs

Task: Display the latest 50 log lines from app1 and follow new output.

Solution command

```
docker logs --tail 50 --follow app1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker logs --tail 50 app1`
4. Interpret the result: Use --follow to stream; press Ctrl+C to stop following without stopping the container.

Why it matters: If logs are empty, check the application's logging behavior and configured logging driver.

Exercise 242 · Read recent logs

Task: Display the latest 50 log lines from app2 and follow new output.

Solution command

```
docker logs --tail 50 --follow app2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker logs --tail 50 app2`
4. Interpret the result: Use --follow to stream; press Ctrl+C to stop following without stopping the container.

Why it matters: If logs are empty, check the application's logging behavior and configured logging driver.

Exercise 243 · Read recent logs

Task: Display the latest 50 log lines from app3 and follow new output.

Solution command

```
docker logs --tail 50 --follow app3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker logs --tail 50 app3`
4. Interpret the result: Use --follow to stream; press Ctrl+C to stop following without stopping the container.

Why it matters: If logs are empty, check the application's logging behavior and configured logging driver.

Exercise 244 · Read recent logs

Task: Display the latest 50 log lines from app4 and follow new output.

Solution command

```
docker logs --tail 50 --follow app4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker logs --tail 50 app4`
4. Interpret the result: Use --follow to stream; press Ctrl+C to stop following without stopping the container.

Why it matters: If logs are empty, check the application's logging behavior and configured logging driver.

Exercise 245 · Read recent logs

Task: Display the latest 50 log lines from app5 and follow new output.

Solution command

```
docker logs --tail 50 --follow app5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker logs --tail 50 app5`
4. Interpret the result: Use --follow to stream; press Ctrl+C to stop following without stopping the container.

Why it matters: If logs are empty, check the application's logging behavior and configured logging driver.

Exercise 246 · Read recent logs

Task: Display the latest 50 log lines from app6 and follow new output.

Solution command

```
docker logs --tail 50 --follow app6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker logs --tail 50 app6`
4. Interpret the result: Use --follow to stream; press Ctrl+C to stop following without stopping the container.

Why it matters: If logs are empty, check the application's logging behavior and configured logging driver.

Exercise 247 · Read recent logs

Task: Display the latest 50 log lines from app7 and follow new output.

Solution command

```
docker logs --tail 50 --follow app7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker logs --tail 50 app7`
4. Interpret the result: Use --follow to stream; press Ctrl+C to stop following without stopping the container.

Why it matters: If logs are empty, check the application's logging behavior and configured logging driver.

Exercise 248 · Read recent logs

Task: Display the latest 50 log lines from app8 and follow new output.

Solution command

```
docker logs --tail 50 --follow app8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker logs --tail 50 app8`
4. Interpret the result: Use --follow to stream; press Ctrl+C to stop following without stopping the container.

Why it matters: If logs are empty, check the application's logging behavior and configured logging driver.

Exercise 249 · Read recent logs

Task: Display the latest 50 log lines from app9 and follow new output.

Solution command

```
docker logs --tail 50 --follow app9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker logs --tail 50 app9`
4. Interpret the result: Use --follow to stream; press Ctrl+C to stop following without stopping the container.

Why it matters: If logs are empty, check the application's logging behavior and configured logging driver.

Exercise 250 · Read recent logs

Task: Display the latest 50 log lines from app10 and follow new output.

Solution command

```
docker logs --tail 50 --follow app10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker logs --tail 50 app10`
4. Interpret the result: Use --follow to stream; press Ctrl+C to stop following without stopping the container.

Why it matters: If logs are empty, check the application's logging behavior and configured logging driver.

Exercise 251 · Inspect live resource usage

Task: Take a one-time CPU and memory snapshot for app1.

Solution command

```
docker stats --no-stream app1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stats --no-stream app1`
4. Interpret the result: Compare CPU %, memory usage/limit, network I/O, and block I/O.

Why it matters: A snapshot is more suitable for a worksheet than a continuously updating display.

Exercise 252 · Inspect live resource usage

Task: Take a one-time CPU and memory snapshot for app2.

Solution command

```
docker stats --no-stream app2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stats --no-stream app2`
4. Interpret the result: Compare CPU %, memory usage/limit, network I/O, and block I/O.

Why it matters: A snapshot is more suitable for a worksheet than a continuously updating display.

Exercise 253 · Inspect live resource usage

Task: Take a one-time CPU and memory snapshot for app3.

Solution command

```
docker stats --no-stream app3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stats --no-stream app3`
4. Interpret the result: Compare CPU %, memory usage/limit, network I/O, and block I/O.

Why it matters: A snapshot is more suitable for a worksheet than a continuously updating display.

Exercise 254 · Inspect live resource usage

Task: Take a one-time CPU and memory snapshot for app4.

Solution command

```
docker stats --no-stream app4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stats --no-stream app4`
4. Interpret the result: Compare CPU %, memory usage/limit, network I/O, and block I/O.

Why it matters: A snapshot is more suitable for a worksheet than a continuously updating display.

Exercise 255 · Inspect live resource usage

Task: Take a one-time CPU and memory snapshot for app5.

Solution command

```
docker stats --no-stream app5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stats --no-stream app5`
4. Interpret the result: Compare CPU %, memory usage/limit, network I/O, and block I/O.

Why it matters: A snapshot is more suitable for a worksheet than a continuously updating display.

Exercise 256 · Inspect live resource usage

Task: Take a one-time CPU and memory snapshot for app6.

Solution command

```
docker stats --no-stream app6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stats --no-stream app6`
4. Interpret the result: Compare CPU %, memory usage/limit, network I/O, and block I/O.

Why it matters: A snapshot is more suitable for a worksheet than a continuously updating display.

Exercise 257 · Inspect live resource usage

Task: Take a one-time CPU and memory snapshot for app7.

Solution command

```
docker stats --no-stream app7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stats --no-stream app7`
4. Interpret the result: Compare CPU %, memory usage/limit, network I/O, and block I/O.

Why it matters: A snapshot is more suitable for a worksheet than a continuously updating display.

Exercise 258 · Inspect live resource usage

Task: Take a one-time CPU and memory snapshot for app8.

Solution command

```
docker stats --no-stream app8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stats --no-stream app8`
4. Interpret the result: Compare CPU %, memory usage/limit, network I/O, and block I/O.

Why it matters: A snapshot is more suitable for a worksheet than a continuously updating display.

Exercise 259 · Inspect live resource usage

Task: Take a one-time CPU and memory snapshot for app9.

Solution command

```
docker stats --no-stream app9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stats --no-stream app9`
4. Interpret the result: Compare CPU %, memory usage/limit, network I/O, and block I/O.

Why it matters: A snapshot is more suitable for a worksheet than a continuously updating display.

Exercise 260 · Inspect live resource usage

Task: Take a one-time CPU and memory snapshot for app10.

Solution command

```
docker stats --no-stream app10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stats --no-stream app10`
4. Interpret the result: Compare CPU %, memory usage/limit, network I/O, and block I/O.

Why it matters: A snapshot is more suitable for a worksheet than a continuously updating display.

Module 14: Container inspection & port mapping

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 261 · Print container state

Task: Get the state and image for container app1 in compact form.

Solution command

```
docker inspect --format '{.Name} {.Config.Image} {.State.Status}' app1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app1`
4. Interpret the result: Use a Go-template format string to extract fields from inspect JSON.

Why it matters: Template fields are case-sensitive and depend on the object type.

Exercise 262 · Print container state

Task: Get the state and image for container app2 in compact form.

Solution command

```
docker inspect --format '{.Name} {.Config.Image} {.State.Status}' app2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app2`
4. Interpret the result: Use a Go-template format string to extract fields from inspect JSON.

Why it matters: Template fields are case-sensitive and depend on the object type.

Exercise 263 · Print container state

Task: Get the state and image for container app3 in compact form.

Solution command

```
docker inspect --format '{.Name} {.Config.Image} {.State.Status}' app3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app3`
4. Interpret the result: Use a Go-template format string to extract fields from inspect JSON.

Why it matters: Template fields are case-sensitive and depend on the object type.

Exercise 264 · Print container state

Task: Get the state and image for container app4 in compact form.

Solution command

```
docker inspect --format '{.Name} {.Config.Image} {.State.Status}' app4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app4`
4. Interpret the result: Use a Go-template format string to extract fields from inspect JSON.

Why it matters: Template fields are case-sensitive and depend on the object type.

Exercise 265 · Print container state

Task: Get the state and image for container app5 in compact form.

Solution command

```
docker inspect --format '{.Name} {.Config.Image} {.State.Status}' app5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app5`
4. Interpret the result: Use a Go-template format string to extract fields from inspect JSON.

Why it matters: Template fields are case-sensitive and depend on the object type.

Exercise 266 · Print container state

Task: Get the state and image for container app6 in compact form.

Solution command

```
docker inspect --format '{.Name} {.Config.Image} {.State.Status}' app6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app6`
4. Interpret the result: Use a Go-template format string to extract fields from inspect JSON.

Why it matters: Template fields are case-sensitive and depend on the object type.

Exercise 267 · Print container state

Task: Get the state and image for container app7 in compact form.

Solution command

```
docker inspect --format '{.Name} {.Config.Image} {.State.Status}' app7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app7`
4. Interpret the result: Use a Go-template format string to extract fields from inspect JSON.

Why it matters: Template fields are case-sensitive and depend on the object type.

Exercise 268 · Print container state

Task: Get the state and image for container app8 in compact form.

Solution command

```
docker inspect --format '{.Name} {.Config.Image} {.State.Status}' app8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app8`
4. Interpret the result: Use a Go-template format string to extract fields from inspect JSON.

Why it matters: Template fields are case-sensitive and depend on the object type.

Exercise 269 · Print container state

Task: Get the state and image for container app9 in compact form.

Solution command

```
docker inspect --format '{.Name} {.Config.Image} {.State.Status}' app9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app9`
4. Interpret the result: Use a Go-template format string to extract fields from inspect JSON.

Why it matters: Template fields are case-sensitive and depend on the object type.

Exercise 270 · Print container state

Task: Get the state and image for container app10 in compact form.

Solution command

```
docker inspect --format '{.Name} {.Config.Image} {.State.Status}' app10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app10`
4. Interpret the result: Use a Go-template format string to extract fields from inspect JSON.

Why it matters: Template fields are case-sensitive and depend on the object type.

Exercise 271 · Find published ports

Task: Show published ports for web1.

Solution command

```
docker port web1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web1`
4. Interpret the result: Compare the container port with the host address and port.

Why it matters: A port is reachable from outside only if host networking/firewall rules allow it.

Exercise 272 · Find published ports

Task: Show published ports for web2.

Solution command

```
docker port web2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web2`
4. Interpret the result: Compare the container port with the host address and port.

Why it matters: A port is reachable from outside only if host networking/firewall rules allow it.

Exercise 273 · Find published ports

Task: Show published ports for web3.

Solution command

```
docker port web3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web3`
4. Interpret the result: Compare the container port with the host address and port.

Why it matters: A port is reachable from outside only if host networking/firewall rules allow it.

Exercise 274 · Find published ports

Task: Show published ports for web4.

Solution command

```
docker port web4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web4`
4. Interpret the result: Compare the container port with the host address and port.

Why it matters: A port is reachable from outside only if host networking/firewall rules allow it.

Exercise 275 · Find published ports

Task: Show published ports for web5.

Solution command

```
docker port web5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web5`
4. Interpret the result: Compare the container port with the host address and port.

Why it matters: A port is reachable from outside only if host networking/firewall rules allow it.

Exercise 276 · Find published ports

Task: Show published ports for web6.

Solution command

```
docker port web6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web6`
4. Interpret the result: Compare the container port with the host address and port.

Why it matters: A port is reachable from outside only if host networking/firewall rules allow it.

Exercise 277 · Find published ports

Task: Show published ports for web7.

Solution command

```
docker port web7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web7`
4. Interpret the result: Compare the container port with the host address and port.

Why it matters: A port is reachable from outside only if host networking/firewall rules allow it.

Exercise 278 · Find published ports

Task: Show published ports for web8.

Solution command

```
docker port web8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web8`
4. Interpret the result: Compare the container port with the host address and port.

Why it matters: A port is reachable from outside only if host networking/firewall rules allow it.

Exercise 279 · Find published ports

Task: Show published ports for web9.

Solution command

```
docker port web9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web9`
4. Interpret the result: Compare the container port with the host address and port.

Why it matters: A port is reachable from outside only if host networking/firewall rules allow it.

Exercise 280 · Find published ports

Task: Show published ports for web10.

Solution command

```
docker port web10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps --filter name=web10`
4. Interpret the result: Compare the container port with the host address and port.

Why it matters: A port is reachable from outside only if host networking/firewall rules allow it.

Module 15: Resource limits, restart policies & health

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 281 · Update resource limits

Task: Limit a running container named app1 to 0.5 CPU and 256 MiB memory.

Solution command

```
docker update --cpus 0.5 --memory 256m app1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.NanoCpus} {.HostConfig.Memory}' app1`
4. Interpret the result: Apply the update, then inspect the relevant HostConfig values.

Why it matters: Support and exact behavior can vary by platform and runtime configuration.

Exercise 282 · Update resource limits

Task: Limit a running container named app2 to 0.5 CPU and 256 MiB memory.

Solution command

```
docker update --cpus 0.5 --memory 256m app2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.NanoCpus} {.HostConfig.Memory}' app2`
4. Interpret the result: Apply the update, then inspect the relevant HostConfig values.

Why it matters: Support and exact behavior can vary by platform and runtime configuration.

Exercise 283 · Update resource limits

Task: Limit a running container named app3 to 0.5 CPU and 256 MiB memory.

Solution command

```
docker update --cpus 0.5 --memory 256m app3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.NanoCpus} {.HostConfig.Memory}' app3`
4. Interpret the result: Apply the update, then inspect the relevant HostConfig values.

Why it matters: Support and exact behavior can vary by platform and runtime configuration.

Exercise 284 · Update resource limits

Task: Limit a running container named app4 to 0.5 CPU and 256 MiB memory.

Solution command

```
docker update --cpus 0.5 --memory 256m app4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.NanoCpus} {.HostConfig.Memory}' app4`
4. Interpret the result: Apply the update, then inspect the relevant HostConfig values.

Why it matters: Support and exact behavior can vary by platform and runtime configuration.

Exercise 285 · Update resource limits

Task: Limit a running container named app5 to 0.5 CPU and 256 MiB memory.

Solution command

```
docker update --cpus 0.5 --memory 256m app5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.NanoCpus} {.HostConfig.Memory}' app5`
4. Interpret the result: Apply the update, then inspect the relevant HostConfig values.

Why it matters: Support and exact behavior can vary by platform and runtime configuration.

Exercise 286 · Update resource limits

Task: Limit a running container named app6 to 0.5 CPU and 256 MiB memory.

Solution command

```
docker update --cpus 0.5 --memory 256m app6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.NanoCpus} {.HostConfig.Memory}' app6`
4. Interpret the result: Apply the update, then inspect the relevant HostConfig values.

Why it matters: Support and exact behavior can vary by platform and runtime configuration.

Exercise 287 · Update resource limits

Task: Limit a running container named app7 to 0.5 CPU and 256 MiB memory.

Solution command

```
docker update --cpus 0.5 --memory 256m app7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.NanoCpus} {.HostConfig.Memory}' app7`
4. Interpret the result: Apply the update, then inspect the relevant HostConfig values.

Why it matters: Support and exact behavior can vary by platform and runtime configuration.

Exercise 288 · Update resource limits

Task: Limit a running container named app8 to 0.5 CPU and 256 MiB memory.

Solution command

```
docker update --cpus 0.5 --memory 256m app8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.NanoCpus} {.HostConfig.Memory}' app8`
4. Interpret the result: Apply the update, then inspect the relevant HostConfig values.

Why it matters: Support and exact behavior can vary by platform and runtime configuration.

Exercise 289 · Update resource limits

Task: Limit a running container named app9 to 0.5 CPU and 256 MiB memory.

Solution command

```
docker update --cpus 0.5 --memory 256m app9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.NanoCpus} {.HostConfig.Memory}' app9`
4. Interpret the result: Apply the update, then inspect the relevant HostConfig values.

Why it matters: Support and exact behavior can vary by platform and runtime configuration.

Exercise 290 · Update resource limits

Task: Limit a running container named app10 to 0.5 CPU and 256 MiB memory.

Solution command

```
docker update --cpus 0.5 --memory 256m app10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.NanoCpus} {.HostConfig.Memory}' app10`
4. Interpret the result: Apply the update, then inspect the relevant HostConfig values.

Why it matters: Support and exact behavior can vary by platform and runtime configuration.

Exercise 291 · Configure restart behavior

Task: Create a disposable service that restarts unless explicitly stopped, case 1.

Solution command

```
docker run -d --name restart1 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 30; done'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.RestartPolicy.Name}' restart1`
4. Interpret the result: Inspect the restart policy; stop and start the container to understand the lifecycle.

Why it matters: Restart policies help recover from process exits but are not a substitute for application health checks.

Exercise 292 · Configure restart behavior

Task: Create a disposable service that restarts unless explicitly stopped, case 2.

Solution command

```
docker run -d --name restart2 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 30; done'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.RestartPolicy.Name}' restart2`
4. Interpret the result: Inspect the restart policy; stop and start the container to understand the lifecycle.

Why it matters: Restart policies help recover from process exits but are not a substitute for application health checks.

Exercise 293 · Configure restart behavior

Task: Create a disposable service that restarts unless explicitly stopped, case 3.

Solution command

```
docker run -d --name restart3 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 30; done'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.RestartPolicy.Name}' restart3`
4. Interpret the result: Inspect the restart policy; stop and start the container to understand the lifecycle.

Why it matters: Restart policies help recover from process exits but are not a substitute for application health checks.

Exercise 294 · Configure restart behavior

Task: Create a disposable service that restarts unless explicitly stopped, case 4.

Solution command

```
docker run -d --name restart4 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 30; done'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.RestartPolicy.Name}' restart4`
4. Interpret the result: Inspect the restart policy; stop and start the container to understand the lifecycle.

Why it matters: Restart policies help recover from process exits but are not a substitute for application health checks.

Exercise 295 · Configure restart behavior

Task: Create a disposable service that restarts unless explicitly stopped, case 5.

Solution command

```
docker run -d --name restart5 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 30; done'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.RestartPolicy.Name}' restart5`
4. Interpret the result: Inspect the restart policy; stop and start the container to understand the lifecycle.

Why it matters: Restart policies help recover from process exits but are not a substitute for application health checks.

Exercise 296 · Configure restart behavior

Task: Create a disposable service that restarts unless explicitly stopped, case 6.

Solution command

```
docker run -d --name restart6 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 30; done'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.RestartPolicy.Name}' restart6`
4. Interpret the result: Inspect the restart policy; stop and start the container to understand the lifecycle.

Why it matters: Restart policies help recover from process exits but are not a substitute for application health checks.

Exercise 297 · Configure restart behavior

Task: Create a disposable service that restarts unless explicitly stopped, case 7.

Solution command

```
docker run -d --name restart7 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 30; done'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.RestartPolicy.Name}' restart7`
4. Interpret the result: Inspect the restart policy; stop and start the container to understand the lifecycle.

Why it matters: Restart policies help recover from process exits but are not a substitute for application health checks.

Exercise 298 · Configure restart behavior

Task: Create a disposable service that restarts unless explicitly stopped, case 8.

Solution command

```
docker run -d --name restart8 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 30; done'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.RestartPolicy.Name}' restart8`
4. Interpret the result: Inspect the restart policy; stop and start the container to understand the lifecycle.

Why it matters: Restart policies help recover from process exits but are not a substitute for application health checks.

Exercise 299 · Configure restart behavior

Task: Create a disposable service that restarts unless explicitly stopped, case 9.

Solution command

```
docker run -d --name restart9 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 30; done'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.RestartPolicy.Name}' restart9`
4. Interpret the result: Inspect the restart policy; stop and start the container to understand the lifecycle.

Why it matters: Restart policies help recover from process exits but are not a substitute for application health checks.

Exercise 300 · Configure restart behavior

Task: Create a disposable service that restarts unless explicitly stopped, case 10.

Solution command

```
docker run -d --name restart10 --restart unless-stopped alpine:3.20 sh -c 'while true; do sleep 30; done'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.HostConfig.RestartPolicy.Name}' restart10`
4. Interpret the result: Inspect the restart policy; stop and start the container to understand the lifecycle.

Why it matters: Restart policies help recover from process exits but are not a substitute for application health checks.

Module 16: Volumes: create & inspect

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 301 · Create a named volume

Task: Create a named volume named data1.

Solution command

```
docker volume create data1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume inspect data1`
4. Interpret the result: Create the volume, then inspect its driver and mountpoint metadata.

Why it matters: Named volumes persist independently of a container lifecycle.

Exercise 302 · Create a named volume

Task: Create a named volume named data2.

Solution command

```
docker volume create data2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume inspect data2`
4. Interpret the result: Create the volume, then inspect its driver and mountpoint metadata.

Why it matters: Named volumes persist independently of a container lifecycle.

Exercise 303 · Create a named volume

Task: Create a named volume named data3.

Solution command

```
docker volume create data3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume inspect data3`
4. Interpret the result: Create the volume, then inspect its driver and mountpoint metadata.

Why it matters: Named volumes persist independently of a container lifecycle.

Exercise 304 · Create a named volume

Task: Create a named volume named data4.

Solution command

```
docker volume create data4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume inspect data4`
4. Interpret the result: Create the volume, then inspect its driver and mountpoint metadata.

Why it matters: Named volumes persist independently of a container lifecycle.

Exercise 305 · Create a named volume

Task: Create a named volume named data5.

Solution command

```
docker volume create data5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume inspect data5`
4. Interpret the result: Create the volume, then inspect its driver and mountpoint metadata.

Why it matters: Named volumes persist independently of a container lifecycle.

Exercise 306 · Create a named volume

Task: Create a named volume named data6.

Solution command

```
docker volume create data6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume inspect data6`
4. Interpret the result: Create the volume, then inspect its driver and mountpoint metadata.

Why it matters: Named volumes persist independently of a container lifecycle.

Exercise 307 · Create a named volume

Task: Create a named volume named data7.

Solution command

```
docker volume create data7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume inspect data7`
4. Interpret the result: Create the volume, then inspect its driver and mountpoint metadata.

Why it matters: Named volumes persist independently of a container lifecycle.

Exercise 308 · Create a named volume

Task: Create a named volume named data8.

Solution command

```
docker volume create data8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume inspect data8`
4. Interpret the result: Create the volume, then inspect its driver and mountpoint metadata.

Why it matters: Named volumes persist independently of a container lifecycle.

Exercise 309 · Create a named volume

Task: Create a named volume named data9.

Solution command

```
docker volume create data9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume inspect data9`
4. Interpret the result: Create the volume, then inspect its driver and mountpoint metadata.

Why it matters: Named volumes persist independently of a container lifecycle.

Exercise 310 · Create a named volume

Task: Create a named volume named data10.

Solution command

```
docker volume create data10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume inspect data10`
4. Interpret the result: Create the volume, then inspect its driver and mountpoint metadata.

Why it matters: Named volumes persist independently of a container lifecycle.

Exercise 311 · List and filter volumes

Task: Inventory volumes and identify data1 if it exists.

Solution command

```
docker volume ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data1`
4. Interpret the result: Compare the full list with the filtered result.

Why it matters: Listing is safe; removing a volume can permanently delete persistent data.

Exercise 312 · List and filter volumes

Task: Inventory volumes and identify data2 if it exists.

Solution command

```
docker volume ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data2`
4. Interpret the result: Compare the full list with the filtered result.

Why it matters: Listing is safe; removing a volume can permanently delete persistent data.

Exercise 313 · List and filter volumes

Task: Inventory volumes and identify data3 if it exists.

Solution command

```
docker volume ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data3`
4. Interpret the result: Compare the full list with the filtered result.

Why it matters: Listing is safe; removing a volume can permanently delete persistent data.

Exercise 314 · List and filter volumes

Task: Inventory volumes and identify data4 if it exists.

Solution command

```
docker volume ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data4`
4. Interpret the result: Compare the full list with the filtered result.

Why it matters: Listing is safe; removing a volume can permanently delete persistent data.

Exercise 315 · List and filter volumes

Task: Inventory volumes and identify data5 if it exists.

Solution command

```
docker volume ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data5`
4. Interpret the result: Compare the full list with the filtered result.

Why it matters: Listing is safe; removing a volume can permanently delete persistent data.

Exercise 316 · List and filter volumes

Task: Inventory volumes and identify data6 if it exists.

Solution command

```
docker volume ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data6`
4. Interpret the result: Compare the full list with the filtered result.

Why it matters: Listing is safe; removing a volume can permanently delete persistent data.

Exercise 317 · List and filter volumes

Task: Inventory volumes and identify data7 if it exists.

Solution command

```
docker volume ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data7`
4. Interpret the result: Compare the full list with the filtered result.

Why it matters: Listing is safe; removing a volume can permanently delete persistent data.

Exercise 318 · List and filter volumes

Task: Inventory volumes and identify data8 if it exists.

Solution command

```
docker volume ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data8`
4. Interpret the result: Compare the full list with the filtered result.

Why it matters: Listing is safe; removing a volume can permanently delete persistent data.

Exercise 319 · List and filter volumes

Task: Inventory volumes and identify data9 if it exists.

Solution command

```
docker volume ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data9`
4. Interpret the result: Compare the full list with the filtered result.

Why it matters: Listing is safe; removing a volume can permanently delete persistent data.

Exercise 320 · List and filter volumes

Task: Inventory volumes and identify data10 if it exists.

Solution command

```
docker volume ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data10`
4. Interpret the result: Compare the full list with the filtered result.

Why it matters: Listing is safe; removing a volume can permanently delete persistent data.

Module 17: Volumes: persistence & backup

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 321 · Write persistent data

Task: Write a marker into named volume data1 and read it from a second container.

Solution command

```
docker run --rm -v data1:/data alpine:3.20 sh -c 'echo case1 > /data/marker.txt && cat /data/marker.txt'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -v data1:/data alpine:3.20 cat /data/marker.txt`
4. Interpret the result: Both containers mount the same named volume at /data; the second container proves persistence.

Why it matters: Use a test volume, not a production data volume, for this exercise.

Exercise 322 · Write persistent data

Task: Write a marker into named volume data2 and read it from a second container.

Solution command

```
docker run --rm -v data2:/data alpine:3.20 sh -c 'echo case2 > /data/marker.txt && cat /data/marker.txt'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -v data2:/data alpine:3.20 cat /data/marker.txt`
4. Interpret the result: Both containers mount the same named volume at /data; the second container proves persistence.

Why it matters: Use a test volume, not a production data volume, for this exercise.

Exercise 323 · Write persistent data

Task: Write a marker into named volume data3 and read it from a second container.

Solution command

```
docker run --rm -v data3:/data alpine:3.20 sh -c 'echo case3 > /data/marker.txt && cat /data/marker.txt'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -v data3:/data alpine:3.20 cat /data/marker.txt`
4. Interpret the result: Both containers mount the same named volume at /data; the second container proves persistence.

Why it matters: Use a test volume, not a production data volume, for this exercise.

Exercise 324 · Write persistent data

Task: Write a marker into named volume data4 and read it from a second container.

Solution command

```
docker run --rm -v data4:/data alpine:3.20 sh -c 'echo case4 > /data/marker.txt && cat /data/marker.txt'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -v data4:/data alpine:3.20 cat /data/marker.txt`
4. Interpret the result: Both containers mount the same named volume at /data; the second container proves persistence.

Why it matters: Use a test volume, not a production data volume, for this exercise.

Exercise 325 · Write persistent data

Task: Write a marker into named volume data5 and read it from a second container.

Solution command

```
docker run --rm -v data5:/data alpine:3.20 sh -c 'echo case5 > /data/marker.txt && cat /data/marker.txt'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -v data5:/data alpine:3.20 cat /data/marker.txt`
4. Interpret the result: Both containers mount the same named volume at /data; the second container proves persistence.

Why it matters: Use a test volume, not a production data volume, for this exercise.

Exercise 326 · Write persistent data

Task: Write a marker into named volume data6 and read it from a second container.

Solution command

```
docker run --rm -v data6:/data alpine:3.20 sh -c 'echo case6 > /data/marker.txt && cat /data/marker.txt'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -v data6:/data alpine:3.20 cat /data/marker.txt`
4. Interpret the result: Both containers mount the same named volume at /data; the second container proves persistence.

Why it matters: Use a test volume, not a production data volume, for this exercise.

Exercise 327 · Write persistent data

Task: Write a marker into named volume data7 and read it from a second container.

Solution command

```
docker run --rm -v data7:/data alpine:3.20 sh -c 'echo case7 > /data/marker.txt && cat /data/marker.txt'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -v data7:/data alpine:3.20 cat /data/marker.txt`
4. Interpret the result: Both containers mount the same named volume at /data; the second container proves persistence.

Why it matters: Use a test volume, not a production data volume, for this exercise.

Exercise 328 · Write persistent data

Task: Write a marker into named volume data8 and read it from a second container.

Solution command

```
docker run --rm -v data8:/data alpine:3.20 sh -c 'echo case8 > /data/marker.txt && cat /data/marker.txt'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -v data8:/data alpine:3.20 cat /data/marker.txt`
4. Interpret the result: Both containers mount the same named volume at /data; the second container proves persistence.

Why it matters: Use a test volume, not a production data volume, for this exercise.

Exercise 329 · Write persistent data

Task: Write a marker into named volume data9 and read it from a second container.

Solution command

```
docker run --rm -v data9:/data alpine:3.20 sh -c 'echo case9 > /data/marker.txt && cat /data/marker.txt'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -v data9:/data alpine:3.20 cat /data/marker.txt`
4. Interpret the result: Both containers mount the same named volume at /data; the second container proves persistence.

Why it matters: Use a test volume, not a production data volume, for this exercise.

Exercise 330 · Write persistent data

Task: Write a marker into named volume data10 and read it from a second container.

Solution command

```
docker run --rm -v data10:/data alpine:3.20 sh -c 'echo case10 > /data/marker.txt && cat /data/marker.txt'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm -v data10:/data alpine:3.20 cat /data/marker.txt`
4. Interpret the result: Both containers mount the same named volume at /data; the second container proves persistence.

Why it matters: Use a test volume, not a production data volume, for this exercise.

Exercise 331 · Copy volume contents to a host archive

Task: Back up data1 into ./data1.tar using a temporary helper container.

Solution command

```
docker run --rm -v data1:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/data1.tar -C /src .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh data1.tar`
4. Interpret the result: Mount the volume readably at /src and the current host directory at /backup, then archive the files.

Why it matters: Restore drills should be tested separately; ensure the archive path is writable on your platform.

Exercise 332 · Copy volume contents to a host archive

Task: Back up data2 into ./data2.tar using a temporary helper container.

Solution command

```
docker run --rm -v data2:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/data2.tar -C /src .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh data2.tar`
4. Interpret the result: Mount the volume readably at /src and the current host directory at /backup, then archive the files.

Why it matters: Restore drills should be tested separately; ensure the archive path is writable on your platform.

Exercise 333 · Copy volume contents to a host archive

Task: Back up data3 into ./data3.tar using a temporary helper container.

Solution command

```
docker run --rm -v data3:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/data3.tar -C /src .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh data3.tar`
4. Interpret the result: Mount the volume readably at /src and the current host directory at /backup, then archive the files.

Why it matters: Restore drills should be tested separately; ensure the archive path is writable on your platform.

Exercise 334 · Copy volume contents to a host archive

Task: Back up data4 into ./data4.tar using a temporary helper container.

Solution command

```
docker run --rm -v data4:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/data4.tar -C /src .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh data4.tar`
4. Interpret the result: Mount the volume readably at /src and the current host directory at /backup, then archive the files.

Why it matters: Restore drills should be tested separately; ensure the archive path is writable on your platform.

Exercise 335 · Copy volume contents to a host archive

Task: Back up data5 into ./data5.tar using a temporary helper container.

Solution command

```
docker run --rm -v data5:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/data5.tar -C /src .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh data5.tar`
4. Interpret the result: Mount the volume readably at /src and the current host directory at /backup, then archive the files.

Why it matters: Restore drills should be tested separately; ensure the archive path is writable on your platform.

Exercise 336 · Copy volume contents to a host archive

Task: Back up data6 into ./data6.tar using a temporary helper container.

Solution command

```
docker run --rm -v data6:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/data6.tar -C /src .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh data6.tar`
4. Interpret the result: Mount the volume readably at /src and the current host directory at /backup, then archive the files.

Why it matters: Restore drills should be tested separately; ensure the archive path is writable on your platform.

Exercise 337 · Copy volume contents to a host archive

Task: Back up data7 into ./data7.tar using a temporary helper container.

Solution command

```
docker run --rm -v data7:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/data7.tar -C /src .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh data7.tar`
4. Interpret the result: Mount the volume readably at /src and the current host directory at /backup, then archive the files.

Why it matters: Restore drills should be tested separately; ensure the archive path is writable on your platform.

Exercise 338 · Copy volume contents to a host archive

Task: Back up data8 into ./data8.tar using a temporary helper container.

Solution command

```
docker run --rm -v data8:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/data8.tar -C /src .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh data8.tar`
4. Interpret the result: Mount the volume readably at /src and the current host directory at /backup, then archive the files.

Why it matters: Restore drills should be tested separately; ensure the archive path is writable on your platform.

Exercise 339 · Copy volume contents to a host archive

Task: Back up data9 into ./data9.tar using a temporary helper container.

Solution command

```
docker run --rm -v data9:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/data9.tar -C /src .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh data9.tar`
4. Interpret the result: Mount the volume readably at /src and the current host directory at /backup, then archive the files.

Why it matters: Restore drills should be tested separately; ensure the archive path is writable on your platform.

Exercise 340 · Copy volume contents to a host archive

Task: Back up data10 into ./data10.tar using a temporary helper container.

Solution command

```
docker run --rm -v data10:/src -v "$PWD":/backup alpine:3.20 tar -cf /backup/data10.tar -C /src .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh data10.tar`
4. Interpret the result: Mount the volume readably at /src and the current host directory at /backup, then archive the files.

Why it matters: Restore drills should be tested separately; ensure the archive path is writable on your platform.

Module 18: Networks: create & inspect

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 341 · Create a bridge network

Task: Create bridge network labnet1.

Solution command

```
docker network create --driver bridge labnet1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet1`
4. Interpret the result: Inspect driver, subnet, gateway, and connected containers.

Why it matters: Bridge networks are commonly used for communication among containers on one Docker host.

Exercise 342 · Create a bridge network

Task: Create bridge network labnet2.

Solution command

```
docker network create --driver bridge labnet2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet2`
4. Interpret the result: Inspect driver, subnet, gateway, and connected containers.

Why it matters: Bridge networks are commonly used for communication among containers on one Docker host.

Exercise 343 · Create a bridge network

Task: Create bridge network labnet3.

Solution command

```
docker network create --driver bridge labnet3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet3`
4. Interpret the result: Inspect driver, subnet, gateway, and connected containers.

Why it matters: Bridge networks are commonly used for communication among containers on one Docker host.

Exercise 344 · Create a bridge network

Task: Create bridge network labnet4.

Solution command

```
docker network create --driver bridge labnet4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet4`
4. Interpret the result: Inspect driver, subnet, gateway, and connected containers.

Why it matters: Bridge networks are commonly used for communication among containers on one Docker host.

Exercise 345 · Create a bridge network

Task: Create bridge network labnet5.

Solution command

```
docker network create --driver bridge labnet5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet5`
4. Interpret the result: Inspect driver, subnet, gateway, and connected containers.

Why it matters: Bridge networks are commonly used for communication among containers on one Docker host.

Exercise 346 · Create a bridge network

Task: Create bridge network labnet6.

Solution command

```
docker network create --driver bridge labnet6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet6`
4. Interpret the result: Inspect driver, subnet, gateway, and connected containers.

Why it matters: Bridge networks are commonly used for communication among containers on one Docker host.

Exercise 347 · Create a bridge network

Task: Create bridge network labnet7.

Solution command

```
docker network create --driver bridge labnet7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet7`
4. Interpret the result: Inspect driver, subnet, gateway, and connected containers.

Why it matters: Bridge networks are commonly used for communication among containers on one Docker host.

Exercise 348 · Create a bridge network

Task: Create bridge network labnet8.

Solution command

```
docker network create --driver bridge labnet8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet8`
4. Interpret the result: Inspect driver, subnet, gateway, and connected containers.

Why it matters: Bridge networks are commonly used for communication among containers on one Docker host.

Exercise 349 · Create a bridge network

Task: Create bridge network labnet9.

Solution command

```
docker network create --driver bridge labnet9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet9`
4. Interpret the result: Inspect driver, subnet, gateway, and connected containers.

Why it matters: Bridge networks are commonly used for communication among containers on one Docker host.

Exercise 350 · Create a bridge network

Task: Create bridge network labnet10.

Solution command

```
docker network create --driver bridge labnet10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet10`
4. Interpret the result: Inspect driver, subnet, gateway, and connected containers.

Why it matters: Bridge networks are commonly used for communication among containers on one Docker host.

Exercise 351 · List networks

Task: List networks and inspect the built-in bridge for networking case 1.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect bridge`
4. Interpret the result: Identify network IDs, names, drivers, and scope.

Why it matters: Do not delete built-in networks as part of a routine lab.

Exercise 352 · List networks

Task: List networks and inspect the built-in bridge for networking case 2.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect bridge`
4. Interpret the result: Identify network IDs, names, drivers, and scope.

Why it matters: Do not delete built-in networks as part of a routine lab.

Exercise 353 · List networks

Task: List networks and inspect the built-in bridge for networking case 3.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect bridge`
4. Interpret the result: Identify network IDs, names, drivers, and scope.

Why it matters: Do not delete built-in networks as part of a routine lab.

Exercise 354 · List networks

Task: List networks and inspect the built-in bridge for networking case 4.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect bridge`
4. Interpret the result: Identify network IDs, names, drivers, and scope.

Why it matters: Do not delete built-in networks as part of a routine lab.

Exercise 355 · List networks

Task: List networks and inspect the built-in bridge for networking case 5.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect bridge`
4. Interpret the result: Identify network IDs, names, drivers, and scope.

Why it matters: Do not delete built-in networks as part of a routine lab.

Exercise 356 · List networks

Task: List networks and inspect the built-in bridge for networking case 6.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect bridge`
4. Interpret the result: Identify network IDs, names, drivers, and scope.

Why it matters: Do not delete built-in networks as part of a routine lab.

Exercise 357 · List networks

Task: List networks and inspect the built-in bridge for networking case 7.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect bridge`
4. Interpret the result: Identify network IDs, names, drivers, and scope.

Why it matters: Do not delete built-in networks as part of a routine lab.

Exercise 358 · List networks

Task: List networks and inspect the built-in bridge for networking case 8.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect bridge`
4. Interpret the result: Identify network IDs, names, drivers, and scope.

Why it matters: Do not delete built-in networks as part of a routine lab.

Exercise 359 · List networks

Task: List networks and inspect the built-in bridge for networking case 9.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect bridge`
4. Interpret the result: Identify network IDs, names, drivers, and scope.

Why it matters: Do not delete built-in networks as part of a routine lab.

Exercise 360 · List networks

Task: List networks and inspect the built-in bridge for networking case 10.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect bridge`
4. Interpret the result: Identify network IDs, names, drivers, and scope.

Why it matters: Do not delete built-in networks as part of a routine lab.

Module 19: Networks: connect & disconnect

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 361 · Connect a running container

Task: Connect app1 to labnet1.

Solution command

```
docker network connect labnet1 app1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet1`
4. Interpret the result: Check the network's Containers section to verify attachment.

Why it matters: The network and container must already exist.

Exercise 362 · Connect a running container

Task: Connect app2 to labnet2.

Solution command

```
docker network connect labnet2 app2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet2`
4. Interpret the result: Check the network's Containers section to verify attachment.

Why it matters: The network and container must already exist.

Exercise 363 · Connect a running container

Task: Connect app3 to labnet3.

Solution command

```
docker network connect labnet3 app3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet3`
4. Interpret the result: Check the network's Containers section to verify attachment.

Why it matters: The network and container must already exist.

Exercise 364 · Connect a running container

Task: Connect app4 to labnet4.

Solution command

```
docker network connect labnet4 app4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet4`
4. Interpret the result: Check the network's Containers section to verify attachment.

Why it matters: The network and container must already exist.

Exercise 365 · Connect a running container

Task: Connect app5 to labnet5.

Solution command

```
docker network connect labnet5 app5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet5`
4. Interpret the result: Check the network's Containers section to verify attachment.

Why it matters: The network and container must already exist.

Exercise 366 · Connect a running container

Task: Connect app6 to labnet6.

Solution command

```
docker network connect labnet6 app6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet6`
4. Interpret the result: Check the network's Containers section to verify attachment.

Why it matters: The network and container must already exist.

Exercise 367 · Connect a running container

Task: Connect app7 to labnet7.

Solution command

```
docker network connect labnet7 app7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet7`
4. Interpret the result: Check the network's Containers section to verify attachment.

Why it matters: The network and container must already exist.

Exercise 368 · Connect a running container

Task: Connect app8 to labnet8.

Solution command

```
docker network connect labnet8 app8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet8`
4. Interpret the result: Check the network's Containers section to verify attachment.

Why it matters: The network and container must already exist.

Exercise 369 · Connect a running container

Task: Connect app9 to labnet9.

Solution command

```
docker network connect labnet9 app9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet9`
4. Interpret the result: Check the network's Containers section to verify attachment.

Why it matters: The network and container must already exist.

Exercise 370 · Connect a running container

Task: Connect app10 to labnet10.

Solution command

```
docker network connect labnet10 app10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet10`
4. Interpret the result: Check the network's Containers section to verify attachment.

Why it matters: The network and container must already exist.

Exercise 371 · Disconnect a container from a network

Task: Disconnect test container app1 from labnet1.

Solution command

```
docker network disconnect labnet1 app1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet1`
4. Interpret the result: Verify that app1 is no longer listed as attached.

Why it matters: Only disconnect a test container; this can interrupt application connectivity.

Exercise 372 · Disconnect a container from a network

Task: Disconnect test container app2 from labnet2.

Solution command

```
docker network disconnect labnet2 app2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet2`
4. Interpret the result: Verify that app2 is no longer listed as attached.

Why it matters: Only disconnect a test container; this can interrupt application connectivity.

Exercise 373 · Disconnect a container from a network

Task: Disconnect test container app3 from labnet3.

Solution command

```
docker network disconnect labnet3 app3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet3`
4. Interpret the result: Verify that app3 is no longer listed as attached.

Why it matters: Only disconnect a test container; this can interrupt application connectivity.

Exercise 374 · Disconnect a container from a network

Task: Disconnect test container app4 from labnet4.

Solution command

```
docker network disconnect labnet4 app4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet4`
4. Interpret the result: Verify that app4 is no longer listed as attached.

Why it matters: Only disconnect a test container; this can interrupt application connectivity.

Exercise 375 · Disconnect a container from a network

Task: Disconnect test container app5 from labnet5.

Solution command

```
docker network disconnect labnet5 app5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet5`
4. Interpret the result: Verify that app5 is no longer listed as attached.

Why it matters: Only disconnect a test container; this can interrupt application connectivity.

Exercise 376 · Disconnect a container from a network

Task: Disconnect test container app6 from labnet6.

Solution command

```
docker network disconnect labnet6 app6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet6`
4. Interpret the result: Verify that app6 is no longer listed as attached.

Why it matters: Only disconnect a test container; this can interrupt application connectivity.

Exercise 377 · Disconnect a container from a network

Task: Disconnect test container app7 from labnet7.

Solution command

```
docker network disconnect labnet7 app7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet7`
4. Interpret the result: Verify that app7 is no longer listed as attached.

Why it matters: Only disconnect a test container; this can interrupt application connectivity.

Exercise 378 · Disconnect a container from a network

Task: Disconnect test container app8 from labnet8.

Solution command

```
docker network disconnect labnet8 app8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet8`
4. Interpret the result: Verify that app8 is no longer listed as attached.

Why it matters: Only disconnect a test container; this can interrupt application connectivity.

Exercise 379 · Disconnect a container from a network

Task: Disconnect test container app9 from labnet9.

Solution command

```
docker network disconnect labnet9 app9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet9`
4. Interpret the result: Verify that app9 is no longer listed as attached.

Why it matters: Only disconnect a test container; this can interrupt application connectivity.

Exercise 380 · Disconnect a container from a network

Task: Disconnect test container app10 from labnet10.

Solution command

```
docker network disconnect labnet10 app10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet10`
4. Interpret the result: Verify that app10 is no longer listed as attached.

Why it matters: Only disconnect a test container; this can interrupt application connectivity.

Module 20: Image builds: build & cache

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 381 · Build an image from a Dockerfile

Task: Build the current directory as local/demo:case1.

Solution command

```
docker build -t local/demo:case1 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case1`
4. Interpret the result: Ensure the directory contains a valid Dockerfile and review the build output for failures.

Why it matters: The final dot is the build context and determines which files are available to COPY.

Exercise 382 · Build an image from a Dockerfile

Task: Build the current directory as local/demo:case2.

Solution command

```
docker build -t local/demo:case2 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case2`
4. Interpret the result: Ensure the directory contains a valid Dockerfile and review the build output for failures.

Why it matters: The final dot is the build context and determines which files are available to COPY.

Exercise 383 · Build an image from a Dockerfile

Task: Build the current directory as local/demo:case3.

Solution command

```
docker build -t local/demo:case3 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case3`
4. Interpret the result: Ensure the directory contains a valid Dockerfile and review the build output for failures.

Why it matters: The final dot is the build context and determines which files are available to COPY.

Exercise 384 · Build an image from a Dockerfile

Task: Build the current directory as local/demo:case4.

Solution command

```
docker build -t local/demo:case4 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case4`
4. Interpret the result: Ensure the directory contains a valid Dockerfile and review the build output for failures.

Why it matters: The final dot is the build context and determines which files are available to COPY.

Exercise 385 · Build an image from a Dockerfile

Task: Build the current directory as local/demo:case5.

Solution command

```
docker build -t local/demo:case5 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case5`
4. Interpret the result: Ensure the directory contains a valid Dockerfile and review the build output for failures.

Why it matters: The final dot is the build context and determines which files are available to COPY.

Exercise 386 · Build an image from a Dockerfile

Task: Build the current directory as local/demo:case6.

Solution command

```
docker build -t local/demo:case6 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case6`
4. Interpret the result: Ensure the directory contains a valid Dockerfile and review the build output for failures.

Why it matters: The final dot is the build context and determines which files are available to COPY.

Exercise 387 · Build an image from a Dockerfile

Task: Build the current directory as local/demo:case7.

Solution command

```
docker build -t local/demo:case7 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case7`
4. Interpret the result: Ensure the directory contains a valid Dockerfile and review the build output for failures.

Why it matters: The final dot is the build context and determines which files are available to COPY.

Exercise 388 · Build an image from a Dockerfile

Task: Build the current directory as local/demo:case8.

Solution command

```
docker build -t local/demo:case8 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case8`
4. Interpret the result: Ensure the directory contains a valid Dockerfile and review the build output for failures.

Why it matters: The final dot is the build context and determines which files are available to COPY.

Exercise 389 · Build an image from a Dockerfile

Task: Build the current directory as local/demo:case9.

Solution command

```
docker build -t local/demo:case9 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case9`
4. Interpret the result: Ensure the directory contains a valid Dockerfile and review the build output for failures.

Why it matters: The final dot is the build context and determines which files are available to COPY.

Exercise 390 · Build an image from a Dockerfile

Task: Build the current directory as local/demo:case10.

Solution command

```
docker build -t local/demo:case10 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case10`
4. Interpret the result: Ensure the directory contains a valid Dockerfile and review the build output for failures.

Why it matters: The final dot is the build context and determines which files are available to COPY.

Exercise 391 · Build without using cache

Task: Rebuild local/demo:case1 without reusing cached layers.

Solution command

```
docker build --no-cache -t local/demo:case1 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case1`
4. Interpret the result: Compare the output with a normal build and confirm the final tag.

Why it matters: A no-cache build can be slower; it does not automatically refresh every base image unless requested.

Exercise 392 · Build without using cache

Task: Rebuild local/demo:case2 without reusing cached layers.

Solution command

```
docker build --no-cache -t local/demo:case2 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case2`
4. Interpret the result: Compare the output with a normal build and confirm the final tag.

Why it matters: A no-cache build can be slower; it does not automatically refresh every base image unless requested.

Exercise 393 · Build without using cache

Task: Rebuild local/demo:case3 without reusing cached layers.

Solution command

```
docker build --no-cache -t local/demo:case3 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case3`
4. Interpret the result: Compare the output with a normal build and confirm the final tag.

Why it matters: A no-cache build can be slower; it does not automatically refresh every base image unless requested.

Exercise 394 · Build without using cache

Task: Rebuild local/demo:case4 without reusing cached layers.

Solution command

```
docker build --no-cache -t local/demo:case4 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case4`
4. Interpret the result: Compare the output with a normal build and confirm the final tag.

Why it matters: A no-cache build can be slower; it does not automatically refresh every base image unless requested.

Exercise 395 · Build without using cache

Task: Rebuild local/demo:case5 without reusing cached layers.

Solution command

```
docker build --no-cache -t local/demo:case5 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case5`
4. Interpret the result: Compare the output with a normal build and confirm the final tag.

Why it matters: A no-cache build can be slower; it does not automatically refresh every base image unless requested.

Exercise 396 · Build without using cache

Task: Rebuild local/demo:case6 without reusing cached layers.

Solution command

```
docker build --no-cache -t local/demo:case6 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case6`
4. Interpret the result: Compare the output with a normal build and confirm the final tag.

Why it matters: A no-cache build can be slower; it does not automatically refresh every base image unless requested.

Exercise 397 · Build without using cache

Task: Rebuild local/demo:case7 without reusing cached layers.

Solution command

```
docker build --no-cache -t local/demo:case7 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case7`
4. Interpret the result: Compare the output with a normal build and confirm the final tag.

Why it matters: A no-cache build can be slower; it does not automatically refresh every base image unless requested.

Exercise 398 · Build without using cache

Task: Rebuild local/demo:case8 without reusing cached layers.

Solution command

```
docker build --no-cache -t local/demo:case8 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case8`
4. Interpret the result: Compare the output with a normal build and confirm the final tag.

Why it matters: A no-cache build can be slower; it does not automatically refresh every base image unless requested.

Exercise 399 · Build without using cache

Task: Rebuild local/demo:case9 without reusing cached layers.

Solution command

```
docker build --no-cache -t local/demo:case9 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case9`
4. Interpret the result: Compare the output with a normal build and confirm the final tag.

Why it matters: A no-cache build can be slower; it does not automatically refresh every base image unless requested.

Exercise 400 · Build without using cache

Task: Rebuild local/demo:case10 without reusing cached layers.

Solution command

```
docker build --no-cache -t local/demo:case10 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case10`
4. Interpret the result: Compare the output with a normal build and confirm the final tag.

Why it matters: A no-cache build can be slower; it does not automatically refresh every base image unless requested.

Module 21: Build arguments, targets & BuildKit

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 401 · Pass a build argument

Task: Build local/demo:case1 with build argument APP_VERSION=case1.

Solution command

```
docker build --build-arg APP_VERSION=case1 -t local/demo:case1 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case1`
4. Interpret the result: Confirm the Dockerfile declares ARG APP_VERSION and uses it where intended.

Why it matters: ARG is a build-time parameter, not a secure place for secrets.

Exercise 402 · Pass a build argument

Task: Build local/demo:case2 with build argument APP_VERSION=case2.

Solution command

```
docker build --build-arg APP_VERSION=case2 -t local/demo:case2 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case2`
4. Interpret the result: Confirm the Dockerfile declares ARG APP_VERSION and uses it where intended.

Why it matters: ARG is a build-time parameter, not a secure place for secrets.

Exercise 403 · Pass a build argument

Task: Build local/demo:case3 with build argument APP_VERSION=case3.

Solution command

```
docker build --build-arg APP_VERSION=case3 -t local/demo:case3 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case3`
4. Interpret the result: Confirm the Dockerfile declares ARG APP_VERSION and uses it where intended.

Why it matters: ARG is a build-time parameter, not a secure place for secrets.

Exercise 404 · Pass a build argument

Task: Build local/demo:case4 with build argument APP_VERSION=case4.

Solution command

```
docker build --build-arg APP_VERSION=case4 -t local/demo:case4 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case4`
4. Interpret the result: Confirm the Dockerfile declares ARG APP_VERSION and uses it where intended.

Why it matters: ARG is a build-time parameter, not a secure place for secrets.

Exercise 405 · Pass a build argument

Task: Build local/demo:case5 with build argument APP_VERSION=case5.

Solution command

```
docker build --build-arg APP_VERSION=case5 -t local/demo:case5 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case5`
4. Interpret the result: Confirm the Dockerfile declares ARG APP_VERSION and uses it where intended.

Why it matters: ARG is a build-time parameter, not a secure place for secrets.

Exercise 406 · Pass a build argument

Task: Build local/demo:case6 with build argument APP_VERSION=case6.

Solution command

```
docker build --build-arg APP_VERSION=case6 -t local/demo:case6 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case6`
4. Interpret the result: Confirm the Dockerfile declares ARG APP_VERSION and uses it where intended.

Why it matters: ARG is a build-time parameter, not a secure place for secrets.

Exercise 407 · Pass a build argument

Task: Build local/demo:case7 with build argument APP_VERSION=case7.

Solution command

```
docker build --build-arg APP_VERSION=case7 -t local/demo:case7 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case7`
4. Interpret the result: Confirm the Dockerfile declares ARG APP_VERSION and uses it where intended.

Why it matters: ARG is a build-time parameter, not a secure place for secrets.

Exercise 408 · Pass a build argument

Task: Build local/demo:case8 with build argument APP_VERSION=case8.

Solution command

```
docker build --build-arg APP_VERSION=case8 -t local/demo:case8 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case8`
4. Interpret the result: Confirm the Dockerfile declares ARG APP_VERSION and uses it where intended.

Why it matters: ARG is a build-time parameter, not a secure place for secrets.

Exercise 409 · Pass a build argument

Task: Build local/demo:case9 with build argument APP_VERSION=case9.

Solution command

```
docker build --build-arg APP_VERSION=case9 -t local/demo:case9 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case9`
4. Interpret the result: Confirm the Dockerfile declares ARG APP_VERSION and uses it where intended.

Why it matters: ARG is a build-time parameter, not a secure place for secrets.

Exercise 410 · Pass a build argument

Task: Build local/demo:case10 with build argument APP_VERSION=case10.

Solution command

```
docker build --build-arg APP_VERSION=case10 -t local/demo:case10 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/demo:case10`
4. Interpret the result: Confirm the Dockerfile declares ARG APP_VERSION and uses it where intended.

Why it matters: ARG is a build-time parameter, not a secure place for secrets.

Exercise 411 · Build a named stage

Task: Build only the stage named test from the current Dockerfile.

Solution command

```
docker build --target test -t local/demo-test:case1 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/demo-test`
4. Interpret the result: The Dockerfile must define a stage with AS test; inspect its FROM ... AS ... line.

Why it matters: Multi-stage builds let you target a particular stage and keep build tools out of the runtime image.

Exercise 412 · Build a named stage

Task: Build only the stage named test from the current Dockerfile.

Solution command

```
docker build --target test -t local/demo-test:case2 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/demo-test`
4. Interpret the result: The Dockerfile must define a stage with AS test; inspect its FROM ... AS ... line.

Why it matters: Multi-stage builds let you target a particular stage and keep build tools out of the runtime image.

Exercise 413 · Build a named stage

Task: Build only the stage named test from the current Dockerfile.

Solution command

```
docker build --target test -t local/demo-test:case3 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/demo-test`
4. Interpret the result: The Dockerfile must define a stage with AS test; inspect its FROM ... AS ... line.

Why it matters: Multi-stage builds let you target a particular stage and keep build tools out of the runtime image.

Exercise 414 · Build a named stage

Task: Build only the stage named test from the current Dockerfile.

Solution command

```
docker build --target test -t local/demo-test:case4 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/demo-test`
4. Interpret the result: The Dockerfile must define a stage with AS test; inspect its FROM ... AS ... line.

Why it matters: Multi-stage builds let you target a particular stage and keep build tools out of the runtime image.

Exercise 415 · Build a named stage

Task: Build only the stage named test from the current Dockerfile.

Solution command

```
docker build --target test -t local/demo-test:case5 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/demo-test`
4. Interpret the result: The Dockerfile must define a stage with AS test; inspect its FROM ... AS ... line.

Why it matters: Multi-stage builds let you target a particular stage and keep build tools out of the runtime image.

Exercise 416 · Build a named stage

Task: Build only the stage named test from the current Dockerfile.

Solution command

```
docker build --target test -t local/demo-test:case6 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/demo-test`
4. Interpret the result: The Dockerfile must define a stage with AS test; inspect its FROM ... AS ... line.

Why it matters: Multi-stage builds let you target a particular stage and keep build tools out of the runtime image.

Exercise 417 · Build a named stage

Task: Build only the stage named test from the current Dockerfile.

Solution command

```
docker build --target test -t local/demo-test:case7 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/demo-test`
4. Interpret the result: The Dockerfile must define a stage with AS test; inspect its FROM ... AS ... line.

Why it matters: Multi-stage builds let you target a particular stage and keep build tools out of the runtime image.

Exercise 418 · Build a named stage

Task: Build only the stage named test from the current Dockerfile.

Solution command

```
docker build --target test -t local/demo-test:case8 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/demo-test`
4. Interpret the result: The Dockerfile must define a stage with AS test; inspect its FROM ... AS ... line.

Why it matters: Multi-stage builds let you target a particular stage and keep build tools out of the runtime image.

Exercise 419 · Build a named stage

Task: Build only the stage named test from the current Dockerfile.

Solution command

```
docker build --target test -t local/demo-test:case9 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/demo-test`
4. Interpret the result: The Dockerfile must define a stage with AS test; inspect its FROM ... AS ... line.

Why it matters: Multi-stage builds let you target a particular stage and keep build tools out of the runtime image.

Exercise 420 · Build a named stage

Task: Build only the stage named test from the current Dockerfile.

Solution command

```
docker build --target test -t local/demo-test:case10 .
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/demo-test`
4. Interpret the result: The Dockerfile must define a stage with AS test; inspect its FROM ... AS ... line.

Why it matters: Multi-stage builds let you target a particular stage and keep build tools out of the runtime image.

Module 22: Docker Compose: validate & launch

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 421 · Validate a Compose model

Task: Render and validate compose.yaml for project case1.

Solution command

```
docker compose -f compose.yaml config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml config --services`
4. Interpret the result: Run from the directory containing compose.yaml or pass the correct file path.

Why it matters: Config output may include resolved environment values; treat it as potentially sensitive.

Exercise 422 · Validate a Compose model

Task: Render and validate compose.yaml for project case2.

Solution command

```
docker compose -f compose.yaml config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml config --services`
4. Interpret the result: Run from the directory containing compose.yaml or pass the correct file path.

Why it matters: Config output may include resolved environment values; treat it as potentially sensitive.

Exercise 423 · Validate a Compose model

Task: Render and validate compose.yaml for project case3.

Solution command

```
docker compose -f compose.yaml config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml config --services`
4. Interpret the result: Run from the directory containing compose.yaml or pass the correct file path.

Why it matters: Config output may include resolved environment values; treat it as potentially sensitive.

Exercise 424 · Validate a Compose model

Task: Render and validate compose.yaml for project case4.

Solution command

```
docker compose -f compose.yaml config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml config --services`
4. Interpret the result: Run from the directory containing compose.yaml or pass the correct file path.

Why it matters: Config output may include resolved environment values; treat it as potentially sensitive.

Exercise 425 · Validate a Compose model

Task: Render and validate compose.yaml for project case5.

Solution command

```
docker compose -f compose.yaml config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml config --services`
4. Interpret the result: Run from the directory containing compose.yaml or pass the correct file path.

Why it matters: Config output may include resolved environment values; treat it as potentially sensitive.

Exercise 426 · Validate a Compose model

Task: Render and validate compose.yaml for project case6.

Solution command

```
docker compose -f compose.yaml config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml config --services`
4. Interpret the result: Run from the directory containing compose.yaml or pass the correct file path.

Why it matters: Config output may include resolved environment values; treat it as potentially sensitive.

Exercise 427 · Validate a Compose model

Task: Render and validate compose.yaml for project case7.

Solution command

```
docker compose -f compose.yaml config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml config --services`
4. Interpret the result: Run from the directory containing compose.yaml or pass the correct file path.

Why it matters: Config output may include resolved environment values; treat it as potentially sensitive.

Exercise 428 · Validate a Compose model

Task: Render and validate compose.yaml for project case8.

Solution command

```
docker compose -f compose.yaml config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml config --services`
4. Interpret the result: Run from the directory containing compose.yaml or pass the correct file path.

Why it matters: Config output may include resolved environment values; treat it as potentially sensitive.

Exercise 429 · Validate a Compose model

Task: Render and validate compose.yaml for project case9.

Solution command

```
docker compose -f compose.yaml config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml config --services`
4. Interpret the result: Run from the directory containing compose.yaml or pass the correct file path.

Why it matters: Config output may include resolved environment values; treat it as potentially sensitive.

Exercise 430 · Validate a Compose model

Task: Render and validate compose.yaml for project case10.

Solution command

```
docker compose -f compose.yaml config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml config --services`
4. Interpret the result: Run from the directory containing compose.yaml or pass the correct file path.

Why it matters: Config output may include resolved environment values; treat it as potentially sensitive.

Exercise 431 · Start a Compose project

Task: Build and start the services defined in compose.yaml for case1.

Solution command

```
docker compose -f compose.yaml up -d --build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml ps`
4. Interpret the result: Compose builds if needed and starts services in the background; check service state.

Why it matters: Use the project directory or -p to keep lab projects isolated.

Exercise 432 · Start a Compose project

Task: Build and start the services defined in compose.yaml for case2.

Solution command

```
docker compose -f compose.yaml up -d --build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml ps`
4. Interpret the result: Compose builds if needed and starts services in the background; check service state.

Why it matters: Use the project directory or -p to keep lab projects isolated.

Exercise 433 · Start a Compose project

Task: Build and start the services defined in compose.yaml for case3.

Solution command

```
docker compose -f compose.yaml up -d --build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml ps`
4. Interpret the result: Compose builds if needed and starts services in the background; check service state.

Why it matters: Use the project directory or -p to keep lab projects isolated.

Exercise 434 · Start a Compose project

Task: Build and start the services defined in compose.yaml for case4.

Solution command

```
docker compose -f compose.yaml up -d --build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml ps`
4. Interpret the result: Compose builds if needed and starts services in the background; check service state.

Why it matters: Use the project directory or -p to keep lab projects isolated.

Exercise 435 · Start a Compose project

Task: Build and start the services defined in compose.yaml for case5.

Solution command

```
docker compose -f compose.yaml up -d --build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml ps`
4. Interpret the result: Compose builds if needed and starts services in the background; check service state.

Why it matters: Use the project directory or -p to keep lab projects isolated.

Exercise 436 · Start a Compose project

Task: Build and start the services defined in compose.yaml for case6.

Solution command

```
docker compose -f compose.yaml up -d --build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml ps`
4. Interpret the result: Compose builds if needed and starts services in the background; check service state.

Why it matters: Use the project directory or -p to keep lab projects isolated.

Exercise 437 · Start a Compose project

Task: Build and start the services defined in compose.yaml for case7.

Solution command

```
docker compose -f compose.yaml up -d --build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml ps`
4. Interpret the result: Compose builds if needed and starts services in the background; check service state.

Why it matters: Use the project directory or -p to keep lab projects isolated.

Exercise 438 · Start a Compose project

Task: Build and start the services defined in compose.yaml for case8.

Solution command

```
docker compose -f compose.yaml up -d --build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml ps`
4. Interpret the result: Compose builds if needed and starts services in the background; check service state.

Why it matters: Use the project directory or -p to keep lab projects isolated.

Exercise 439 · Start a Compose project

Task: Build and start the services defined in compose.yaml for case9.

Solution command

```
docker compose -f compose.yaml up -d --build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml ps`
4. Interpret the result: Compose builds if needed and starts services in the background; check service state.

Why it matters: Use the project directory or -p to keep lab projects isolated.

Exercise 440 · Start a Compose project

Task: Build and start the services defined in compose.yaml for case10.

Solution command

```
docker compose -f compose.yaml up -d --build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose -f compose.yaml ps`
4. Interpret the result: Compose builds if needed and starts services in the background; check service state.

Why it matters: Use the project directory or -p to keep lab projects isolated.

Module 23: Docker Compose: observe services

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 441 · List Compose services

Task: List service and container status for the current Compose project, case1.

Solution command

```
docker compose ps
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Compare running services with all created service containers.

Why it matters: Compose commands should be run from the project directory or with -f.

Exercise 442 · List Compose services

Task: List service and container status for the current Compose project, case2.

Solution command

```
docker compose ps
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Compare running services with all created service containers.

Why it matters: Compose commands should be run from the project directory or with -f.

Exercise 443 · List Compose services

Task: List service and container status for the current Compose project, case3.

Solution command

```
docker compose ps
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Compare running services with all created service containers.

Why it matters: Compose commands should be run from the project directory or with -f.

Exercise 444 · List Compose services

Task: List service and container status for the current Compose project, case4.

Solution command

```
docker compose ps
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Compare running services with all created service containers.

Why it matters: Compose commands should be run from the project directory or with -f.

Exercise 445 · List Compose services

Task: List service and container status for the current Compose project, case5.

Solution command

```
docker compose ps
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Compare running services with all created service containers.

Why it matters: Compose commands should be run from the project directory or with -f.

Exercise 446 · List Compose services

Task: List service and container status for the current Compose project, case6.

Solution command

```
docker compose ps
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Compare running services with all created service containers.

Why it matters: Compose commands should be run from the project directory or with -f.

Exercise 447 · List Compose services

Task: List service and container status for the current Compose project, case7.

Solution command

```
docker compose ps
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Compare running services with all created service containers.

Why it matters: Compose commands should be run from the project directory or with -f.

Exercise 448 · List Compose services

Task: List service and container status for the current Compose project, case8.

Solution command

```
docker compose ps
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Compare running services with all created service containers.

Why it matters: Compose commands should be run from the project directory or with -f.

Exercise 449 · List Compose services

Task: List service and container status for the current Compose project, case9.

Solution command

```
docker compose ps
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Compare running services with all created service containers.

Why it matters: Compose commands should be run from the project directory or with -f.

Exercise 450 · List Compose services

Task: List service and container status for the current Compose project, case10.

Solution command

```
docker compose ps
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Compare running services with all created service containers.

Why it matters: Compose commands should be run from the project directory or with -f.

Exercise 451 · Follow service logs

Task: Follow logs for service api in Compose project case1.

Solution command

```
docker compose logs --tail 100 --follow api
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose logs --tail 100 api`
4. Interpret the result: Review startup errors and press Ctrl+C to stop following logs.

Why it matters: The service name comes from the Compose file, not necessarily the container name.

Exercise 452 · Follow service logs

Task: Follow logs for service api in Compose project case2.

Solution command

```
docker compose logs --tail 100 --follow api
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose logs --tail 100 api`
4. Interpret the result: Review startup errors and press Ctrl+C to stop following logs.

Why it matters: The service name comes from the Compose file, not necessarily the container name.

Exercise 453 · Follow service logs

Task: Follow logs for service api in Compose project case3.

Solution command

```
docker compose logs --tail 100 --follow api
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose logs --tail 100 api`
4. Interpret the result: Review startup errors and press Ctrl+C to stop following logs.

Why it matters: The service name comes from the Compose file, not necessarily the container name.

Exercise 454 · Follow service logs

Task: Follow logs for service api in Compose project case4.

Solution command

```
docker compose logs --tail 100 --follow api
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose logs --tail 100 api`
4. Interpret the result: Review startup errors and press Ctrl+C to stop following logs.

Why it matters: The service name comes from the Compose file, not necessarily the container name.

Exercise 455 · Follow service logs

Task: Follow logs for service api in Compose project case5.

Solution command

```
docker compose logs --tail 100 --follow api
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose logs --tail 100 api`
4. Interpret the result: Review startup errors and press Ctrl+C to stop following logs.

Why it matters: The service name comes from the Compose file, not necessarily the container name.

Exercise 456 · Follow service logs

Task: Follow logs for service api in Compose project case6.

Solution command

```
docker compose logs --tail 100 --follow api
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose logs --tail 100 api`
4. Interpret the result: Review startup errors and press Ctrl+C to stop following logs.

Why it matters: The service name comes from the Compose file, not necessarily the container name.

Exercise 457 · Follow service logs

Task: Follow logs for service api in Compose project case7.

Solution command

```
docker compose logs --tail 100 --follow api
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose logs --tail 100 api`
4. Interpret the result: Review startup errors and press Ctrl+C to stop following logs.

Why it matters: The service name comes from the Compose file, not necessarily the container name.

Exercise 458 · Follow service logs

Task: Follow logs for service api in Compose project case8.

Solution command

```
docker compose logs --tail 100 --follow api
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose logs --tail 100 api`
4. Interpret the result: Review startup errors and press Ctrl+C to stop following logs.

Why it matters: The service name comes from the Compose file, not necessarily the container name.

Exercise 459 · Follow service logs

Task: Follow logs for service api in Compose project case9.

Solution command

```
docker compose logs --tail 100 --follow api
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose logs --tail 100 api`
4. Interpret the result: Review startup errors and press Ctrl+C to stop following logs.

Why it matters: The service name comes from the Compose file, not necessarily the container name.

Exercise 460 · Follow service logs

Task: Follow logs for service api in Compose project case10.

Solution command

```
docker compose logs --tail 100 --follow api
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose logs --tail 100 api`
4. Interpret the result: Review startup errors and press Ctrl+C to stop following logs.

Why it matters: The service name comes from the Compose file, not necessarily the container name.

Module 24: Docker Compose: exec & lifecycle

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 461 · Run a command in a service

Task: Run `id` inside the running Compose service api for case1.

Solution command

```
docker compose exec api id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose exec api pwd`
4. Interpret the result: The service must be running; use `run --rm` instead if you need a one-off service container.

Why it matters: Compose exec targets a service, whereas docker exec targets a container.

Exercise 462 · Run a command in a service

Task: Run `id` inside the running Compose service api for case2.

Solution command

```
docker compose exec api id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose exec api pwd`
4. Interpret the result: The service must be running; use `run --rm` instead if you need a one-off service container.

Why it matters: Compose exec targets a service, whereas docker exec targets a container.

Exercise 463 · Run a command in a service

Task: Run `id` inside the running Compose service api for case3.

Solution command

```
docker compose exec api id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose exec api pwd`
4. Interpret the result: The service must be running; use `run --rm` instead if you need a one-off service container.

Why it matters: Compose exec targets a service, whereas docker exec targets a container.

Exercise 464 · Run a command in a service

Task: Run `id` inside the running Compose service api for case4.

Solution command

```
docker compose exec api id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose exec api pwd`
4. Interpret the result: The service must be running; use `run --rm` instead if you need a one-off service container.

Why it matters: Compose exec targets a service, whereas docker exec targets a container.

Exercise 465 · Run a command in a service

Task: Run `id` inside the running Compose service api for case5.

Solution command

```
docker compose exec api id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose exec api pwd`
4. Interpret the result: The service must be running; use `run --rm` instead if you need a one-off service container.

Why it matters: Compose exec targets a service, whereas docker exec targets a container.

Exercise 466 · Run a command in a service

Task: Run `id` inside the running Compose service api for case6.

Solution command

```
docker compose exec api id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose exec api pwd`
4. Interpret the result: The service must be running; use `run --rm` instead if you need a one-off service container.

Why it matters: Compose exec targets a service, whereas docker exec targets a container.

Exercise 467 · Run a command in a service

Task: Run `id` inside the running Compose service api for case7.

Solution command

```
docker compose exec api id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose exec api pwd`
4. Interpret the result: The service must be running; use `run --rm` instead if you need a one-off service container.

Why it matters: Compose exec targets a service, whereas docker exec targets a container.

Exercise 468 · Run a command in a service

Task: Run `id` inside the running Compose service api for case8.

Solution command

```
docker compose exec api id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose exec api pwd`
4. Interpret the result: The service must be running; use `run --rm` instead if you need a one-off service container.

Why it matters: Compose exec targets a service, whereas docker exec targets a container.

Exercise 469 · Run a command in a service

Task: Run `id` inside the running Compose service api for case9.

Solution command

```
docker compose exec api id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose exec api pwd`
4. Interpret the result: The service must be running; use `run --rm` instead if you need a one-off service container.

Why it matters: Compose exec targets a service, whereas docker exec targets a container.

Exercise 470 · Run a command in a service

Task: Run `id` inside the running Compose service api for case10.

Solution command

```
docker compose exec api id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose exec api pwd`
4. Interpret the result: The service must be running; use `run --rm` instead if you need a one-off service container.

Why it matters: Compose exec targets a service, whereas docker exec targets a container.

Exercise 471 · Tear down a Compose project

Task: Stop and remove the lab Compose project's containers and network for case1.

Solution command

```
docker compose down
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Run only in the intended project directory; inspect project identity before down.

Why it matters: By default, named volumes are retained; options such as --volumes can delete data.

Exercise 472 · Tear down a Compose project

Task: Stop and remove the lab Compose project's containers and network for case2.

Solution command

```
docker compose down
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Run only in the intended project directory; inspect project identity before down.

Why it matters: By default, named volumes are retained; options such as --volumes can delete data.

Exercise 473 · Tear down a Compose project

Task: Stop and remove the lab Compose project's containers and network for case3.

Solution command

```
docker compose down
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Run only in the intended project directory; inspect project identity before down.

Why it matters: By default, named volumes are retained; options such as --volumes can delete data.

Exercise 474 · Tear down a Compose project

Task: Stop and remove the lab Compose project's containers and network for case4.

Solution command

```
docker compose down
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Run only in the intended project directory; inspect project identity before down.

Why it matters: By default, named volumes are retained; options such as --volumes can delete data.

Exercise 475 · Tear down a Compose project

Task: Stop and remove the lab Compose project's containers and network for case5.

Solution command

```
docker compose down
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Run only in the intended project directory; inspect project identity before down.

Why it matters: By default, named volumes are retained; options such as --volumes can delete data.

Exercise 476 · Tear down a Compose project

Task: Stop and remove the lab Compose project's containers and network for case6.

Solution command

```
docker compose down
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Run only in the intended project directory; inspect project identity before down.

Why it matters: By default, named volumes are retained; options such as --volumes can delete data.

Exercise 477 · Tear down a Compose project

Task: Stop and remove the lab Compose project's containers and network for case7.

Solution command

```
docker compose down
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Run only in the intended project directory; inspect project identity before down.

Why it matters: By default, named volumes are retained; options such as --volumes can delete data.

Exercise 478 · Tear down a Compose project

Task: Stop and remove the lab Compose project's containers and network for case8.

Solution command

```
docker compose down
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Run only in the intended project directory; inspect project identity before down.

Why it matters: By default, named volumes are retained; options such as --volumes can delete data.

Exercise 479 · Tear down a Compose project

Task: Stop and remove the lab Compose project's containers and network for case9.

Solution command

```
docker compose down
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Run only in the intended project directory; inspect project identity before down.

Why it matters: By default, named volumes are retained; options such as --volumes can delete data.

Exercise 480 · Tear down a Compose project

Task: Stop and remove the lab Compose project's containers and network for case10.

Solution command

```
docker compose down
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: Run only in the intended project directory; inspect project identity before down.

Why it matters: By default, named volumes are retained; options such as --volumes can delete data.

Module 25: Contexts & daemon endpoints

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 481 · List Docker contexts

Task: List configured Docker contexts and identify the active context for case1.

Solution command

```
docker context ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context show`
4. Interpret the result: The asterisk in context ls indicates the active context.

Why it matters: Changing contexts can redirect commands to a different Docker host.

Exercise 482 · List Docker contexts

Task: List configured Docker contexts and identify the active context for case2.

Solution command

```
docker context ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context show`
4. Interpret the result: The asterisk in context ls indicates the active context.

Why it matters: Changing contexts can redirect commands to a different Docker host.

Exercise 483 · List Docker contexts

Task: List configured Docker contexts and identify the active context for case3.

Solution command

```
docker context ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context show`
4. Interpret the result: The asterisk in context ls indicates the active context.

Why it matters: Changing contexts can redirect commands to a different Docker host.

Exercise 484 · List Docker contexts

Task: List configured Docker contexts and identify the active context for case4.

Solution command

```
docker context ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context show`
4. Interpret the result: The asterisk in context ls indicates the active context.

Why it matters: Changing contexts can redirect commands to a different Docker host.

Exercise 485 · List Docker contexts

Task: List configured Docker contexts and identify the active context for case5.

Solution command

```
docker context ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context show`
4. Interpret the result: The asterisk in context ls indicates the active context.

Why it matters: Changing contexts can redirect commands to a different Docker host.

Exercise 486 · List Docker contexts

Task: List configured Docker contexts and identify the active context for case6.

Solution command

```
docker context ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context show`
4. Interpret the result: The asterisk in context ls indicates the active context.

Why it matters: Changing contexts can redirect commands to a different Docker host.

Exercise 487 · List Docker contexts

Task: List configured Docker contexts and identify the active context for case7.

Solution command

```
docker context ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context show`
4. Interpret the result: The asterisk in context ls indicates the active context.

Why it matters: Changing contexts can redirect commands to a different Docker host.

Exercise 488 · List Docker contexts

Task: List configured Docker contexts and identify the active context for case8.

Solution command

```
docker context ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context show`
4. Interpret the result: The asterisk in context ls indicates the active context.

Why it matters: Changing contexts can redirect commands to a different Docker host.

Exercise 489 · List Docker contexts

Task: List configured Docker contexts and identify the active context for case9.

Solution command

```
docker context ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context show`
4. Interpret the result: The asterisk in context ls indicates the active context.

Why it matters: Changing contexts can redirect commands to a different Docker host.

Exercise 490 · List Docker contexts

Task: List configured Docker contexts and identify the active context for case10.

Solution command

```
docker context ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context show`
4. Interpret the result: The asterisk in context ls indicates the active context.

Why it matters: Changing contexts can redirect commands to a different Docker host.

Exercise 491 · Create and inspect a context

Task: Create a context named labctx1 for a Docker endpoint you control.

Solution command

```
docker context create labctx1 --docker host=unix:///var/run/docker.sock
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context inspect labctx1`
4. Interpret the result: Use a valid endpoint for your operating system; inspect before switching to it.

Why it matters: The Unix socket example is Linux-specific; Docker Desktop may use a different endpoint.

Exercise 492 · Create and inspect a context

Task: Create a context named labctx2 for a Docker endpoint you control.

Solution command

```
docker context create labctx2 --docker host=unix:///var/run/docker.sock
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context inspect labctx2`
4. Interpret the result: Use a valid endpoint for your operating system; inspect before switching to it.

Why it matters: The Unix socket example is Linux-specific; Docker Desktop may use a different endpoint.

Exercise 493 · Create and inspect a context

Task: Create a context named labctx3 for a Docker endpoint you control.

Solution command

```
docker context create labctx3 --docker host=unix:///var/run/docker.sock
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context inspect labctx3`
4. Interpret the result: Use a valid endpoint for your operating system; inspect before switching to it.

Why it matters: The Unix socket example is Linux-specific; Docker Desktop may use a different endpoint.

Exercise 494 · Create and inspect a context

Task: Create a context named labctx4 for a Docker endpoint you control.

Solution command

```
docker context create labctx4 --docker host=unix:///var/run/docker.sock
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context inspect labctx4`
4. Interpret the result: Use a valid endpoint for your operating system; inspect before switching to it.

Why it matters: The Unix socket example is Linux-specific; Docker Desktop may use a different endpoint.

Exercise 495 · Create and inspect a context

Task: Create a context named labctx5 for a Docker endpoint you control.

Solution command

```
docker context create labctx5 --docker host=unix:///var/run/docker.sock
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context inspect labctx5`
4. Interpret the result: Use a valid endpoint for your operating system; inspect before switching to it.

Why it matters: The Unix socket example is Linux-specific; Docker Desktop may use a different endpoint.

Exercise 496 · Create and inspect a context

Task: Create a context named labctx6 for a Docker endpoint you control.

Solution command

```
docker context create labctx6 --docker host=unix:///var/run/docker.sock
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context inspect labctx6`
4. Interpret the result: Use a valid endpoint for your operating system; inspect before switching to it.

Why it matters: The Unix socket example is Linux-specific; Docker Desktop may use a different endpoint.

Exercise 497 · Create and inspect a context

Task: Create a context named labctx7 for a Docker endpoint you control.

Solution command

```
docker context create labctx7 --docker host=unix:///var/run/docker.sock
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context inspect labctx7`
4. Interpret the result: Use a valid endpoint for your operating system; inspect before switching to it.

Why it matters: The Unix socket example is Linux-specific; Docker Desktop may use a different endpoint.

Exercise 498 · Create and inspect a context

Task: Create a context named labctx8 for a Docker endpoint you control.

Solution command

```
docker context create labctx8 --docker host=unix:///var/run/docker.sock
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context inspect labctx8`
4. Interpret the result: Use a valid endpoint for your operating system; inspect before switching to it.

Why it matters: The Unix socket example is Linux-specific; Docker Desktop may use a different endpoint.

Exercise 499 · Create and inspect a context

Task: Create a context named labctx9 for a Docker endpoint you control.

Solution command

```
docker context create labctx9 --docker host=unix:///var/run/docker.sock
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context inspect labctx9`
4. Interpret the result: Use a valid endpoint for your operating system; inspect before switching to it.

Why it matters: The Unix socket example is Linux-specific; Docker Desktop may use a different endpoint.

Exercise 500 · Create and inspect a context

Task: Create a context named labctx10 for a Docker endpoint you control.

Solution command

```
docker context create labctx10 --docker host=unix:///var/run/docker.sock
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker context inspect labctx10`
4. Interpret the result: Use a valid endpoint for your operating system; inspect before switching to it.

Why it matters: The Unix socket example is Linux-specific; Docker Desktop may use a different endpoint.

Module 26: Docker Swarm: cluster basics

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 501 · Initialize a Swarm manager

Task: Initialize a single-node Swarm for isolated lab case1.

Solution command

```
docker swarm init
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run only on a disposable lab host that is not already part of a Swarm.

Why it matters: Swarm initialization changes node state; use `docker swarm leave --force` only on a lab manager you intend to dismantle.

Exercise 502 · Initialize a Swarm manager

Task: Initialize a single-node Swarm for isolated lab case2.

Solution command

```
docker swarm init
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run only on a disposable lab host that is not already part of a Swarm.

Why it matters: Swarm initialization changes node state; use `docker swarm leave --force` only on a lab manager you intend to dismantle.

Exercise 503 · Initialize a Swarm manager

Task: Initialize a single-node Swarm for isolated lab case3.

Solution command

```
docker swarm init
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run only on a disposable lab host that is not already part of a Swarm.

Why it matters: Swarm initialization changes node state; use `docker swarm leave --force` only on a lab manager you intend to dismantle.

Exercise 504 · Initialize a Swarm manager

Task: Initialize a single-node Swarm for isolated lab case4.

Solution command

```
docker swarm init
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run only on a disposable lab host that is not already part of a Swarm.

Why it matters: Swarm initialization changes node state; use `docker swarm leave --force` only on a lab manager you intend to dismantle.

Exercise 505 · Initialize a Swarm manager

Task: Initialize a single-node Swarm for isolated lab case5.

Solution command

```
docker swarm init
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run only on a disposable lab host that is not already part of a Swarm.

Why it matters: Swarm initialization changes node state; use `docker swarm leave --force` only on a lab manager you intend to dismantle.

Exercise 506 · Initialize a Swarm manager

Task: Initialize a single-node Swarm for isolated lab case6.

Solution command

```
docker swarm init
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run only on a disposable lab host that is not already part of a Swarm.

Why it matters: Swarm initialization changes node state; use `docker swarm leave --force` only on a lab manager you intend to dismantle.

Exercise 507 · Initialize a Swarm manager

Task: Initialize a single-node Swarm for isolated lab case7.

Solution command

```
docker swarm init
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run only on a disposable lab host that is not already part of a Swarm.

Why it matters: Swarm initialization changes node state; use `docker swarm leave --force` only on a lab manager you intend to dismantle.

Exercise 508 · Initialize a Swarm manager

Task: Initialize a single-node Swarm for isolated lab case8.

Solution command

```
docker swarm init
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run only on a disposable lab host that is not already part of a Swarm.

Why it matters: Swarm initialization changes node state; use `docker swarm leave --force` only on a lab manager you intend to dismantle.

Exercise 509 · Initialize a Swarm manager

Task: Initialize a single-node Swarm for isolated lab case9.

Solution command

```
docker swarm init
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run only on a disposable lab host that is not already part of a Swarm.

Why it matters: Swarm initialization changes node state; use `docker swarm leave --force` only on a lab manager you intend to dismantle.

Exercise 510 · Initialize a Swarm manager

Task: Initialize a single-node Swarm for isolated lab case10.

Solution command

```
docker swarm init
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run only on a disposable lab host that is not already part of a Swarm.

Why it matters: Swarm initialization changes node state; use `docker swarm leave --force` only on a lab manager you intend to dismantle.

Exercise 511 · List Swarm nodes

Task: On a manager, list nodes and inspect their availability for case1.

Solution command

```
docker node ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node inspect self`
4. Interpret the result: These commands require manager permissions; worker nodes cannot list the whole cluster.

Why it matters: Node state and availability are different concepts.

Exercise 512 · List Swarm nodes

Task: On a manager, list nodes and inspect their availability for case2.

Solution command

```
docker node ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node inspect self`
4. Interpret the result: These commands require manager permissions; worker nodes cannot list the whole cluster.

Why it matters: Node state and availability are different concepts.

Exercise 513 · List Swarm nodes

Task: On a manager, list nodes and inspect their availability for case3.

Solution command

```
docker node ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node inspect self`
4. Interpret the result: These commands require manager permissions; worker nodes cannot list the whole cluster.

Why it matters: Node state and availability are different concepts.

Exercise 514 · List Swarm nodes

Task: On a manager, list nodes and inspect their availability for case4.

Solution command

```
docker node ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node inspect self`
4. Interpret the result: These commands require manager permissions; worker nodes cannot list the whole cluster.

Why it matters: Node state and availability are different concepts.

Exercise 515 · List Swarm nodes

Task: On a manager, list nodes and inspect their availability for case5.

Solution command

```
docker node ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node inspect self`
4. Interpret the result: These commands require manager permissions; worker nodes cannot list the whole cluster.

Why it matters: Node state and availability are different concepts.

Exercise 516 · List Swarm nodes

Task: On a manager, list nodes and inspect their availability for case6.

Solution command

```
docker node ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node inspect self`
4. Interpret the result: These commands require manager permissions; worker nodes cannot list the whole cluster.

Why it matters: Node state and availability are different concepts.

Exercise 517 · List Swarm nodes

Task: On a manager, list nodes and inspect their availability for case7.

Solution command

```
docker node ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node inspect self`
4. Interpret the result: These commands require manager permissions; worker nodes cannot list the whole cluster.

Why it matters: Node state and availability are different concepts.

Exercise 518 · List Swarm nodes

Task: On a manager, list nodes and inspect their availability for case8.

Solution command

```
docker node ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node inspect self`
4. Interpret the result: These commands require manager permissions; worker nodes cannot list the whole cluster.

Why it matters: Node state and availability are different concepts.

Exercise 519 · List Swarm nodes

Task: On a manager, list nodes and inspect their availability for case9.

Solution command

```
docker node ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node inspect self`
4. Interpret the result: These commands require manager permissions; worker nodes cannot list the whole cluster.

Why it matters: Node state and availability are different concepts.

Exercise 520 · List Swarm nodes

Task: On a manager, list nodes and inspect their availability for case10.

Solution command

```
docker node ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node inspect self`
4. Interpret the result: These commands require manager permissions; worker nodes cannot list the whole cluster.

Why it matters: Node state and availability are different concepts.

Module 27: Docker Swarm: services & scaling

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 521 · Create a replicated service

Task: Create service web1 with two replicas using nginx:alpine.

Solution command

```
docker service create --name web1 --replicas 2 -p 8081:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web1`
4. Interpret the result: Wait for tasks to be scheduled, then inspect task state and node placement.

Why it matters: This requires a Swarm manager; choose an unused published port.

Exercise 522 · Create a replicated service

Task: Create service web2 with two replicas using nginx:alpine.

Solution command

```
docker service create --name web2 --replicas 2 -p 8082:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web2`
4. Interpret the result: Wait for tasks to be scheduled, then inspect task state and node placement.

Why it matters: This requires a Swarm manager; choose an unused published port.

Exercise 523 · Create a replicated service

Task: Create service web3 with two replicas using nginx:alpine.

Solution command

```
docker service create --name web3 --replicas 2 -p 8083:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web3`
4. Interpret the result: Wait for tasks to be scheduled, then inspect task state and node placement.

Why it matters: This requires a Swarm manager; choose an unused published port.

Exercise 524 · Create a replicated service

Task: Create service web4 with two replicas using nginx:alpine.

Solution command

```
docker service create --name web4 --replicas 2 -p 8084:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web4`
4. Interpret the result: Wait for tasks to be scheduled, then inspect task state and node placement.

Why it matters: This requires a Swarm manager; choose an unused published port.

Exercise 525 · Create a replicated service

Task: Create service web5 with two replicas using nginx:alpine.

Solution command

```
docker service create --name web5 --replicas 2 -p 8085:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web5`
4. Interpret the result: Wait for tasks to be scheduled, then inspect task state and node placement.

Why it matters: This requires a Swarm manager; choose an unused published port.

Exercise 526 · Create a replicated service

Task: Create service web6 with two replicas using nginx:alpine.

Solution command

```
docker service create --name web6 --replicas 2 -p 8086:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web6`
4. Interpret the result: Wait for tasks to be scheduled, then inspect task state and node placement.

Why it matters: This requires a Swarm manager; choose an unused published port.

Exercise 527 · Create a replicated service

Task: Create service web7 with two replicas using nginx:alpine.

Solution command

```
docker service create --name web7 --replicas 2 -p 8087:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web7`
4. Interpret the result: Wait for tasks to be scheduled, then inspect task state and node placement.

Why it matters: This requires a Swarm manager; choose an unused published port.

Exercise 528 · Create a replicated service

Task: Create service web8 with two replicas using nginx:alpine.

Solution command

```
docker service create --name web8 --replicas 2 -p 8088:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web8`
4. Interpret the result: Wait for tasks to be scheduled, then inspect task state and node placement.

Why it matters: This requires a Swarm manager; choose an unused published port.

Exercise 529 · Create a replicated service

Task: Create service web9 with two replicas using nginx:alpine.

Solution command

```
docker service create --name web9 --replicas 2 -p 8089:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web9`
4. Interpret the result: Wait for tasks to be scheduled, then inspect task state and node placement.

Why it matters: This requires a Swarm manager; choose an unused published port.

Exercise 530 · Create a replicated service

Task: Create service web10 with two replicas using nginx:alpine.

Solution command

```
docker service create --name web10 --replicas 2 -p 8090:80 nginx:alpine
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web10`
4. Interpret the result: Wait for tasks to be scheduled, then inspect task state and node placement.

Why it matters: This requires a Swarm manager; choose an unused published port.

Exercise 531 · Scale a Swarm service

Task: Scale Swarm service web1 to three replicas.

Solution command

```
docker service scale web1=3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web1`
4. Interpret the result: Check desired and current task counts; allow time for scheduling.

Why it matters: Scaling applies to a service, not to a standalone container.

Exercise 532 · Scale a Swarm service

Task: Scale Swarm service web2 to three replicas.

Solution command

```
docker service scale web2=3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web2`
4. Interpret the result: Check desired and current task counts; allow time for scheduling.

Why it matters: Scaling applies to a service, not to a standalone container.

Exercise 533 · Scale a Swarm service

Task: Scale Swarm service web3 to three replicas.

Solution command

```
docker service scale web3=3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web3`
4. Interpret the result: Check desired and current task counts; allow time for scheduling.

Why it matters: Scaling applies to a service, not to a standalone container.

Exercise 534 · Scale a Swarm service

Task: Scale Swarm service web4 to three replicas.

Solution command

```
docker service scale web4=3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web4`
4. Interpret the result: Check desired and current task counts; allow time for scheduling.

Why it matters: Scaling applies to a service, not to a standalone container.

Exercise 535 · Scale a Swarm service

Task: Scale Swarm service web5 to three replicas.

Solution command

```
docker service scale web5=3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web5`
4. Interpret the result: Check desired and current task counts; allow time for scheduling.

Why it matters: Scaling applies to a service, not to a standalone container.

Exercise 536 · Scale a Swarm service

Task: Scale Swarm service web6 to three replicas.

Solution command

```
docker service scale web6=3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web6`
4. Interpret the result: Check desired and current task counts; allow time for scheduling.

Why it matters: Scaling applies to a service, not to a standalone container.

Exercise 537 · Scale a Swarm service

Task: Scale Swarm service web7 to three replicas.

Solution command

```
docker service scale web7=3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web7`
4. Interpret the result: Check desired and current task counts; allow time for scheduling.

Why it matters: Scaling applies to a service, not to a standalone container.

Exercise 538 · Scale a Swarm service

Task: Scale Swarm service web8 to three replicas.

Solution command

```
docker service scale web8=3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web8`
4. Interpret the result: Check desired and current task counts; allow time for scheduling.

Why it matters: Scaling applies to a service, not to a standalone container.

Exercise 539 · Scale a Swarm service

Task: Scale Swarm service web9 to three replicas.

Solution command

```
docker service scale web9=3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web9`
4. Interpret the result: Check desired and current task counts; allow time for scheduling.

Why it matters: Scaling applies to a service, not to a standalone container.

Exercise 540 · Scale a Swarm service

Task: Scale Swarm service web10 to three replicas.

Solution command

```
docker service scale web10=3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web10`
4. Interpret the result: Check desired and current task counts; allow time for scheduling.

Why it matters: Scaling applies to a service, not to a standalone container.

Module 28: Swarm secrets, configs & stacks

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 541 · Create a Swarm secret

Task: Create secret labsecret1 from a local file you created for the lab.

Solution command

```
docker secret create labsecret1 ./secret-1.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker secret ls`
4. Interpret the result: Create a short-lived test file first; the secret's content is not shown by secret ls.

Why it matters: Do not commit secret files to source control or use real credentials in practice labs.

Exercise 542 · Create a Swarm secret

Task: Create secret labsecret2 from a local file you created for the lab.

Solution command

```
docker secret create labsecret2 ./secret-2.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker secret ls`
4. Interpret the result: Create a short-lived test file first; the secret's content is not shown by secret ls.

Why it matters: Do not commit secret files to source control or use real credentials in practice labs.

Exercise 543 · Create a Swarm secret

Task: Create secret labsecret3 from a local file you created for the lab.

Solution command

```
docker secret create labsecret3 ./secret-3.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker secret ls`
4. Interpret the result: Create a short-lived test file first; the secret's content is not shown by secret ls.

Why it matters: Do not commit secret files to source control or use real credentials in practice labs.

Exercise 544 · Create a Swarm secret

Task: Create secret labsecret4 from a local file you created for the lab.

Solution command

```
docker secret create labsecret4 ./secret-4.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker secret ls`
4. Interpret the result: Create a short-lived test file first; the secret's content is not shown by secret ls.

Why it matters: Do not commit secret files to source control or use real credentials in practice labs.

Exercise 545 · Create a Swarm secret

Task: Create secret labsecret5 from a local file you created for the lab.

Solution command

```
docker secret create labsecret5 ./secret-5.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker secret ls`
4. Interpret the result: Create a short-lived test file first; the secret's content is not shown by secret ls.

Why it matters: Do not commit secret files to source control or use real credentials in practice labs.

Exercise 546 · Create a Swarm secret

Task: Create secret labsecret6 from a local file you created for the lab.

Solution command

```
docker secret create labsecret6 ./secret-6.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker secret ls`
4. Interpret the result: Create a short-lived test file first; the secret's content is not shown by secret ls.

Why it matters: Do not commit secret files to source control or use real credentials in practice labs.

Exercise 547 · Create a Swarm secret

Task: Create secret labsecret7 from a local file you created for the lab.

Solution command

```
docker secret create labsecret7 ./secret-7.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker secret ls`
4. Interpret the result: Create a short-lived test file first; the secret's content is not shown by secret ls.

Why it matters: Do not commit secret files to source control or use real credentials in practice labs.

Exercise 548 · Create a Swarm secret

Task: Create secret labsecret8 from a local file you created for the lab.

Solution command

```
docker secret create labsecret8 ./secret-8.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker secret ls`
4. Interpret the result: Create a short-lived test file first; the secret's content is not shown by secret ls.

Why it matters: Do not commit secret files to source control or use real credentials in practice labs.

Exercise 549 · Create a Swarm secret

Task: Create secret labsecret9 from a local file you created for the lab.

Solution command

```
docker secret create labsecret9 ./secret-9.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker secret ls`
4. Interpret the result: Create a short-lived test file first; the secret's content is not shown by secret ls.

Why it matters: Do not commit secret files to source control or use real credentials in practice labs.

Exercise 550 · Create a Swarm secret

Task: Create secret labsecret10 from a local file you created for the lab.

Solution command

```
docker secret create labsecret10 ./secret-10.txt
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker secret ls`
4. Interpret the result: Create a short-lived test file first; the secret's content is not shown by secret ls.

Why it matters: Do not commit secret files to source control or use real credentials in practice labs.

Exercise 551 · Deploy a stack

Task: Deploy stack labstack1 from a Compose-compatible stack file you control.

Solution command

```
docker stack deploy -c stack.yml labstack1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack1`
4. Interpret the result: Check that stack.yml uses options supported by Swarm stack deployment.

Why it matters: Stack deployment requires a Swarm manager and differs from local Compose behavior.

Exercise 552 · Deploy a stack

Task: Deploy stack labstack2 from a Compose-compatible stack file you control.

Solution command

```
docker stack deploy -c stack.yml labstack2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack2`
4. Interpret the result: Check that stack.yml uses options supported by Swarm stack deployment.

Why it matters: Stack deployment requires a Swarm manager and differs from local Compose behavior.

Exercise 553 · Deploy a stack

Task: Deploy stack labstack3 from a Compose-compatible stack file you control.

Solution command

```
docker stack deploy -c stack.yml labstack3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack3`
4. Interpret the result: Check that stack.yml uses options supported by Swarm stack deployment.

Why it matters: Stack deployment requires a Swarm manager and differs from local Compose behavior.

Exercise 554 · Deploy a stack

Task: Deploy stack labstack4 from a Compose-compatible stack file you control.

Solution command

```
docker stack deploy -c stack.yml labstack4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack4`
4. Interpret the result: Check that stack.yml uses options supported by Swarm stack deployment.

Why it matters: Stack deployment requires a Swarm manager and differs from local Compose behavior.

Exercise 555 · Deploy a stack

Task: Deploy stack labstack5 from a Compose-compatible stack file you control.

Solution command

```
docker stack deploy -c stack.yml labstack5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack5`
4. Interpret the result: Check that stack.yml uses options supported by Swarm stack deployment.

Why it matters: Stack deployment requires a Swarm manager and differs from local Compose behavior.

Exercise 556 · Deploy a stack

Task: Deploy stack labstack6 from a Compose-compatible stack file you control.

Solution command

```
docker stack deploy -c stack.yml labstack6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack6`
4. Interpret the result: Check that stack.yml uses options supported by Swarm stack deployment.

Why it matters: Stack deployment requires a Swarm manager and differs from local Compose behavior.

Exercise 557 · Deploy a stack

Task: Deploy stack labstack7 from a Compose-compatible stack file you control.

Solution command

```
docker stack deploy -c stack.yml labstack7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack7`
4. Interpret the result: Check that stack.yml uses options supported by Swarm stack deployment.

Why it matters: Stack deployment requires a Swarm manager and differs from local Compose behavior.

Exercise 558 · Deploy a stack

Task: Deploy stack labstack8 from a Compose-compatible stack file you control.

Solution command

```
docker stack deploy -c stack.yml labstack8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack8`
4. Interpret the result: Check that stack.yml uses options supported by Swarm stack deployment.

Why it matters: Stack deployment requires a Swarm manager and differs from local Compose behavior.

Exercise 559 · Deploy a stack

Task: Deploy stack labstack9 from a Compose-compatible stack file you control.

Solution command

```
docker stack deploy -c stack.yml labstack9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack9`
4. Interpret the result: Check that stack.yml uses options supported by Swarm stack deployment.

Why it matters: Stack deployment requires a Swarm manager and differs from local Compose behavior.

Exercise 560 · Deploy a stack

Task: Deploy stack labstack10 from a Compose-compatible stack file you control.

Solution command

```
docker stack deploy -c stack.yml labstack10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack10`
4. Interpret the result: Check that stack.yml uses options supported by Swarm stack deployment.

Why it matters: Stack deployment requires a Swarm manager and differs from local Compose behavior.

Module 29: Runtime security & permissions

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 561 · Run as a non-root user

Task: Run a test image with a numeric non-root user for case1.

Solution command

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --user 10001:10001 alpine:3.20 id`
4. Interpret the result: Check the UID and GID in the output.

Why it matters: Some images require specific ownership or privileges; validate before applying to production.

Exercise 562 · Run as a non-root user

Task: Run a test image with a numeric non-root user for case2.

Solution command

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --user 10001:10001 alpine:3.20 id`
4. Interpret the result: Check the UID and GID in the output.

Why it matters: Some images require specific ownership or privileges; validate before applying to production.

Exercise 563 · Run as a non-root user

Task: Run a test image with a numeric non-root user for case3.

Solution command

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --user 10001:10001 alpine:3.20 id`
4. Interpret the result: Check the UID and GID in the output.

Why it matters: Some images require specific ownership or privileges; validate before applying to production.

Exercise 564 · Run as a non-root user

Task: Run a test image with a numeric non-root user for case4.

Solution command

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --user 10001:10001 alpine:3.20 id`
4. Interpret the result: Check the UID and GID in the output.

Why it matters: Some images require specific ownership or privileges; validate before applying to production.

Exercise 565 · Run as a non-root user

Task: Run a test image with a numeric non-root user for case5.

Solution command

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --user 10001:10001 alpine:3.20 id`
4. Interpret the result: Check the UID and GID in the output.

Why it matters: Some images require specific ownership or privileges; validate before applying to production.

Exercise 566 · Run as a non-root user

Task: Run a test image with a numeric non-root user for case6.

Solution command

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --user 10001:10001 alpine:3.20 id`
4. Interpret the result: Check the UID and GID in the output.

Why it matters: Some images require specific ownership or privileges; validate before applying to production.

Exercise 567 · Run as a non-root user

Task: Run a test image with a numeric non-root user for case7.

Solution command

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --user 10001:10001 alpine:3.20 id`
4. Interpret the result: Check the UID and GID in the output.

Why it matters: Some images require specific ownership or privileges; validate before applying to production.

Exercise 568 · Run as a non-root user

Task: Run a test image with a numeric non-root user for case8.

Solution command

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --user 10001:10001 alpine:3.20 id`
4. Interpret the result: Check the UID and GID in the output.

Why it matters: Some images require specific ownership or privileges; validate before applying to production.

Exercise 569 · Run as a non-root user

Task: Run a test image with a numeric non-root user for case9.

Solution command

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --user 10001:10001 alpine:3.20 id`
4. Interpret the result: Check the UID and GID in the output.

Why it matters: Some images require specific ownership or privileges; validate before applying to production.

Exercise 570 · Run as a non-root user

Task: Run a test image with a numeric non-root user for case10.

Solution command

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --user 10001:10001 alpine:3.20 id`
4. Interpret the result: Check the UID and GID in the output.

Why it matters: Some images require specific ownership or privileges; validate before applying to production.

Exercise 571 · Drop Linux capabilities

Task: Run a test container with all Linux capabilities dropped for case1.

Solution command

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --cap-drop ALL alpine:3.20 id`
4. Interpret the result: Observe that a simple identity command works without extra capabilities.

Why it matters: Add back only the specific capability an application demonstrably needs.

Exercise 572 · Drop Linux capabilities

Task: Run a test container with all Linux capabilities dropped for case2.

Solution command

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --cap-drop ALL alpine:3.20 id`
4. Interpret the result: Observe that a simple identity command works without extra capabilities.

Why it matters: Add back only the specific capability an application demonstrably needs.

Exercise 573 · Drop Linux capabilities

Task: Run a test container with all Linux capabilities dropped for case3.

Solution command

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --cap-drop ALL alpine:3.20 id`
4. Interpret the result: Observe that a simple identity command works without extra capabilities.

Why it matters: Add back only the specific capability an application demonstrably needs.

Exercise 574 · Drop Linux capabilities

Task: Run a test container with all Linux capabilities dropped for case4.

Solution command

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --cap-drop ALL alpine:3.20 id`
4. Interpret the result: Observe that a simple identity command works without extra capabilities.

Why it matters: Add back only the specific capability an application demonstrably needs.

Exercise 575 · Drop Linux capabilities

Task: Run a test container with all Linux capabilities dropped for case5.

Solution command

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --cap-drop ALL alpine:3.20 id`
4. Interpret the result: Observe that a simple identity command works without extra capabilities.

Why it matters: Add back only the specific capability an application demonstrably needs.

Exercise 576 · Drop Linux capabilities

Task: Run a test container with all Linux capabilities dropped for case6.

Solution command

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --cap-drop ALL alpine:3.20 id`
4. Interpret the result: Observe that a simple identity command works without extra capabilities.

Why it matters: Add back only the specific capability an application demonstrably needs.

Exercise 577 · Drop Linux capabilities

Task: Run a test container with all Linux capabilities dropped for case7.

Solution command

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --cap-drop ALL alpine:3.20 id`
4. Interpret the result: Observe that a simple identity command works without extra capabilities.

Why it matters: Add back only the specific capability an application demonstrably needs.

Exercise 578 · Drop Linux capabilities

Task: Run a test container with all Linux capabilities dropped for case8.

Solution command

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --cap-drop ALL alpine:3.20 id`
4. Interpret the result: Observe that a simple identity command works without extra capabilities.

Why it matters: Add back only the specific capability an application demonstrably needs.

Exercise 579 · Drop Linux capabilities

Task: Run a test container with all Linux capabilities dropped for case9.

Solution command

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --cap-drop ALL alpine:3.20 id`
4. Interpret the result: Observe that a simple identity command works without extra capabilities.

Why it matters: Add back only the specific capability an application demonstrably needs.

Exercise 580 · Drop Linux capabilities

Task: Run a test container with all Linux capabilities dropped for case10.

Solution command

```
docker run --rm --cap-drop ALL alpine:3.20 id
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker run --rm --cap-drop ALL alpine:3.20 id`
4. Interpret the result: Observe that a simple identity command works without extra capabilities.

Why it matters: Add back only the specific capability an application demonstrably needs.

Module 30: Health checks, multi-platform builds & final workflow

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 581 · Inspect health status

Task: Inspect health state for container app1 if its image defines a HEALTHCHECK.

Solution command

```
docker inspect --format '{if .State.Health}{.State.Health.Status}{else}no-healthcheck{end}' app1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app1`
4. Interpret the result: If output says no-healthcheck, the image may not define a health check.

Why it matters: A running container is not necessarily a healthy application.

Exercise 582 · Inspect health status

Task: Inspect health state for container app2 if its image defines a HEALTHCHECK.

Solution command

```
docker inspect --format '{if .State.Health}{.State.Health.Status}{else}no-healthcheck{end}' app2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app2`
4. Interpret the result: If output says no-healthcheck, the image may not define a health check.

Why it matters: A running container is not necessarily a healthy application.

Exercise 583 · Inspect health status

Task: Inspect health state for container app3 if its image defines a HEALTHCHECK.

Solution command

```
docker inspect --format '{if .State.Health}{.State.Health.Status}{else}no-healthcheck{end}' app3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app3`
4. Interpret the result: If output says no-healthcheck, the image may not define a health check.

Why it matters: A running container is not necessarily a healthy application.

Exercise 584 · Inspect health status

Task: Inspect health state for container app4 if its image defines a HEALTHCHECK.

Solution command

```
docker inspect --format '{if .State.Health}{.State.Health.Status}{else}no-healthcheck{end}' app4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app4`
4. Interpret the result: If output says no-healthcheck, the image may not define a health check.

Why it matters: A running container is not necessarily a healthy application.

Exercise 585 · Inspect health status

Task: Inspect health state for container app5 if its image defines a HEALTHCHECK.

Solution command

```
docker inspect --format '{if .State.Health}{.State.Health.Status}{else}no-healthcheck{end}' app5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app5`
4. Interpret the result: If output says no-healthcheck, the image may not define a health check.

Why it matters: A running container is not necessarily a healthy application.

Exercise 586 · Inspect health status

Task: Inspect health state for container app6 if its image defines a HEALTHCHECK.

Solution command

```
docker inspect --format '{if .State.Health}{.State.Health.Status}{else}no-healthcheck{end}' app6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app6`
4. Interpret the result: If output says no-healthcheck, the image may not define a health check.

Why it matters: A running container is not necessarily a healthy application.

Exercise 587 · Inspect health status

Task: Inspect health state for container app7 if its image defines a HEALTHCHECK.

Solution command

```
docker inspect --format '{if .State.Health}{.State.Health.Status}{else}no-healthcheck{end}' app7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app7`
4. Interpret the result: If output says no-healthcheck, the image may not define a health check.

Why it matters: A running container is not necessarily a healthy application.

Exercise 588 · Inspect health status

Task: Inspect health state for container app8 if its image defines a HEALTHCHECK.

Solution command

```
docker inspect --format '{if .State.Health}{.State.Health.Status}{else}no-healthcheck{end}' app8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app8`
4. Interpret the result: If output says no-healthcheck, the image may not define a health check.

Why it matters: A running container is not necessarily a healthy application.

Exercise 589 · Inspect health status

Task: Inspect health state for container app9 if its image defines a HEALTHCHECK.

Solution command

```
docker inspect --format '{if .State.Health}{.State.Health.Status}{else}no-healthcheck{end}' app9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app9`
4. Interpret the result: If output says no-healthcheck, the image may not define a health check.

Why it matters: A running container is not necessarily a healthy application.

Exercise 590 · Inspect health status

Task: Inspect health state for container app10 if its image defines a HEALTHCHECK.

Solution command

```
docker inspect --format '{if .State.Health}{.State.Health.Status}{else}no-healthcheck{end}' app10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect app10`
4. Interpret the result: If output says no-healthcheck, the image may not define a health check.

Why it matters: A running container is not necessarily a healthy application.

Exercise 591 · List Buildx builders

Task: List configured Buildx builders before a multi-platform build, case1.

Solution command

```
docker buildx ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx inspect`
4. Interpret the result: Review the active builder and supported platforms.

Why it matters: Multi-platform output may require a suitable builder and registry push; availability varies by installation.

Exercise 592 · List Buildx builders

Task: List configured Buildx builders before a multi-platform build, case2.

Solution command

```
docker buildx ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx inspect`
4. Interpret the result: Review the active builder and supported platforms.

Why it matters: Multi-platform output may require a suitable builder and registry push; availability varies by installation.

Exercise 593 · List Buildx builders

Task: List configured Buildx builders before a multi-platform build, case3.

Solution command

```
docker buildx ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx inspect`
4. Interpret the result: Review the active builder and supported platforms.

Why it matters: Multi-platform output may require a suitable builder and registry push; availability varies by installation.

Exercise 594 · List Buildx builders

Task: List configured Buildx builders before a multi-platform build, case4.

Solution command

```
docker buildx ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx inspect`
4. Interpret the result: Review the active builder and supported platforms.

Why it matters: Multi-platform output may require a suitable builder and registry push; availability varies by installation.

Exercise 595 · List Buildx builders

Task: List configured Buildx builders before a multi-platform build, case5.

Solution command

```
docker buildx ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx inspect`
4. Interpret the result: Review the active builder and supported platforms.

Why it matters: Multi-platform output may require a suitable builder and registry push; availability varies by installation.

Exercise 596 · List Buildx builders

Task: List configured Buildx builders before a multi-platform build, case6.

Solution command

```
docker buildx ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx inspect`
4. Interpret the result: Review the active builder and supported platforms.

Why it matters: Multi-platform output may require a suitable builder and registry push; availability varies by installation.

Exercise 597 · List Buildx builders

Task: List configured Buildx builders before a multi-platform build, case7.

Solution command

```
docker buildx ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx inspect`
4. Interpret the result: Review the active builder and supported platforms.

Why it matters: Multi-platform output may require a suitable builder and registry push; availability varies by installation.

Exercise 598 · List Buildx builders

Task: List configured Buildx builders before a multi-platform build, case8.

Solution command

```
docker buildx ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx inspect`
4. Interpret the result: Review the active builder and supported platforms.

Why it matters: Multi-platform output may require a suitable builder and registry push; availability varies by installation.

Exercise 599 · List Buildx builders

Task: List configured Buildx builders before a multi-platform build, case9.

Solution command

```
docker buildx ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx inspect`
4. Interpret the result: Review the active builder and supported platforms.

Why it matters: Multi-platform output may require a suitable builder and registry push; availability varies by installation.

Exercise 600 · List Buildx builders

Task: List configured Buildx builders before a multi-platform build, case10.

Solution command

```
docker buildx ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx inspect`
4. Interpret the result: Review the active builder and supported platforms.

Why it matters: Multi-platform output may require a suitable builder and registry push; availability varies by installation.

Module 31: Container signals & exit status

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 601 · Force-stop a lab container

Task: Send the configured kill signal to the disposable container kill1.

Solution command

```
docker kill kill1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' kill1`
4. Interpret the result: Confirm the target is the disposable lab container before sending the signal; inspect its final state.

Why it matters: Kill terminates immediately, unlike a graceful stop. Use only on a test container.

Exercise 602 · Force-stop a lab container

Task: Send the configured kill signal to the disposable container kill2.

Solution command

```
docker kill kill2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' kill2`
4. Interpret the result: Confirm the target is the disposable lab container before sending the signal; inspect its final state.

Why it matters: Kill terminates immediately, unlike a graceful stop. Use only on a test container.

Exercise 603 · Force-stop a lab container

Task: Send the configured kill signal to the disposable container kill3.

Solution command

```
docker kill kill3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' kill3`
4. Interpret the result: Confirm the target is the disposable lab container before sending the signal; inspect its final state.

Why it matters: Kill terminates immediately, unlike a graceful stop. Use only on a test container.

Exercise 604 · Force-stop a lab container

Task: Send the configured kill signal to the disposable container kill4.

Solution command

```
docker kill kill4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' kill4`
4. Interpret the result: Confirm the target is the disposable lab container before sending the signal; inspect its final state.

Why it matters: Kill terminates immediately, unlike a graceful stop. Use only on a test container.

Exercise 605 · Force-stop a lab container

Task: Send the configured kill signal to the disposable container kill5.

Solution command

```
docker kill kill5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' kill5`
4. Interpret the result: Confirm the target is the disposable lab container before sending the signal; inspect its final state.

Why it matters: Kill terminates immediately, unlike a graceful stop. Use only on a test container.

Exercise 606 · Force-stop a lab container

Task: Send the configured kill signal to the disposable container kill6.

Solution command

```
docker kill kill6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' kill6`
4. Interpret the result: Confirm the target is the disposable lab container before sending the signal; inspect its final state.

Why it matters: Kill terminates immediately, unlike a graceful stop. Use only on a test container.

Exercise 607 · Force-stop a lab container

Task: Send the configured kill signal to the disposable container kill7.

Solution command

```
docker kill kill7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' kill7`
4. Interpret the result: Confirm the target is the disposable lab container before sending the signal; inspect its final state.

Why it matters: Kill terminates immediately, unlike a graceful stop. Use only on a test container.

Exercise 608 · Force-stop a lab container

Task: Send the configured kill signal to the disposable container kill8.

Solution command

```
docker kill kill8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' kill8`
4. Interpret the result: Confirm the target is the disposable lab container before sending the signal; inspect its final state.

Why it matters: Kill terminates immediately, unlike a graceful stop. Use only on a test container.

Exercise 609 · Force-stop a lab container

Task: Send the configured kill signal to the disposable container kill9.

Solution command

```
docker kill kill9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' kill9`
4. Interpret the result: Confirm the target is the disposable lab container before sending the signal; inspect its final state.

Why it matters: Kill terminates immediately, unlike a graceful stop. Use only on a test container.

Exercise 610 · Force-stop a lab container

Task: Send the configured kill signal to the disposable container kill10.

Solution command

```
docker kill kill10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' kill10`
4. Interpret the result: Confirm the target is the disposable lab container before sending the signal; inspect its final state.

Why it matters: Kill terminates immediately, unlike a graceful stop. Use only on a test container.

Exercise 611 · Wait for a container to exit

Task: Read the exit code of the stopped one-shot container job1.

Solution command

```
docker wait job1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.ExitCode}' job1`
4. Interpret the result: The command blocks until the container exits and prints its exit code.

Why it matters: Exit codes help automation distinguish success from failure.

Exercise 612 · Wait for a container to exit

Task: Read the exit code of the stopped one-shot container job2.

Solution command

```
docker wait job2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.ExitCode}' job2`
4. Interpret the result: The command blocks until the container exits and prints its exit code.

Why it matters: Exit codes help automation distinguish success from failure.

Exercise 613 · Wait for a container to exit

Task: Read the exit code of the stopped one-shot container job3.

Solution command

```
docker wait job3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.ExitCode}' job3`
4. Interpret the result: The command blocks until the container exits and prints its exit code.

Why it matters: Exit codes help automation distinguish success from failure.

Exercise 614 · Wait for a container to exit

Task: Read the exit code of the stopped one-shot container job4.

Solution command

```
docker wait job4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.ExitCode}' job4`
4. Interpret the result: The command blocks until the container exits and prints its exit code.

Why it matters: Exit codes help automation distinguish success from failure.

Exercise 615 · Wait for a container to exit

Task: Read the exit code of the stopped one-shot container job5.

Solution command

```
docker wait job5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.ExitCode}' job5`
4. Interpret the result: The command blocks until the container exits and prints its exit code.

Why it matters: Exit codes help automation distinguish success from failure.

Exercise 616 · Wait for a container to exit

Task: Read the exit code of the stopped one-shot container job6.

Solution command

```
docker wait job6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.ExitCode}' job6`
4. Interpret the result: The command blocks until the container exits and prints its exit code.

Why it matters: Exit codes help automation distinguish success from failure.

Exercise 617 · Wait for a container to exit

Task: Read the exit code of the stopped one-shot container job7.

Solution command

```
docker wait job7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.ExitCode}' job7`
4. Interpret the result: The command blocks until the container exits and prints its exit code.

Why it matters: Exit codes help automation distinguish success from failure.

Exercise 618 · Wait for a container to exit

Task: Read the exit code of the stopped one-shot container job8.

Solution command

```
docker wait job8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.ExitCode}' job8`
4. Interpret the result: The command blocks until the container exits and prints its exit code.

Why it matters: Exit codes help automation distinguish success from failure.

Exercise 619 · Wait for a container to exit

Task: Read the exit code of the stopped one-shot container job9.

Solution command

```
docker wait job9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.ExitCode}' job9`
4. Interpret the result: The command blocks until the container exits and prints its exit code.

Why it matters: Exit codes help automation distinguish success from failure.

Exercise 620 · Wait for a container to exit

Task: Read the exit code of the stopped one-shot container job10.

Solution command

```
docker wait job10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.ExitCode}' job10`
4. Interpret the result: The command blocks until the container exits and prints its exit code.

Why it matters: Exit codes help automation distinguish success from failure.

Module 32: Container identity & terminal attachment

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 621 · Rename a container

Task: Rename the disposable container worker1 to worker-renamed1.

Solution command

```
docker rename worker1 worker-renamed1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=worker-renamed1`
4. Interpret the result: Use a unique new name and verify the new name appears.

Why it matters: Renaming changes the container's name, not its image or internal application configuration.

Exercise 622 · Rename a container

Task: Rename the disposable container worker2 to worker-renamed2.

Solution command

```
docker rename worker2 worker-renamed2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=worker-renamed2`
4. Interpret the result: Use a unique new name and verify the new name appears.

Why it matters: Renaming changes the container's name, not its image or internal application configuration.

Exercise 623 · Rename a container

Task: Rename the disposable container worker3 to worker-renamed3.

Solution command

```
docker rename worker3 worker-renamed3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=worker-renamed3`
4. Interpret the result: Use a unique new name and verify the new name appears.

Why it matters: Renaming changes the container's name, not its image or internal application configuration.

Exercise 624 · Rename a container

Task: Rename the disposable container worker4 to worker-renamed4.

Solution command

```
docker rename worker4 worker-renamed4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=worker-renamed4`
4. Interpret the result: Use a unique new name and verify the new name appears.

Why it matters: Renaming changes the container's name, not its image or internal application configuration.

Exercise 625 · Rename a container

Task: Rename the disposable container worker5 to worker-renamed5.

Solution command

```
docker rename worker5 worker-renamed5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=worker-renamed5`
4. Interpret the result: Use a unique new name and verify the new name appears.

Why it matters: Renaming changes the container's name, not its image or internal application configuration.

Exercise 626 · Rename a container

Task: Rename the disposable container worker6 to worker-renamed6.

Solution command

```
docker rename worker6 worker-renamed6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=worker-renamed6`
4. Interpret the result: Use a unique new name and verify the new name appears.

Why it matters: Renaming changes the container's name, not its image or internal application configuration.

Exercise 627 · Rename a container

Task: Rename the disposable container worker7 to worker-renamed7.

Solution command

```
docker rename worker7 worker-renamed7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=worker-renamed7`
4. Interpret the result: Use a unique new name and verify the new name appears.

Why it matters: Renaming changes the container's name, not its image or internal application configuration.

Exercise 628 · Rename a container

Task: Rename the disposable container worker8 to worker-renamed8.

Solution command

```
docker rename worker8 worker-renamed8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=worker-renamed8`
4. Interpret the result: Use a unique new name and verify the new name appears.

Why it matters: Renaming changes the container's name, not its image or internal application configuration.

Exercise 629 · Rename a container

Task: Rename the disposable container worker9 to worker-renamed9.

Solution command

```
docker rename worker9 worker-renamed9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=worker-renamed9`
4. Interpret the result: Use a unique new name and verify the new name appears.

Why it matters: Renaming changes the container's name, not its image or internal application configuration.

Exercise 630 · Rename a container

Task: Rename the disposable container worker10 to worker-renamed10.

Solution command

```
docker rename worker10 worker-renamed10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker ps -a --filter name=worker-renamed10`
4. Interpret the result: Use a unique new name and verify the new name appears.

Why it matters: Renaming changes the container's name, not its image or internal application configuration.

Exercise 631 · Attach to a container's main process

Task: Attach to the main process of disposable container console1.

Solution command

```
docker attach console1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' console1`
4. Interpret the result: Use only when you intend to interact with the main process; detach behavior depends on the terminal key sequence.

Why it matters: Attach is different from exec: attach connects to the existing main process.

Exercise 632 · Attach to a container's main process

Task: Attach to the main process of disposable container console2.

Solution command

```
docker attach console2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' console2`
4. Interpret the result: Use only when you intend to interact with the main process; detach behavior depends on the terminal key sequence.

Why it matters: Attach is different from exec: attach connects to the existing main process.

Exercise 633 · Attach to a container's main process

Task: Attach to the main process of disposable container console3.

Solution command

```
docker attach console3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' console3`
4. Interpret the result: Use only when you intend to interact with the main process; detach behavior depends on the terminal key sequence.

Why it matters: Attach is different from exec: attach connects to the existing main process.

Exercise 634 · Attach to a container's main process

Task: Attach to the main process of disposable container console4.

Solution command

```
docker attach console4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' console4`
4. Interpret the result: Use only when you intend to interact with the main process; detach behavior depends on the terminal key sequence.

Why it matters: Attach is different from exec: attach connects to the existing main process.

Exercise 635 · Attach to a container's main process

Task: Attach to the main process of disposable container console5.

Solution command

```
docker attach console5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' console5`
4. Interpret the result: Use only when you intend to interact with the main process; detach behavior depends on the terminal key sequence.

Why it matters: Attach is different from exec: attach connects to the existing main process.

Exercise 636 · Attach to a container's main process

Task: Attach to the main process of disposable container console6.

Solution command

```
docker attach console6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' console6`
4. Interpret the result: Use only when you intend to interact with the main process; detach behavior depends on the terminal key sequence.

Why it matters: Attach is different from exec: attach connects to the existing main process.

Exercise 637 · Attach to a container's main process

Task: Attach to the main process of disposable container console7.

Solution command

```
docker attach console7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' console7`
4. Interpret the result: Use only when you intend to interact with the main process; detach behavior depends on the terminal key sequence.

Why it matters: Attach is different from exec: attach connects to the existing main process.

Exercise 638 · Attach to a container's main process

Task: Attach to the main process of disposable container console8.

Solution command

```
docker attach console8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' console8`
4. Interpret the result: Use only when you intend to interact with the main process; detach behavior depends on the terminal key sequence.

Why it matters: Attach is different from exec: attach connects to the existing main process.

Exercise 639 · Attach to a container's main process

Task: Attach to the main process of disposable container console9.

Solution command

```
docker attach console9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' console9`
4. Interpret the result: Use only when you intend to interact with the main process; detach behavior depends on the terminal key sequence.

Why it matters: Attach is different from exec: attach connects to the existing main process.

Exercise 640 · Attach to a container's main process

Task: Attach to the main process of disposable container console10.

Solution command

```
docker attach console10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker inspect --format '{.State.Status}' console10`
4. Interpret the result: Use only when you intend to interact with the main process; detach behavior depends on the terminal key sequence.

Why it matters: Attach is different from exec: attach connects to the existing main process.

Module 33: Image/container filesystem changes

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 641 · Inspect filesystem changes

Task: List filesystem changes made to container app1.

Solution command

```
docker container diff app1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker container diff app1`
4. Interpret the result: Review A (added), C (changed), and D (deleted) entries.

Why it matters: This is a diagnostic view of the container writable layer, not a backup.

Exercise 642 · Inspect filesystem changes

Task: List filesystem changes made to container app2.

Solution command

```
docker container diff app2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker container diff app2`
4. Interpret the result: Review A (added), C (changed), and D (deleted) entries.

Why it matters: This is a diagnostic view of the container writable layer, not a backup.

Exercise 643 · Inspect filesystem changes

Task: List filesystem changes made to container app3.

Solution command

```
docker container diff app3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker container diff app3`
4. Interpret the result: Review A (added), C (changed), and D (deleted) entries.

Why it matters: This is a diagnostic view of the container writable layer, not a backup.

Exercise 644 · Inspect filesystem changes

Task: List filesystem changes made to container app4.

Solution command

```
docker container diff app4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker container diff app4`
4. Interpret the result: Review A (added), C (changed), and D (deleted) entries.

Why it matters: This is a diagnostic view of the container writable layer, not a backup.

Exercise 645 · Inspect filesystem changes

Task: List filesystem changes made to container app5.

Solution command

```
docker container diff app5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker container diff app5`
4. Interpret the result: Review A (added), C (changed), and D (deleted) entries.

Why it matters: This is a diagnostic view of the container writable layer, not a backup.

Exercise 646 · Inspect filesystem changes

Task: List filesystem changes made to container app6.

Solution command

```
docker container diff app6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker container diff app6`
4. Interpret the result: Review A (added), C (changed), and D (deleted) entries.

Why it matters: This is a diagnostic view of the container writable layer, not a backup.

Exercise 647 · Inspect filesystem changes

Task: List filesystem changes made to container app7.

Solution command

```
docker container diff app7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker container diff app7`
4. Interpret the result: Review A (added), C (changed), and D (deleted) entries.

Why it matters: This is a diagnostic view of the container writable layer, not a backup.

Exercise 648 · Inspect filesystem changes

Task: List filesystem changes made to container app8.

Solution command

```
docker container diff app8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker container diff app8`
4. Interpret the result: Review A (added), C (changed), and D (deleted) entries.

Why it matters: This is a diagnostic view of the container writable layer, not a backup.

Exercise 649 · Inspect filesystem changes

Task: List filesystem changes made to container app9.

Solution command

```
docker container diff app9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker container diff app9`
4. Interpret the result: Review A (added), C (changed), and D (deleted) entries.

Why it matters: This is a diagnostic view of the container writable layer, not a backup.

Exercise 650 · Inspect filesystem changes

Task: List filesystem changes made to container app10.

Solution command

```
docker container diff app10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker container diff app10`
4. Interpret the result: Review A (added), C (changed), and D (deleted) entries.

Why it matters: This is a diagnostic view of the container writable layer, not a backup.

Exercise 651 · Commit a lab container as an image

Task: Create a snapshot image local/snapshot:case1 from disposable container app1.

Solution command

```
docker container commit app1 local/snapshot:case1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/snapshot:case1`
4. Interpret the result: Commit only a lab container and inspect the resulting image.

Why it matters: Prefer a Dockerfile for reproducible builds; commit can capture unintended data.

Exercise 652 · Commit a lab container as an image

Task: Create a snapshot image local/snapshot:case2 from disposable container app2.

Solution command

```
docker container commit app2 local/snapshot:case2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/snapshot:case2`
4. Interpret the result: Commit only a lab container and inspect the resulting image.

Why it matters: Prefer a Dockerfile for reproducible builds; commit can capture unintended data.

Exercise 653 · Commit a lab container as an image

Task: Create a snapshot image local/snapshot:case3 from disposable container app3.

Solution command

```
docker container commit app3 local/snapshot:case3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/snapshot:case3`
4. Interpret the result: Commit only a lab container and inspect the resulting image.

Why it matters: Prefer a Dockerfile for reproducible builds; commit can capture unintended data.

Exercise 654 · Commit a lab container as an image

Task: Create a snapshot image local/snapshot:case4 from disposable container app4.

Solution command

```
docker container commit app4 local/snapshot:case4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/snapshot:case4`
4. Interpret the result: Commit only a lab container and inspect the resulting image.

Why it matters: Prefer a Dockerfile for reproducible builds; commit can capture unintended data.

Exercise 655 · Commit a lab container as an image

Task: Create a snapshot image local/snapshot:case5 from disposable container app5.

Solution command

```
docker container commit app5 local/snapshot:case5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/snapshot:case5`
4. Interpret the result: Commit only a lab container and inspect the resulting image.

Why it matters: Prefer a Dockerfile for reproducible builds; commit can capture unintended data.

Exercise 656 · Commit a lab container as an image

Task: Create a snapshot image local/snapshot:case6 from disposable container app6.

Solution command

```
docker container commit app6 local/snapshot:case6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/snapshot:case6`
4. Interpret the result: Commit only a lab container and inspect the resulting image.

Why it matters: Prefer a Dockerfile for reproducible builds; commit can capture unintended data.

Exercise 657 · Commit a lab container as an image

Task: Create a snapshot image local/snapshot:case7 from disposable container app7.

Solution command

```
docker container commit app7 local/snapshot:case7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/snapshot:case7`
4. Interpret the result: Commit only a lab container and inspect the resulting image.

Why it matters: Prefer a Dockerfile for reproducible builds; commit can capture unintended data.

Exercise 658 · Commit a lab container as an image

Task: Create a snapshot image local/snapshot:case8 from disposable container app8.

Solution command

```
docker container commit app8 local/snapshot:case8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/snapshot:case8`
4. Interpret the result: Commit only a lab container and inspect the resulting image.

Why it matters: Prefer a Dockerfile for reproducible builds; commit can capture unintended data.

Exercise 659 · Commit a lab container as an image

Task: Create a snapshot image local/snapshot:case9 from disposable container app9.

Solution command

```
docker container commit app9 local/snapshot:case9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/snapshot:case9`
4. Interpret the result: Commit only a lab container and inspect the resulting image.

Why it matters: Prefer a Dockerfile for reproducible builds; commit can capture unintended data.

Exercise 660 · Commit a lab container as an image

Task: Create a snapshot image local/snapshot:case10 from disposable container app10.

Solution command

```
docker container commit app10 local/snapshot:case10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/snapshot:case10`
4. Interpret the result: Commit only a lab container and inspect the resulting image.

Why it matters: Prefer a Dockerfile for reproducible builds; commit can capture unintended data.

Module 34: Image transfer & import

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 661 · Export a container filesystem

Task: Export the filesystem of disposable container export1 to export1.tar.

Solution command

```
docker container export -o export1.tar export1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh export1.tar`
4. Interpret the result: Confirm the container exists, export its filesystem, and check the archive.

Why it matters: Export does not preserve image history, volumes, or full container configuration.

Exercise 662 · Export a container filesystem

Task: Export the filesystem of disposable container export2 to export2.tar.

Solution command

```
docker container export -o export2.tar export2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh export2.tar`
4. Interpret the result: Confirm the container exists, export its filesystem, and check the archive.

Why it matters: Export does not preserve image history, volumes, or full container configuration.

Exercise 663 · Export a container filesystem

Task: Export the filesystem of disposable container export3 to export3.tar.

Solution command

```
docker container export -o export3.tar export3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh export3.tar`
4. Interpret the result: Confirm the container exists, export its filesystem, and check the archive.

Why it matters: Export does not preserve image history, volumes, or full container configuration.

Exercise 664 · Export a container filesystem

Task: Export the filesystem of disposable container export4 to export4.tar.

Solution command

```
docker container export -o export4.tar export4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh export4.tar`
4. Interpret the result: Confirm the container exists, export its filesystem, and check the archive.

Why it matters: Export does not preserve image history, volumes, or full container configuration.

Exercise 665 · Export a container filesystem

Task: Export the filesystem of disposable container export5 to export5.tar.

Solution command

```
docker container export -o export5.tar export5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh export5.tar`
4. Interpret the result: Confirm the container exists, export its filesystem, and check the archive.

Why it matters: Export does not preserve image history, volumes, or full container configuration.

Exercise 666 · Export a container filesystem

Task: Export the filesystem of disposable container export6 to export6.tar.

Solution command

```
docker container export -o export6.tar export6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh export6.tar`
4. Interpret the result: Confirm the container exists, export its filesystem, and check the archive.

Why it matters: Export does not preserve image history, volumes, or full container configuration.

Exercise 667 · Export a container filesystem

Task: Export the filesystem of disposable container export7 to export7.tar.

Solution command

```
docker container export -o export7.tar export7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh export7.tar`
4. Interpret the result: Confirm the container exists, export its filesystem, and check the archive.

Why it matters: Export does not preserve image history, volumes, or full container configuration.

Exercise 668 · Export a container filesystem

Task: Export the filesystem of disposable container export8 to export8.tar.

Solution command

```
docker container export -o export8.tar export8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh export8.tar`
4. Interpret the result: Confirm the container exists, export its filesystem, and check the archive.

Why it matters: Export does not preserve image history, volumes, or full container configuration.

Exercise 669 · Export a container filesystem

Task: Export the filesystem of disposable container export9 to export9.tar.

Solution command

```
docker container export -o export9.tar export9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh export9.tar`
4. Interpret the result: Confirm the container exists, export its filesystem, and check the archive.

Why it matters: Export does not preserve image history, volumes, or full container configuration.

Exercise 670 · Export a container filesystem

Task: Export the filesystem of disposable container export10 to export10.tar.

Solution command

```
docker container export -o export10.tar export10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `ls -lh export10.tar`
4. Interpret the result: Confirm the container exists, export its filesystem, and check the archive.

Why it matters: Export does not preserve image history, volumes, or full container configuration.

Exercise 671 · Import a filesystem archive as an image

Task: Import a prepared filesystem archive named rootfs1.tar as local/imported:case1.

Solution command

```
docker image import rootfs1.tar local/imported:case1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/imported:case1`
4. Interpret the result: Use an archive you created for the lab and inspect the imported image.

Why it matters: Import creates a filesystem-based image; it is not the same as loading a docker save archive.

Exercise 672 · Import a filesystem archive as an image

Task: Import a prepared filesystem archive named rootfs2.tar as local/imported:case2.

Solution command

```
docker image import rootfs2.tar local/imported:case2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/imported:case2`
4. Interpret the result: Use an archive you created for the lab and inspect the imported image.

Why it matters: Import creates a filesystem-based image; it is not the same as loading a docker save archive.

Exercise 673 · Import a filesystem archive as an image

Task: Import a prepared filesystem archive named rootfs3.tar as local/imported:case3.

Solution command

```
docker image import rootfs3.tar local/imported:case3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/imported:case3`
4. Interpret the result: Use an archive you created for the lab and inspect the imported image.

Why it matters: Import creates a filesystem-based image; it is not the same as loading a docker save archive.

Exercise 674 · Import a filesystem archive as an image

Task: Import a prepared filesystem archive named rootfs4.tar as local/imported:case4.

Solution command

```
docker image import rootfs4.tar local/imported:case4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/imported:case4`
4. Interpret the result: Use an archive you created for the lab and inspect the imported image.

Why it matters: Import creates a filesystem-based image; it is not the same as loading a docker save archive.

Exercise 675 · Import a filesystem archive as an image

Task: Import a prepared filesystem archive named rootfs5.tar as local/imported:case5.

Solution command

```
docker image import rootfs5.tar local/imported:case5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/imported:case5`
4. Interpret the result: Use an archive you created for the lab and inspect the imported image.

Why it matters: Import creates a filesystem-based image; it is not the same as loading a docker save archive.

Exercise 676 · Import a filesystem archive as an image

Task: Import a prepared filesystem archive named rootfs6.tar as local/imported:case6.

Solution command

```
docker image import rootfs6.tar local/imported:case6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/imported:case6`
4. Interpret the result: Use an archive you created for the lab and inspect the imported image.

Why it matters: Import creates a filesystem-based image; it is not the same as loading a docker save archive.

Exercise 677 · Import a filesystem archive as an image

Task: Import a prepared filesystem archive named rootfs7.tar as local/imported:case7.

Solution command

```
docker image import rootfs7.tar local/imported:case7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/imported:case7`
4. Interpret the result: Use an archive you created for the lab and inspect the imported image.

Why it matters: Import creates a filesystem-based image; it is not the same as loading a docker save archive.

Exercise 678 · Import a filesystem archive as an image

Task: Import a prepared filesystem archive named rootfs8.tar as local/imported:case8.

Solution command

```
docker image import rootfs8.tar local/imported:case8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/imported:case8`
4. Interpret the result: Use an archive you created for the lab and inspect the imported image.

Why it matters: Import creates a filesystem-based image; it is not the same as loading a docker save archive.

Exercise 679 · Import a filesystem archive as an image

Task: Import a prepared filesystem archive named rootfs9.tar as local/imported:case9.

Solution command

```
docker image import rootfs9.tar local/imported:case9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/imported:case9`
4. Interpret the result: Use an archive you created for the lab and inspect the imported image.

Why it matters: Import creates a filesystem-based image; it is not the same as loading a docker save archive.

Exercise 680 · Import a filesystem archive as an image

Task: Import a prepared filesystem archive named rootfs10.tar as local/imported:case10.

Solution command

```
docker image import rootfs10.tar local/imported:case10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image inspect local/imported:case10`
4. Interpret the result: Use an archive you created for the lab and inspect the imported image.

Why it matters: Import creates a filesystem-based image; it is not the same as loading a docker save archive.

Module 35: Image transfer & import

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 681 · Load a saved image archive

Task: Load image metadata and layers from alpine-case1.tar.

Solution command

```
docker image load -i alpine-case1.tar
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls`
4. Interpret the result: The archive must have been created with docker image save and be available at the specified path.

Why it matters: Load restores image tags and layers from a Docker image archive.

Exercise 682 · Load a saved image archive

Task: Load image metadata and layers from alpine-case2.tar.

Solution command

```
docker image load -i alpine-case2.tar
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls`
4. Interpret the result: The archive must have been created with docker image save and be available at the specified path.

Why it matters: Load restores image tags and layers from a Docker image archive.

Exercise 683 · Load a saved image archive

Task: Load image metadata and layers from alpine-case3.tar.

Solution command

```
docker image load -i alpine-case3.tar
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls`
4. Interpret the result: The archive must have been created with docker image save and be available at the specified path.

Why it matters: Load restores image tags and layers from a Docker image archive.

Exercise 684 · Load a saved image archive

Task: Load image metadata and layers from alpine-case4.tar.

Solution command

```
docker image load -i alpine-case4.tar
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls`
4. Interpret the result: The archive must have been created with docker image save and be available at the specified path.

Why it matters: Load restores image tags and layers from a Docker image archive.

Exercise 685 · Load a saved image archive

Task: Load image metadata and layers from alpine-case5.tar.

Solution command

```
docker image load -i alpine-case5.tar
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls`
4. Interpret the result: The archive must have been created with docker image save and be available at the specified path.

Why it matters: Load restores image tags and layers from a Docker image archive.

Exercise 686 · Load a saved image archive

Task: Load image metadata and layers from alpine-case6.tar.

Solution command

```
docker image load -i alpine-case6.tar
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls`
4. Interpret the result: The archive must have been created with docker image save and be available at the specified path.

Why it matters: Load restores image tags and layers from a Docker image archive.

Exercise 687 · Load a saved image archive

Task: Load image metadata and layers from alpine-case7.tar.

Solution command

```
docker image load -i alpine-case7.tar
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls`
4. Interpret the result: The archive must have been created with docker image save and be available at the specified path.

Why it matters: Load restores image tags and layers from a Docker image archive.

Exercise 688 · Load a saved image archive

Task: Load image metadata and layers from alpine-case8.tar.

Solution command

```
docker image load -i alpine-case8.tar
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls`
4. Interpret the result: The archive must have been created with docker image save and be available at the specified path.

Why it matters: Load restores image tags and layers from a Docker image archive.

Exercise 689 · Load a saved image archive

Task: Load image metadata and layers from alpine-case9.tar.

Solution command

```
docker image load -i alpine-case9.tar
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls`
4. Interpret the result: The archive must have been created with docker image save and be available at the specified path.

Why it matters: Load restores image tags and layers from a Docker image archive.

Exercise 690 · Load a saved image archive

Task: Load image metadata and layers from alpine-case10.tar.

Solution command

```
docker image load -i alpine-case10.tar
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls`
4. Interpret the result: The archive must have been created with docker image save and be available at the specified path.

Why it matters: Load restores image tags and layers from a Docker image archive.

Exercise 691 · Remove a specific local image tag

Task: Remove the local tag local/alpine-lab:case1 after confirming no needed container depends on it.

Solution command

```
docker image rm local/alpine-lab:case1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Check the exact repository and tag before removal; image layers may remain if referenced elsewhere.

Why it matters: Image removal is different from removing a container.

Exercise 692 · Remove a specific local image tag

Task: Remove the local tag local/alpine-lab:case2 after confirming no needed container depends on it.

Solution command

```
docker image rm local/alpine-lab:case2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Check the exact repository and tag before removal; image layers may remain if referenced elsewhere.

Why it matters: Image removal is different from removing a container.

Exercise 693 · Remove a specific local image tag

Task: Remove the local tag local/alpine-lab:case3 after confirming no needed container depends on it.

Solution command

```
docker image rm local/alpine-lab:case3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Check the exact repository and tag before removal; image layers may remain if referenced elsewhere.

Why it matters: Image removal is different from removing a container.

Exercise 694 · Remove a specific local image tag

Task: Remove the local tag local/alpine-lab:case4 after confirming no needed container depends on it.

Solution command

```
docker image rm local/alpine-lab:case4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Check the exact repository and tag before removal; image layers may remain if referenced elsewhere.

Why it matters: Image removal is different from removing a container.

Exercise 695 · Remove a specific local image tag

Task: Remove the local tag local/alpine-lab:case5 after confirming no needed container depends on it.

Solution command

```
docker image rm local/alpine-lab:case5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Check the exact repository and tag before removal; image layers may remain if referenced elsewhere.

Why it matters: Image removal is different from removing a container.

Exercise 696 · Remove a specific local image tag

Task: Remove the local tag local/alpine-lab:case6 after confirming no needed container depends on it.

Solution command

```
docker image rm local/alpine-lab:case6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Check the exact repository and tag before removal; image layers may remain if referenced elsewhere.

Why it matters: Image removal is different from removing a container.

Exercise 697 · Remove a specific local image tag

Task: Remove the local tag local/alpine-lab:case7 after confirming no needed container depends on it.

Solution command

```
docker image rm local/alpine-lab:case7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Check the exact repository and tag before removal; image layers may remain if referenced elsewhere.

Why it matters: Image removal is different from removing a container.

Exercise 698 · Remove a specific local image tag

Task: Remove the local tag local/alpine-lab:case8 after confirming no needed container depends on it.

Solution command

```
docker image rm local/alpine-lab:case8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Check the exact repository and tag before removal; image layers may remain if referenced elsewhere.

Why it matters: Image removal is different from removing a container.

Exercise 699 · Remove a specific local image tag

Task: Remove the local tag local/alpine-lab:case9 after confirming no needed container depends on it.

Solution command

```
docker image rm local/alpine-lab:case9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Check the exact repository and tag before removal; image layers may remain if referenced elsewhere.

Why it matters: Image removal is different from removing a container.

Exercise 700 · Remove a specific local image tag

Task: Remove the local tag local/alpine-lab:case10 after confirming no needed container depends on it.

Solution command

```
docker image rm local/alpine-lab:case10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker image ls local/alpine-lab`
4. Interpret the result: Check the exact repository and tag before removal; image layers may remain if referenced elsewhere.

Why it matters: Image removal is different from removing a container.

Module 36: Volume lifecycle & safe removal

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 701 · Remove an unused named volume

Task: Remove the test volume data1 only after confirming it is not mounted by any container.

Solution command

```
docker volume rm data1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data1`
4. Interpret the result: Inspect containers and the volume first; remove only the disposable lab volume.

Why it matters: Removing a volume permanently deletes its stored data.

Exercise 702 · Remove an unused named volume

Task: Remove the test volume data2 only after confirming it is not mounted by any container.

Solution command

```
docker volume rm data2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data2`
4. Interpret the result: Inspect containers and the volume first; remove only the disposable lab volume.

Why it matters: Removing a volume permanently deletes its stored data.

Exercise 703 · Remove an unused named volume

Task: Remove the test volume data3 only after confirming it is not mounted by any container.

Solution command

```
docker volume rm data3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data3`
4. Interpret the result: Inspect containers and the volume first; remove only the disposable lab volume.

Why it matters: Removing a volume permanently deletes its stored data.

Exercise 704 · Remove an unused named volume

Task: Remove the test volume data4 only after confirming it is not mounted by any container.

Solution command

```
docker volume rm data4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data4`
4. Interpret the result: Inspect containers and the volume first; remove only the disposable lab volume.

Why it matters: Removing a volume permanently deletes its stored data.

Exercise 705 · Remove an unused named volume

Task: Remove the test volume data5 only after confirming it is not mounted by any container.

Solution command

```
docker volume rm data5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data5`
4. Interpret the result: Inspect containers and the volume first; remove only the disposable lab volume.

Why it matters: Removing a volume permanently deletes its stored data.

Exercise 706 · Remove an unused named volume

Task: Remove the test volume data6 only after confirming it is not mounted by any container.

Solution command

```
docker volume rm data6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data6`
4. Interpret the result: Inspect containers and the volume first; remove only the disposable lab volume.

Why it matters: Removing a volume permanently deletes its stored data.

Exercise 707 · Remove an unused named volume

Task: Remove the test volume data7 only after confirming it is not mounted by any container.

Solution command

```
docker volume rm data7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data7`
4. Interpret the result: Inspect containers and the volume first; remove only the disposable lab volume.

Why it matters: Removing a volume permanently deletes its stored data.

Exercise 708 · Remove an unused named volume

Task: Remove the test volume data8 only after confirming it is not mounted by any container.

Solution command

```
docker volume rm data8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data8`
4. Interpret the result: Inspect containers and the volume first; remove only the disposable lab volume.

Why it matters: Removing a volume permanently deletes its stored data.

Exercise 709 · Remove an unused named volume

Task: Remove the test volume data9 only after confirming it is not mounted by any container.

Solution command

```
docker volume rm data9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data9`
4. Interpret the result: Inspect containers and the volume first; remove only the disposable lab volume.

Why it matters: Removing a volume permanently deletes its stored data.

Exercise 710 · Remove an unused named volume

Task: Remove the test volume data10 only after confirming it is not mounted by any container.

Solution command

```
docker volume rm data10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker volume ls --filter name=data10`
4. Interpret the result: Inspect containers and the volume first; remove only the disposable lab volume.

Why it matters: Removing a volume permanently deletes its stored data.

Exercise 711 · Review dangling volumes before pruning

Task: List unused volumes and decide whether any are safe to remove for cleanup case 1.

Solution command

```
docker volume ls --filter dangling=true
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df -v`
4. Interpret the result: Inventory each volume and its owner/purpose before choosing a removal command.

Why it matters: Do not prune volumes blindly: volumes can contain important persistent data.

Exercise 712 · Review dangling volumes before pruning

Task: List unused volumes and decide whether any are safe to remove for cleanup case 2.

Solution command

```
docker volume ls --filter dangling=true
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df -v`
4. Interpret the result: Inventory each volume and its owner/purpose before choosing a removal command.

Why it matters: Do not prune volumes blindly: volumes can contain important persistent data.

Exercise 713 · Review dangling volumes before pruning

Task: List unused volumes and decide whether any are safe to remove for cleanup case 3.

Solution command

```
docker volume ls --filter dangling=true
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df -v`
4. Interpret the result: Inventory each volume and its owner/purpose before choosing a removal command.

Why it matters: Do not prune volumes blindly: volumes can contain important persistent data.

Exercise 714 · Review dangling volumes before pruning

Task: List unused volumes and decide whether any are safe to remove for cleanup case 4.

Solution command

```
docker volume ls --filter dangling=true
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df -v`
4. Interpret the result: Inventory each volume and its owner/purpose before choosing a removal command.

Why it matters: Do not prune volumes blindly: volumes can contain important persistent data.

Exercise 715 · Review dangling volumes before pruning

Task: List unused volumes and decide whether any are safe to remove for cleanup case 5.

Solution command

```
docker volume ls --filter dangling=true
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df -v`
4. Interpret the result: Inventory each volume and its owner/purpose before choosing a removal command.

Why it matters: Do not prune volumes blindly: volumes can contain important persistent data.

Exercise 716 · Review dangling volumes before pruning

Task: List unused volumes and decide whether any are safe to remove for cleanup case 6.

Solution command

```
docker volume ls --filter dangling=true
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df -v`
4. Interpret the result: Inventory each volume and its owner/purpose before choosing a removal command.

Why it matters: Do not prune volumes blindly: volumes can contain important persistent data.

Exercise 717 · Review dangling volumes before pruning

Task: List unused volumes and decide whether any are safe to remove for cleanup case 7.

Solution command

```
docker volume ls --filter dangling=true
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df -v`
4. Interpret the result: Inventory each volume and its owner/purpose before choosing a removal command.

Why it matters: Do not prune volumes blindly: volumes can contain important persistent data.

Exercise 718 · Review dangling volumes before pruning

Task: List unused volumes and decide whether any are safe to remove for cleanup case 8.

Solution command

```
docker volume ls --filter dangling=true
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df -v`
4. Interpret the result: Inventory each volume and its owner/purpose before choosing a removal command.

Why it matters: Do not prune volumes blindly: volumes can contain important persistent data.

Exercise 719 · Review dangling volumes before pruning

Task: List unused volumes and decide whether any are safe to remove for cleanup case 9.

Solution command

```
docker volume ls --filter dangling=true
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df -v`
4. Interpret the result: Inventory each volume and its owner/purpose before choosing a removal command.

Why it matters: Do not prune volumes blindly: volumes can contain important persistent data.

Exercise 720 · Review dangling volumes before pruning

Task: List unused volumes and decide whether any are safe to remove for cleanup case 10.

Solution command

```
docker volume ls --filter dangling=true
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df -v`
4. Interpret the result: Inventory each volume and its owner/purpose before choosing a removal command.

Why it matters: Do not prune volumes blindly: volumes can contain important persistent data.

Module 37: Network lifecycle & cleanup

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 721 · Remove an unused custom network

Task: Remove custom network labnet1 after disconnecting/removing its test containers.

Solution command

```
docker network rm labnet1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network ls`
4. Interpret the result: Confirm the network is not in use and is not a built-in network.

Why it matters: Removing a network disconnects workloads and should be scoped to your lab.

Exercise 722 · Remove an unused custom network

Task: Remove custom network labnet2 after disconnecting/removing its test containers.

Solution command

```
docker network rm labnet2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network ls`
4. Interpret the result: Confirm the network is not in use and is not a built-in network.

Why it matters: Removing a network disconnects workloads and should be scoped to your lab.

Exercise 723 · Remove an unused custom network

Task: Remove custom network labnet3 after disconnecting/removing its test containers.

Solution command

```
docker network rm labnet3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network ls`
4. Interpret the result: Confirm the network is not in use and is not a built-in network.

Why it matters: Removing a network disconnects workloads and should be scoped to your lab.

Exercise 724 · Remove an unused custom network

Task: Remove custom network labnet4 after disconnecting/removing its test containers.

Solution command

```
docker network rm labnet4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network ls`
4. Interpret the result: Confirm the network is not in use and is not a built-in network.

Why it matters: Removing a network disconnects workloads and should be scoped to your lab.

Exercise 725 · Remove an unused custom network

Task: Remove custom network labnet5 after disconnecting/removing its test containers.

Solution command

```
docker network rm labnet5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network ls`
4. Interpret the result: Confirm the network is not in use and is not a built-in network.

Why it matters: Removing a network disconnects workloads and should be scoped to your lab.

Exercise 726 · Remove an unused custom network

Task: Remove custom network labnet6 after disconnecting/removing its test containers.

Solution command

```
docker network rm labnet6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network ls`
4. Interpret the result: Confirm the network is not in use and is not a built-in network.

Why it matters: Removing a network disconnects workloads and should be scoped to your lab.

Exercise 727 · Remove an unused custom network

Task: Remove custom network labnet7 after disconnecting/removing its test containers.

Solution command

```
docker network rm labnet7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network ls`
4. Interpret the result: Confirm the network is not in use and is not a built-in network.

Why it matters: Removing a network disconnects workloads and should be scoped to your lab.

Exercise 728 · Remove an unused custom network

Task: Remove custom network labnet8 after disconnecting/removing its test containers.

Solution command

```
docker network rm labnet8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network ls`
4. Interpret the result: Confirm the network is not in use and is not a built-in network.

Why it matters: Removing a network disconnects workloads and should be scoped to your lab.

Exercise 729 · Remove an unused custom network

Task: Remove custom network labnet9 after disconnecting/removing its test containers.

Solution command

```
docker network rm labnet9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network ls`
4. Interpret the result: Confirm the network is not in use and is not a built-in network.

Why it matters: Removing a network disconnects workloads and should be scoped to your lab.

Exercise 730 · Remove an unused custom network

Task: Remove custom network labnet10 after disconnecting/removing its test containers.

Solution command

```
docker network rm labnet10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network ls`
4. Interpret the result: Confirm the network is not in use and is not a built-in network.

Why it matters: Removing a network disconnects workloads and should be scoped to your lab.

Exercise 731 · Review networks before cleanup

Task: List custom networks and identify stale lab networks for case 1.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet1`
4. Interpret the result: Inspect the candidate network and connected containers before deciding whether to remove it.

Why it matters: Inspection first prevents accidental disruption to other projects.

Exercise 732 · Review networks before cleanup

Task: List custom networks and identify stale lab networks for case 2.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet2`
4. Interpret the result: Inspect the candidate network and connected containers before deciding whether to remove it.

Why it matters: Inspection first prevents accidental disruption to other projects.

Exercise 733 · Review networks before cleanup

Task: List custom networks and identify stale lab networks for case 3.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet3`
4. Interpret the result: Inspect the candidate network and connected containers before deciding whether to remove it.

Why it matters: Inspection first prevents accidental disruption to other projects.

Exercise 734 · Review networks before cleanup

Task: List custom networks and identify stale lab networks for case 4.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet4`
4. Interpret the result: Inspect the candidate network and connected containers before deciding whether to remove it.

Why it matters: Inspection first prevents accidental disruption to other projects.

Exercise 735 · Review networks before cleanup

Task: List custom networks and identify stale lab networks for case 5.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet5`
4. Interpret the result: Inspect the candidate network and connected containers before deciding whether to remove it.

Why it matters: Inspection first prevents accidental disruption to other projects.

Exercise 736 · Review networks before cleanup

Task: List custom networks and identify stale lab networks for case 6.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet6`
4. Interpret the result: Inspect the candidate network and connected containers before deciding whether to remove it.

Why it matters: Inspection first prevents accidental disruption to other projects.

Exercise 737 · Review networks before cleanup

Task: List custom networks and identify stale lab networks for case 7.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet7`
4. Interpret the result: Inspect the candidate network and connected containers before deciding whether to remove it.

Why it matters: Inspection first prevents accidental disruption to other projects.

Exercise 738 · Review networks before cleanup

Task: List custom networks and identify stale lab networks for case 8.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet8`
4. Interpret the result: Inspect the candidate network and connected containers before deciding whether to remove it.

Why it matters: Inspection first prevents accidental disruption to other projects.

Exercise 739 · Review networks before cleanup

Task: List custom networks and identify stale lab networks for case 9.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet9`
4. Interpret the result: Inspect the candidate network and connected containers before deciding whether to remove it.

Why it matters: Inspection first prevents accidental disruption to other projects.

Exercise 740 · Review networks before cleanup

Task: List custom networks and identify stale lab networks for case 10.

Solution command

```
docker network ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker network inspect labnet10`
4. Interpret the result: Inspect the candidate network and connected containers before deciding whether to remove it.

Why it matters: Inspection first prevents accidental disruption to other projects.

Module 38: Compose build & image management

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 741 · Build Compose service images

Task: Build images declared by the current Compose project for case1.

Solution command

```
docker compose build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Run from the intended project directory and check that services with build sections complete successfully.

Why it matters: Compose build uses build settings from the Compose model.

Exercise 742 · Build Compose service images

Task: Build images declared by the current Compose project for case2.

Solution command

```
docker compose build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Run from the intended project directory and check that services with build sections complete successfully.

Why it matters: Compose build uses build settings from the Compose model.

Exercise 743 · Build Compose service images

Task: Build images declared by the current Compose project for case3.

Solution command

```
docker compose build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Run from the intended project directory and check that services with build sections complete successfully.

Why it matters: Compose build uses build settings from the Compose model.

Exercise 744 · Build Compose service images

Task: Build images declared by the current Compose project for case4.

Solution command

```
docker compose build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Run from the intended project directory and check that services with build sections complete successfully.

Why it matters: Compose build uses build settings from the Compose model.

Exercise 745 · Build Compose service images

Task: Build images declared by the current Compose project for case5.

Solution command

```
docker compose build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Run from the intended project directory and check that services with build sections complete successfully.

Why it matters: Compose build uses build settings from the Compose model.

Exercise 746 · Build Compose service images

Task: Build images declared by the current Compose project for case6.

Solution command

```
docker compose build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Run from the intended project directory and check that services with build sections complete successfully.

Why it matters: Compose build uses build settings from the Compose model.

Exercise 747 · Build Compose service images

Task: Build images declared by the current Compose project for case7.

Solution command

```
docker compose build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Run from the intended project directory and check that services with build sections complete successfully.

Why it matters: Compose build uses build settings from the Compose model.

Exercise 748 · Build Compose service images

Task: Build images declared by the current Compose project for case8.

Solution command

```
docker compose build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Run from the intended project directory and check that services with build sections complete successfully.

Why it matters: Compose build uses build settings from the Compose model.

Exercise 749 · Build Compose service images

Task: Build images declared by the current Compose project for case9.

Solution command

```
docker compose build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Run from the intended project directory and check that services with build sections complete successfully.

Why it matters: Compose build uses build settings from the Compose model.

Exercise 750 · Build Compose service images

Task: Build images declared by the current Compose project for case10.

Solution command

```
docker compose build
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Run from the intended project directory and check that services with build sections complete successfully.

Why it matters: Compose build uses build settings from the Compose model.

Exercise 751 · Pull Compose service images

Task: Pull images required by the current Compose project before deployment, case1.

Solution command

```
docker compose pull
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Review the Compose file's image references and confirm the pulls complete.

Why it matters: Pull downloads images but does not start services.

Exercise 752 · Pull Compose service images

Task: Pull images required by the current Compose project before deployment, case2.

Solution command

```
docker compose pull
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Review the Compose file's image references and confirm the pulls complete.

Why it matters: Pull downloads images but does not start services.

Exercise 753 · Pull Compose service images

Task: Pull images required by the current Compose project before deployment, case3.

Solution command

```
docker compose pull
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Review the Compose file's image references and confirm the pulls complete.

Why it matters: Pull downloads images but does not start services.

Exercise 754 · Pull Compose service images

Task: Pull images required by the current Compose project before deployment, case4.

Solution command

```
docker compose pull
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Review the Compose file's image references and confirm the pulls complete.

Why it matters: Pull downloads images but does not start services.

Exercise 755 · Pull Compose service images

Task: Pull images required by the current Compose project before deployment, case5.

Solution command

```
docker compose pull
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Review the Compose file's image references and confirm the pulls complete.

Why it matters: Pull downloads images but does not start services.

Exercise 756 · Pull Compose service images

Task: Pull images required by the current Compose project before deployment, case6.

Solution command

```
docker compose pull
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Review the Compose file's image references and confirm the pulls complete.

Why it matters: Pull downloads images but does not start services.

Exercise 757 · Pull Compose service images

Task: Pull images required by the current Compose project before deployment, case7.

Solution command

```
docker compose pull
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Review the Compose file's image references and confirm the pulls complete.

Why it matters: Pull downloads images but does not start services.

Exercise 758 · Pull Compose service images

Task: Pull images required by the current Compose project before deployment, case8.

Solution command

```
docker compose pull
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Review the Compose file's image references and confirm the pulls complete.

Why it matters: Pull downloads images but does not start services.

Exercise 759 · Pull Compose service images

Task: Pull images required by the current Compose project before deployment, case9.

Solution command

```
docker compose pull
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Review the Compose file's image references and confirm the pulls complete.

Why it matters: Pull downloads images but does not start services.

Exercise 760 · Pull Compose service images

Task: Pull images required by the current Compose project before deployment, case10.

Solution command

```
docker compose pull
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose images`
4. Interpret the result: Review the Compose file's image references and confirm the pulls complete.

Why it matters: Pull downloads images but does not start services.

Module 39: Compose one-off jobs & profiles

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 761 · Run a one-off Compose service

Task: Run a one-off command in service worker using the current Compose project, case1.

Solution command

```
docker compose run --rm worker sh -c 'echo case1'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: The worker service must exist in the Compose file; --rm removes the one-off container after it exits.

Why it matters: Use run for one-off tasks and exec for commands inside an already running service.

Exercise 762 · Run a one-off Compose service

Task: Run a one-off command in service worker using the current Compose project, case2.

Solution command

```
docker compose run --rm worker sh -c 'echo case2'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: The worker service must exist in the Compose file; --rm removes the one-off container after it exits.

Why it matters: Use run for one-off tasks and exec for commands inside an already running service.

Exercise 763 · Run a one-off Compose service

Task: Run a one-off command in service worker using the current Compose project, case3.

Solution command

```
docker compose run --rm worker sh -c 'echo case3'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: The worker service must exist in the Compose file; --rm removes the one-off container after it exits.

Why it matters: Use run for one-off tasks and exec for commands inside an already running service.

Exercise 764 · Run a one-off Compose service

Task: Run a one-off command in service worker using the current Compose project, case4.

Solution command

```
docker compose run --rm worker sh -c 'echo case4'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: The worker service must exist in the Compose file; --rm removes the one-off container after it exits.

Why it matters: Use run for one-off tasks and exec for commands inside an already running service.

Exercise 765 · Run a one-off Compose service

Task: Run a one-off command in service worker using the current Compose project, case5.

Solution command

```
docker compose run --rm worker sh -c 'echo case5'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: The worker service must exist in the Compose file; --rm removes the one-off container after it exits.

Why it matters: Use run for one-off tasks and exec for commands inside an already running service.

Exercise 766 · Run a one-off Compose service

Task: Run a one-off command in service worker using the current Compose project, case6.

Solution command

```
docker compose run --rm worker sh -c 'echo case6'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: The worker service must exist in the Compose file; --rm removes the one-off container after it exits.

Why it matters: Use run for one-off tasks and exec for commands inside an already running service.

Exercise 767 · Run a one-off Compose service

Task: Run a one-off command in service worker using the current Compose project, case7.

Solution command

```
docker compose run --rm worker sh -c 'echo case7'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: The worker service must exist in the Compose file; --rm removes the one-off container after it exits.

Why it matters: Use run for one-off tasks and exec for commands inside an already running service.

Exercise 768 · Run a one-off Compose service

Task: Run a one-off command in service worker using the current Compose project, case8.

Solution command

```
docker compose run --rm worker sh -c 'echo case8'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: The worker service must exist in the Compose file; --rm removes the one-off container after it exits.

Why it matters: Use run for one-off tasks and exec for commands inside an already running service.

Exercise 769 · Run a one-off Compose service

Task: Run a one-off command in service worker using the current Compose project, case9.

Solution command

```
docker compose run --rm worker sh -c 'echo case9'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: The worker service must exist in the Compose file; --rm removes the one-off container after it exits.

Why it matters: Use run for one-off tasks and exec for commands inside an already running service.

Exercise 770 · Run a one-off Compose service

Task: Run a one-off command in service worker using the current Compose project, case10.

Solution command

```
docker compose run --rm worker sh -c 'echo case10'
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose ps --all`
4. Interpret the result: The worker service must exist in the Compose file; --rm removes the one-off container after it exits.

Why it matters: Use run for one-off tasks and exec for commands inside an already running service.

Exercise 771 · List Compose profiles in the resolved model

Task: Inspect the resolved Compose configuration and identify services/profile declarations for case1.

Solution command

```
docker compose config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose config --services`
4. Interpret the result: Review the rendered model and profile declarations in the Compose file.

Why it matters: Compose features and flags depend on the installed Compose plugin version.

Exercise 772 · List Compose profiles in the resolved model

Task: Inspect the resolved Compose configuration and identify services/profile declarations for case2.

Solution command

```
docker compose config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose config --services`
4. Interpret the result: Review the rendered model and profile declarations in the Compose file.

Why it matters: Compose features and flags depend on the installed Compose plugin version.

Exercise 773 · List Compose profiles in the resolved model

Task: Inspect the resolved Compose configuration and identify services/profile declarations for case3.

Solution command

```
docker compose config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose config --services`
4. Interpret the result: Review the rendered model and profile declarations in the Compose file.

Why it matters: Compose features and flags depend on the installed Compose plugin version.

Exercise 774 · List Compose profiles in the resolved model

Task: Inspect the resolved Compose configuration and identify services/profile declarations for case4.

Solution command

```
docker compose config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose config --services`
4. Interpret the result: Review the rendered model and profile declarations in the Compose file.

Why it matters: Compose features and flags depend on the installed Compose plugin version.

Exercise 775 · List Compose profiles in the resolved model

Task: Inspect the resolved Compose configuration and identify services/profile declarations for case5.

Solution command

```
docker compose config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose config --services`
4. Interpret the result: Review the rendered model and profile declarations in the Compose file.

Why it matters: Compose features and flags depend on the installed Compose plugin version.

Exercise 776 · List Compose profiles in the resolved model

Task: Inspect the resolved Compose configuration and identify services/profile declarations for case6.

Solution command

```
docker compose config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose config --services`
4. Interpret the result: Review the rendered model and profile declarations in the Compose file.

Why it matters: Compose features and flags depend on the installed Compose plugin version.

Exercise 777 · List Compose profiles in the resolved model

Task: Inspect the resolved Compose configuration and identify services/profile declarations for case7.

Solution command

```
docker compose config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose config --services`
4. Interpret the result: Review the rendered model and profile declarations in the Compose file.

Why it matters: Compose features and flags depend on the installed Compose plugin version.

Exercise 778 · List Compose profiles in the resolved model

Task: Inspect the resolved Compose configuration and identify services/profile declarations for case8.

Solution command

```
docker compose config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose config --services`
4. Interpret the result: Review the rendered model and profile declarations in the Compose file.

Why it matters: Compose features and flags depend on the installed Compose plugin version.

Exercise 779 · List Compose profiles in the resolved model

Task: Inspect the resolved Compose configuration and identify services/profile declarations for case9.

Solution command

```
docker compose config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose config --services`
4. Interpret the result: Review the rendered model and profile declarations in the Compose file.

Why it matters: Compose features and flags depend on the installed Compose plugin version.

Exercise 780 · List Compose profiles in the resolved model

Task: Inspect the resolved Compose configuration and identify services/profile declarations for case10.

Solution command

```
docker compose config
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker compose config --services`
4. Interpret the result: Review the rendered model and profile declarations in the Compose file.

Why it matters: Compose features and flags depend on the installed Compose plugin version.

Module 40: Swarm service updates & rollback

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 781 · Update a Swarm service image

Task: Update service web1 to the image nginx:alpine and watch rollout status.

Solution command

```
docker service update --image nginx:alpine web1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web1`
4. Interpret the result: Run on a Swarm manager and inspect task history until the desired state is reached.

Why it matters: Use a specific version tag or digest for controlled production rollouts.

Exercise 782 · Update a Swarm service image

Task: Update service web2 to the image nginx:alpine and watch rollout status.

Solution command

```
docker service update --image nginx:alpine web2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web2`
4. Interpret the result: Run on a Swarm manager and inspect task history until the desired state is reached.

Why it matters: Use a specific version tag or digest for controlled production rollouts.

Exercise 783 · Update a Swarm service image

Task: Update service web3 to the image nginx:alpine and watch rollout status.

Solution command

```
docker service update --image nginx:alpine web3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web3`
4. Interpret the result: Run on a Swarm manager and inspect task history until the desired state is reached.

Why it matters: Use a specific version tag or digest for controlled production rollouts.

Exercise 784 · Update a Swarm service image

Task: Update service web4 to the image nginx:alpine and watch rollout status.

Solution command

```
docker service update --image nginx:alpine web4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web4`
4. Interpret the result: Run on a Swarm manager and inspect task history until the desired state is reached.

Why it matters: Use a specific version tag or digest for controlled production rollouts.

Exercise 785 · Update a Swarm service image

Task: Update service web5 to the image nginx:alpine and watch rollout status.

Solution command

```
docker service update --image nginx:alpine web5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web5`
4. Interpret the result: Run on a Swarm manager and inspect task history until the desired state is reached.

Why it matters: Use a specific version tag or digest for controlled production rollouts.

Exercise 786 · Update a Swarm service image

Task: Update service web6 to the image nginx:alpine and watch rollout status.

Solution command

```
docker service update --image nginx:alpine web6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web6`
4. Interpret the result: Run on a Swarm manager and inspect task history until the desired state is reached.

Why it matters: Use a specific version tag or digest for controlled production rollouts.

Exercise 787 · Update a Swarm service image

Task: Update service web7 to the image nginx:alpine and watch rollout status.

Solution command

```
docker service update --image nginx:alpine web7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web7`
4. Interpret the result: Run on a Swarm manager and inspect task history until the desired state is reached.

Why it matters: Use a specific version tag or digest for controlled production rollouts.

Exercise 788 · Update a Swarm service image

Task: Update service web8 to the image nginx:alpine and watch rollout status.

Solution command

```
docker service update --image nginx:alpine web8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web8`
4. Interpret the result: Run on a Swarm manager and inspect task history until the desired state is reached.

Why it matters: Use a specific version tag or digest for controlled production rollouts.

Exercise 789 · Update a Swarm service image

Task: Update service web9 to the image nginx:alpine and watch rollout status.

Solution command

```
docker service update --image nginx:alpine web9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web9`
4. Interpret the result: Run on a Swarm manager and inspect task history until the desired state is reached.

Why it matters: Use a specific version tag or digest for controlled production rollouts.

Exercise 790 · Update a Swarm service image

Task: Update service web10 to the image nginx:alpine and watch rollout status.

Solution command

```
docker service update --image nginx:alpine web10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web10`
4. Interpret the result: Run on a Swarm manager and inspect task history until the desired state is reached.

Why it matters: Use a specific version tag or digest for controlled production rollouts.

Exercise 791 · Roll back a Swarm service update

Task: Roll service web1 back to its previous service specification in a lab.

Solution command

```
docker service rollback web1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web1`
4. Interpret the result: Use only if a previous service update exists; inspect tasks and service state after rollback.

Why it matters: Rollback reverts the service specification to the previous version when available.

Exercise 792 · Roll back a Swarm service update

Task: Roll service web2 back to its previous service specification in a lab.

Solution command

```
docker service rollback web2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web2`
4. Interpret the result: Use only if a previous service update exists; inspect tasks and service state after rollback.

Why it matters: Rollback reverts the service specification to the previous version when available.

Exercise 793 · Roll back a Swarm service update

Task: Roll service web3 back to its previous service specification in a lab.

Solution command

```
docker service rollback web3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web3`
4. Interpret the result: Use only if a previous service update exists; inspect tasks and service state after rollback.

Why it matters: Rollback reverts the service specification to the previous version when available.

Exercise 794 · Roll back a Swarm service update

Task: Roll service web4 back to its previous service specification in a lab.

Solution command

```
docker service rollback web4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web4`
4. Interpret the result: Use only if a previous service update exists; inspect tasks and service state after rollback.

Why it matters: Rollback reverts the service specification to the previous version when available.

Exercise 795 · Roll back a Swarm service update

Task: Roll service web5 back to its previous service specification in a lab.

Solution command

```
docker service rollback web5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web5`
4. Interpret the result: Use only if a previous service update exists; inspect tasks and service state after rollback.

Why it matters: Rollback reverts the service specification to the previous version when available.

Exercise 796 · Roll back a Swarm service update

Task: Roll service web6 back to its previous service specification in a lab.

Solution command

```
docker service rollback web6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web6`
4. Interpret the result: Use only if a previous service update exists; inspect tasks and service state after rollback.

Why it matters: Rollback reverts the service specification to the previous version when available.

Exercise 797 · Roll back a Swarm service update

Task: Roll service web7 back to its previous service specification in a lab.

Solution command

```
docker service rollback web7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web7`
4. Interpret the result: Use only if a previous service update exists; inspect tasks and service state after rollback.

Why it matters: Rollback reverts the service specification to the previous version when available.

Exercise 798 · Roll back a Swarm service update

Task: Roll service web8 back to its previous service specification in a lab.

Solution command

```
docker service rollback web8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web8`
4. Interpret the result: Use only if a previous service update exists; inspect tasks and service state after rollback.

Why it matters: Rollback reverts the service specification to the previous version when available.

Exercise 799 · Roll back a Swarm service update

Task: Roll service web9 back to its previous service specification in a lab.

Solution command

```
docker service rollback web9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web9`
4. Interpret the result: Use only if a previous service update exists; inspect tasks and service state after rollback.

Why it matters: Rollback reverts the service specification to the previous version when available.

Exercise 800 · Roll back a Swarm service update

Task: Roll service web10 back to its previous service specification in a lab.

Solution command

```
docker service rollback web10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker service ps web10`
4. Interpret the result: Use only if a previous service update exists; inspect tasks and service state after rollback.

Why it matters: Rollback reverts the service specification to the previous version when available.

Module 41: Swarm node operations

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 801 · Drain a Swarm node

Task: Mark node NODE_NAME unavailable for new tasks while draining existing tasks in a lab.

Solution command

```
docker node update --availability drain NODE_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node ls`
4. Interpret the result: Replace NODE_NAME with a node shown by docker node ls; run this only on a lab cluster.

Why it matters: Draining can reschedule workloads and disrupt services; never practice on production nodes.

Exercise 802 · Drain a Swarm node

Task: Mark node NODE_NAME unavailable for new tasks while draining existing tasks in a lab.

Solution command

```
docker node update --availability drain NODE_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node ls`
4. Interpret the result: Replace NODE_NAME with a node shown by docker node ls; run this only on a lab cluster.

Why it matters: Draining can reschedule workloads and disrupt services; never practice on production nodes.

Exercise 803 · Drain a Swarm node

Task: Mark node NODE_NAME unavailable for new tasks while draining existing tasks in a lab.

Solution command

```
docker node update --availability drain NODE_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node ls`
4. Interpret the result: Replace NODE_NAME with a node shown by docker node ls; run this only on a lab cluster.

Why it matters: Draining can reschedule workloads and disrupt services; never practice on production nodes.

Exercise 804 · Drain a Swarm node

Task: Mark node NODE_NAME unavailable for new tasks while draining existing tasks in a lab.

Solution command

```
docker node update --availability drain NODE_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node ls`
4. Interpret the result: Replace NODE_NAME with a node shown by docker node ls; run this only on a lab cluster.

Why it matters: Draining can reschedule workloads and disrupt services; never practice on production nodes.

Exercise 805 · Drain a Swarm node

Task: Mark node NODE_NAME unavailable for new tasks while draining existing tasks in a lab.

Solution command

```
docker node update --availability drain NODE_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node ls`
4. Interpret the result: Replace NODE_NAME with a node shown by docker node ls; run this only on a lab cluster.

Why it matters: Draining can reschedule workloads and disrupt services; never practice on production nodes.

Exercise 806 · Drain a Swarm node

Task: Mark node NODE_NAME unavailable for new tasks while draining existing tasks in a lab.

Solution command

```
docker node update --availability drain NODE_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node ls`
4. Interpret the result: Replace NODE_NAME with a node shown by docker node ls; run this only on a lab cluster.

Why it matters: Draining can reschedule workloads and disrupt services; never practice on production nodes.

Exercise 807 · Drain a Swarm node

Task: Mark node NODE_NAME unavailable for new tasks while draining existing tasks in a lab.

Solution command

```
docker node update --availability drain NODE_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node ls`
4. Interpret the result: Replace NODE_NAME with a node shown by docker node ls; run this only on a lab cluster.

Why it matters: Draining can reschedule workloads and disrupt services; never practice on production nodes.

Exercise 808 · Drain a Swarm node

Task: Mark node NODE_NAME unavailable for new tasks while draining existing tasks in a lab.

Solution command

```
docker node update --availability drain NODE_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node ls`
4. Interpret the result: Replace NODE_NAME with a node shown by docker node ls; run this only on a lab cluster.

Why it matters: Draining can reschedule workloads and disrupt services; never practice on production nodes.

Exercise 809 · Drain a Swarm node

Task: Mark node NODE_NAME unavailable for new tasks while draining existing tasks in a lab.

Solution command

```
docker node update --availability drain NODE_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node ls`
4. Interpret the result: Replace NODE_NAME with a node shown by docker node ls; run this only on a lab cluster.

Why it matters: Draining can reschedule workloads and disrupt services; never practice on production nodes.

Exercise 810 · Drain a Swarm node

Task: Mark node NODE_NAME unavailable for new tasks while draining existing tasks in a lab.

Solution command

```
docker node update --availability drain NODE_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker node ls`
4. Interpret the result: Replace NODE_NAME with a node shown by docker node ls; run this only on a lab cluster.

Why it matters: Draining can reschedule workloads and disrupt services; never practice on production nodes.

Exercise 811 · Leave a Swarm cluster

Task: Remove a disposable worker node from its Swarm after verifying its role.

Solution command

```
docker swarm leave
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run on the intended worker node, not casually on a manager; verify the local state afterward.

Why it matters: Leaving a cluster changes orchestration membership.

Exercise 812 · Leave a Swarm cluster

Task: Remove a disposable worker node from its Swarm after verifying its role.

Solution command

```
docker swarm leave
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run on the intended worker node, not casually on a manager; verify the local state afterward.

Why it matters: Leaving a cluster changes orchestration membership.

Exercise 813 · Leave a Swarm cluster

Task: Remove a disposable worker node from its Swarm after verifying its role.

Solution command

```
docker swarm leave
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run on the intended worker node, not casually on a manager; verify the local state afterward.

Why it matters: Leaving a cluster changes orchestration membership.

Exercise 814 · Leave a Swarm cluster

Task: Remove a disposable worker node from its Swarm after verifying its role.

Solution command

```
docker swarm leave
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run on the intended worker node, not casually on a manager; verify the local state afterward.

Why it matters: Leaving a cluster changes orchestration membership.

Exercise 815 · Leave a Swarm cluster

Task: Remove a disposable worker node from its Swarm after verifying its role.

Solution command

```
docker swarm leave
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run on the intended worker node, not casually on a manager; verify the local state afterward.

Why it matters: Leaving a cluster changes orchestration membership.

Exercise 816 · Leave a Swarm cluster

Task: Remove a disposable worker node from its Swarm after verifying its role.

Solution command

```
docker swarm leave
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run on the intended worker node, not casually on a manager; verify the local state afterward.

Why it matters: Leaving a cluster changes orchestration membership.

Exercise 817 · Leave a Swarm cluster

Task: Remove a disposable worker node from its Swarm after verifying its role.

Solution command

```
docker swarm leave
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run on the intended worker node, not casually on a manager; verify the local state afterward.

Why it matters: Leaving a cluster changes orchestration membership.

Exercise 818 · Leave a Swarm cluster

Task: Remove a disposable worker node from its Swarm after verifying its role.

Solution command

```
docker swarm leave
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run on the intended worker node, not casually on a manager; verify the local state afterward.

Why it matters: Leaving a cluster changes orchestration membership.

Exercise 819 · Leave a Swarm cluster

Task: Remove a disposable worker node from its Swarm after verifying its role.

Solution command

```
docker swarm leave
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run on the intended worker node, not casually on a manager; verify the local state afterward.

Why it matters: Leaving a cluster changes orchestration membership.

Exercise 820 · Leave a Swarm cluster

Task: Remove a disposable worker node from its Swarm after verifying its role.

Solution command

```
docker swarm leave
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker info --format '{.Swarm.LocalNodeState}'`
4. Interpret the result: Run on the intended worker node, not casually on a manager; verify the local state afterward.

Why it matters: Leaving a cluster changes orchestration membership.

Module 42: Swarm stack operations

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 821 · Inspect stack tasks

Task: List tasks for stack labstack1 and review their state.

Solution command

```
docker stack ps labstack1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack1`
4. Interpret the result: Run on a Swarm manager and compare task state with the desired service replicas.

Why it matters: Task inspection is often the fastest way to find scheduling or image-pull failures.

Exercise 822 · Inspect stack tasks

Task: List tasks for stack labstack2 and review their state.

Solution command

```
docker stack ps labstack2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack2`
4. Interpret the result: Run on a Swarm manager and compare task state with the desired service replicas.

Why it matters: Task inspection is often the fastest way to find scheduling or image-pull failures.

Exercise 823 · Inspect stack tasks

Task: List tasks for stack labstack3 and review their state.

Solution command

```
docker stack ps labstack3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack3`
4. Interpret the result: Run on a Swarm manager and compare task state with the desired service replicas.

Why it matters: Task inspection is often the fastest way to find scheduling or image-pull failures.

Exercise 824 · Inspect stack tasks

Task: List tasks for stack labstack4 and review their state.

Solution command

```
docker stack ps labstack4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack4`
4. Interpret the result: Run on a Swarm manager and compare task state with the desired service replicas.

Why it matters: Task inspection is often the fastest way to find scheduling or image-pull failures.

Exercise 825 · Inspect stack tasks

Task: List tasks for stack labstack5 and review their state.

Solution command

```
docker stack ps labstack5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack5`
4. Interpret the result: Run on a Swarm manager and compare task state with the desired service replicas.

Why it matters: Task inspection is often the fastest way to find scheduling or image-pull failures.

Exercise 826 · Inspect stack tasks

Task: List tasks for stack labstack6 and review their state.

Solution command

```
docker stack ps labstack6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack6`
4. Interpret the result: Run on a Swarm manager and compare task state with the desired service replicas.

Why it matters: Task inspection is often the fastest way to find scheduling or image-pull failures.

Exercise 827 · Inspect stack tasks

Task: List tasks for stack labstack7 and review their state.

Solution command

```
docker stack ps labstack7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack7`
4. Interpret the result: Run on a Swarm manager and compare task state with the desired service replicas.

Why it matters: Task inspection is often the fastest way to find scheduling or image-pull failures.

Exercise 828 · Inspect stack tasks

Task: List tasks for stack labstack8 and review their state.

Solution command

```
docker stack ps labstack8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack8`
4. Interpret the result: Run on a Swarm manager and compare task state with the desired service replicas.

Why it matters: Task inspection is often the fastest way to find scheduling or image-pull failures.

Exercise 829 · Inspect stack tasks

Task: List tasks for stack labstack9 and review their state.

Solution command

```
docker stack ps labstack9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack9`
4. Interpret the result: Run on a Swarm manager and compare task state with the desired service replicas.

Why it matters: Task inspection is often the fastest way to find scheduling or image-pull failures.

Exercise 830 · Inspect stack tasks

Task: List tasks for stack labstack10 and review their state.

Solution command

```
docker stack ps labstack10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack services labstack10`
4. Interpret the result: Run on a Swarm manager and compare task state with the desired service replicas.

Why it matters: Task inspection is often the fastest way to find scheduling or image-pull failures.

Exercise 831 · Remove a lab stack

Task: Remove the disposable Swarm stack labstack1 after checking its services.

Solution command

```
docker stack rm labstack1
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack ls`
4. Interpret the result: Verify the stack name and ensure no needed workloads are deployed under it before removal.

Why it matters: Stack removal stops and removes stack-managed services; use only for lab resources.

Exercise 832 · Remove a lab stack

Task: Remove the disposable Swarm stack labstack2 after checking its services.

Solution command

```
docker stack rm labstack2
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack ls`
4. Interpret the result: Verify the stack name and ensure no needed workloads are deployed under it before removal.

Why it matters: Stack removal stops and removes stack-managed services; use only for lab resources.

Exercise 833 · Remove a lab stack

Task: Remove the disposable Swarm stack labstack3 after checking its services.

Solution command

```
docker stack rm labstack3
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack ls`
4. Interpret the result: Verify the stack name and ensure no needed workloads are deployed under it before removal.

Why it matters: Stack removal stops and removes stack-managed services; use only for lab resources.

Exercise 834 · Remove a lab stack

Task: Remove the disposable Swarm stack labstack4 after checking its services.

Solution command

```
docker stack rm labstack4
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack ls`
4. Interpret the result: Verify the stack name and ensure no needed workloads are deployed under it before removal.

Why it matters: Stack removal stops and removes stack-managed services; use only for lab resources.

Exercise 835 · Remove a lab stack

Task: Remove the disposable Swarm stack labstack5 after checking its services.

Solution command

```
docker stack rm labstack5
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack ls`
4. Interpret the result: Verify the stack name and ensure no needed workloads are deployed under it before removal.

Why it matters: Stack removal stops and removes stack-managed services; use only for lab resources.

Exercise 836 · Remove a lab stack

Task: Remove the disposable Swarm stack labstack6 after checking its services.

Solution command

```
docker stack rm labstack6
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack ls`
4. Interpret the result: Verify the stack name and ensure no needed workloads are deployed under it before removal.

Why it matters: Stack removal stops and removes stack-managed services; use only for lab resources.

Exercise 837 · Remove a lab stack

Task: Remove the disposable Swarm stack labstack7 after checking its services.

Solution command

```
docker stack rm labstack7
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack ls`
4. Interpret the result: Verify the stack name and ensure no needed workloads are deployed under it before removal.

Why it matters: Stack removal stops and removes stack-managed services; use only for lab resources.

Exercise 838 · Remove a lab stack

Task: Remove the disposable Swarm stack labstack8 after checking its services.

Solution command

```
docker stack rm labstack8
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack ls`
4. Interpret the result: Verify the stack name and ensure no needed workloads are deployed under it before removal.

Why it matters: Stack removal stops and removes stack-managed services; use only for lab resources.

Exercise 839 · Remove a lab stack

Task: Remove the disposable Swarm stack labstack9 after checking its services.

Solution command

```
docker stack rm labstack9
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack ls`
4. Interpret the result: Verify the stack name and ensure no needed workloads are deployed under it before removal.

Why it matters: Stack removal stops and removes stack-managed services; use only for lab resources.

Exercise 840 · Remove a lab stack

Task: Remove the disposable Swarm stack labstack10 after checking its services.

Solution command

```
docker stack rm labstack10
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker stack ls`
4. Interpret the result: Verify the stack name and ensure no needed workloads are deployed under it before removal.

Why it matters: Stack removal stops and removes stack-managed services; use only for lab resources.

Module 43: Docker plugins & extension discovery

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 841 · List installed Docker plugins

Task: Inventory Docker plugins available to the current Engine for case1.

Solution command

```
docker plugin ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Review plugin names, IDs, descriptions, and enabled status.

Why it matters: Plugin support and availability depend on the Engine build and platform.

Exercise 842 · List installed Docker plugins

Task: Inventory Docker plugins available to the current Engine for case2.

Solution command

```
docker plugin ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Review plugin names, IDs, descriptions, and enabled status.

Why it matters: Plugin support and availability depend on the Engine build and platform.

Exercise 843 · List installed Docker plugins

Task: Inventory Docker plugins available to the current Engine for case3.

Solution command

```
docker plugin ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Review plugin names, IDs, descriptions, and enabled status.

Why it matters: Plugin support and availability depend on the Engine build and platform.

Exercise 844 · List installed Docker plugins

Task: Inventory Docker plugins available to the current Engine for case4.

Solution command

```
docker plugin ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Review plugin names, IDs, descriptions, and enabled status.

Why it matters: Plugin support and availability depend on the Engine build and platform.

Exercise 845 · List installed Docker plugins

Task: Inventory Docker plugins available to the current Engine for case5.

Solution command

```
docker plugin ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Review plugin names, IDs, descriptions, and enabled status.

Why it matters: Plugin support and availability depend on the Engine build and platform.

Exercise 846 · List installed Docker plugins

Task: Inventory Docker plugins available to the current Engine for case6.

Solution command

```
docker plugin ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Review plugin names, IDs, descriptions, and enabled status.

Why it matters: Plugin support and availability depend on the Engine build and platform.

Exercise 847 · List installed Docker plugins

Task: Inventory Docker plugins available to the current Engine for case7.

Solution command

```
docker plugin ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Review plugin names, IDs, descriptions, and enabled status.

Why it matters: Plugin support and availability depend on the Engine build and platform.

Exercise 848 · List installed Docker plugins

Task: Inventory Docker plugins available to the current Engine for case8.

Solution command

```
docker plugin ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Review plugin names, IDs, descriptions, and enabled status.

Why it matters: Plugin support and availability depend on the Engine build and platform.

Exercise 849 · List installed Docker plugins

Task: Inventory Docker plugins available to the current Engine for case9.

Solution command

```
docker plugin ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Review plugin names, IDs, descriptions, and enabled status.

Why it matters: Plugin support and availability depend on the Engine build and platform.

Exercise 850 · List installed Docker plugins

Task: Inventory Docker plugins available to the current Engine for case10.

Solution command

```
docker plugin ls
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Review plugin names, IDs, descriptions, and enabled status.

Why it matters: Plugin support and availability depend on the Engine build and platform.

Exercise 851 · Inspect a plugin

Task: Inspect metadata for plugin PLUGIN_NAME before enabling or changing it.

Solution command

```
docker plugin inspect PLUGIN_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Replace PLUGIN_NAME with an installed plugin's name or ID.

Why it matters: Do not install or enable unknown plugins on a machine with sensitive workloads.

Exercise 852 · Inspect a plugin

Task: Inspect metadata for plugin PLUGIN_NAME before enabling or changing it.

Solution command

```
docker plugin inspect PLUGIN_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Replace PLUGIN_NAME with an installed plugin's name or ID.

Why it matters: Do not install or enable unknown plugins on a machine with sensitive workloads.

Exercise 853 · Inspect a plugin

Task: Inspect metadata for plugin PLUGIN_NAME before enabling or changing it.

Solution command

```
docker plugin inspect PLUGIN_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Replace PLUGIN_NAME with an installed plugin's name or ID.

Why it matters: Do not install or enable unknown plugins on a machine with sensitive workloads.

Exercise 854 · Inspect a plugin

Task: Inspect metadata for plugin PLUGIN_NAME before enabling or changing it.

Solution command

```
docker plugin inspect PLUGIN_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Replace PLUGIN_NAME with an installed plugin's name or ID.

Why it matters: Do not install or enable unknown plugins on a machine with sensitive workloads.

Exercise 855 · Inspect a plugin

Task: Inspect metadata for plugin PLUGIN_NAME before enabling or changing it.

Solution command

```
docker plugin inspect PLUGIN_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Replace PLUGIN_NAME with an installed plugin's name or ID.

Why it matters: Do not install or enable unknown plugins on a machine with sensitive workloads.

Exercise 856 · Inspect a plugin

Task: Inspect metadata for plugin PLUGIN_NAME before enabling or changing it.

Solution command

```
docker plugin inspect PLUGIN_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Replace PLUGIN_NAME with an installed plugin's name or ID.

Why it matters: Do not install or enable unknown plugins on a machine with sensitive workloads.

Exercise 857 · Inspect a plugin

Task: Inspect metadata for plugin PLUGIN_NAME before enabling or changing it.

Solution command

```
docker plugin inspect PLUGIN_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Replace PLUGIN_NAME with an installed plugin's name or ID.

Why it matters: Do not install or enable unknown plugins on a machine with sensitive workloads.

Exercise 858 · Inspect a plugin

Task: Inspect metadata for plugin PLUGIN_NAME before enabling or changing it.

Solution command

```
docker plugin inspect PLUGIN_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Replace PLUGIN_NAME with an installed plugin's name or ID.

Why it matters: Do not install or enable unknown plugins on a machine with sensitive workloads.

Exercise 859 · Inspect a plugin

Task: Inspect metadata for plugin PLUGIN_NAME before enabling or changing it.

Solution command

```
docker plugin inspect PLUGIN_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Replace PLUGIN_NAME with an installed plugin's name or ID.

Why it matters: Do not install or enable unknown plugins on a machine with sensitive workloads.

Exercise 860 · Inspect a plugin

Task: Inspect metadata for plugin PLUGIN_NAME before enabling or changing it.

Solution command

```
docker plugin inspect PLUGIN_NAME
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker plugin ls`
4. Interpret the result: Replace PLUGIN_NAME with an installed plugin's name or ID.

Why it matters: Do not install or enable unknown plugins on a machine with sensitive workloads.

Module 44: Build cache & system cleanup

Each module contains two command families with ten variations each. If a task depends on an existing object, create that object in your lab first or adapt the name to one you already have.

Exercise 861 · Review builder cache before pruning

Task: Inspect build-cache usage before deciding whether cleanup is necessary, case1.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx du`
4. Interpret the result: Review reclaimable cache and builder usage before deleting any cache.

Why it matters: Build cache removal can make future builds slower.

Exercise 862 · Review builder cache before pruning

Task: Inspect build-cache usage before deciding whether cleanup is necessary, case2.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx du`
4. Interpret the result: Review reclaimable cache and builder usage before deleting any cache.

Why it matters: Build cache removal can make future builds slower.

Exercise 863 · Review builder cache before pruning

Task: Inspect build-cache usage before deciding whether cleanup is necessary, case3.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx du`
4. Interpret the result: Review reclaimable cache and builder usage before deleting any cache.

Why it matters: Build cache removal can make future builds slower.

Exercise 864 · Review builder cache before pruning

Task: Inspect build-cache usage before deciding whether cleanup is necessary, case4.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx du`
4. Interpret the result: Review reclaimable cache and builder usage before deleting any cache.

Why it matters: Build cache removal can make future builds slower.

Exercise 865 · Review builder cache before pruning

Task: Inspect build-cache usage before deciding whether cleanup is necessary, case5.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx du`
4. Interpret the result: Review reclaimable cache and builder usage before deleting any cache.

Why it matters: Build cache removal can make future builds slower.

Exercise 866 · Review builder cache before pruning

Task: Inspect build-cache usage before deciding whether cleanup is necessary, case6.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx du`
4. Interpret the result: Review reclaimable cache and builder usage before deleting any cache.

Why it matters: Build cache removal can make future builds slower.

Exercise 867 · Review builder cache before pruning

Task: Inspect build-cache usage before deciding whether cleanup is necessary, case7.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx du`
4. Interpret the result: Review reclaimable cache and builder usage before deleting any cache.

Why it matters: Build cache removal can make future builds slower.

Exercise 868 · Review builder cache before pruning

Task: Inspect build-cache usage before deciding whether cleanup is necessary, case8.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx du`
4. Interpret the result: Review reclaimable cache and builder usage before deleting any cache.

Why it matters: Build cache removal can make future builds slower.

Exercise 869 · Review builder cache before pruning

Task: Inspect build-cache usage before deciding whether cleanup is necessary, case9.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx du`
4. Interpret the result: Review reclaimable cache and builder usage before deleting any cache.

Why it matters: Build cache removal can make future builds slower.

Exercise 870 · Review builder cache before pruning

Task: Inspect build-cache usage before deciding whether cleanup is necessary, case10.

Solution command

```
docker system df -v
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker buildx du`
4. Interpret the result: Review reclaimable cache and builder usage before deleting any cache.

Why it matters: Build cache removal can make future builds slower.

Exercise 871 · Prune build cache with an age filter

Task: In a disposable lab, remove build cache older than 24 hours for cleanup case1.

Solution command

```
docker builder prune --filter until=24h
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Read the confirmation prompt and verify the scope before proceeding.

Why it matters: This is destructive to cache data; do not use broad cleanup on shared or production hosts.

Exercise 872 · Prune build cache with an age filter

Task: In a disposable lab, remove build cache older than 24 hours for cleanup case2.

Solution command

```
docker builder prune --filter until=24h
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Read the confirmation prompt and verify the scope before proceeding.

Why it matters: This is destructive to cache data; do not use broad cleanup on shared or production hosts.

Exercise 873 · Prune build cache with an age filter

Task: In a disposable lab, remove build cache older than 24 hours for cleanup case3.

Solution command

```
docker builder prune --filter until=24h
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Read the confirmation prompt and verify the scope before proceeding.

Why it matters: This is destructive to cache data; do not use broad cleanup on shared or production hosts.

Exercise 874 · Prune build cache with an age filter

Task: In a disposable lab, remove build cache older than 24 hours for cleanup case4.

Solution command

```
docker builder prune --filter until=24h
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Read the confirmation prompt and verify the scope before proceeding.

Why it matters: This is destructive to cache data; do not use broad cleanup on shared or production hosts.

Exercise 875 · Prune build cache with an age filter

Task: In a disposable lab, remove build cache older than 24 hours for cleanup case5.

Solution command

```
docker builder prune --filter until=24h
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Read the confirmation prompt and verify the scope before proceeding.

Why it matters: This is destructive to cache data; do not use broad cleanup on shared or production hosts.

Exercise 876 · Prune build cache with an age filter

Task: In a disposable lab, remove build cache older than 24 hours for cleanup case6.

Solution command

```
docker builder prune --filter until=24h
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Read the confirmation prompt and verify the scope before proceeding.

Why it matters: This is destructive to cache data; do not use broad cleanup on shared or production hosts.

Exercise 877 · Prune build cache with an age filter

Task: In a disposable lab, remove build cache older than 24 hours for cleanup case7.

Solution command

```
docker builder prune --filter until=24h
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Read the confirmation prompt and verify the scope before proceeding.

Why it matters: This is destructive to cache data; do not use broad cleanup on shared or production hosts.

Exercise 878 · Prune build cache with an age filter

Task: In a disposable lab, remove build cache older than 24 hours for cleanup case8.

Solution command

```
docker builder prune --filter until=24h
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Read the confirmation prompt and verify the scope before proceeding.

Why it matters: This is destructive to cache data; do not use broad cleanup on shared or production hosts.

Exercise 879 · Prune build cache with an age filter

Task: In a disposable lab, remove build cache older than 24 hours for cleanup case9.

Solution command

```
docker builder prune --filter until=24h
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Read the confirmation prompt and verify the scope before proceeding.

Why it matters: This is destructive to cache data; do not use broad cleanup on shared or production hosts.

Exercise 880 · Prune build cache with an age filter

Task: In a disposable lab, remove build cache older than 24 hours for cleanup case10.

Solution command

```
docker builder prune --filter until=24h
```

Step-by-step solution

1. Identify the target object, image, service, file, or context named in the task.
2. Run the command above. Replace placeholders only where the task says to do so.
3. Verify the result: `docker system df`
4. Interpret the result: Read the confirmation prompt and verify the scope before proceeding.

Why it matters: This is destructive to cache data; do not use broad cleanup on shared or production hosts.

Final review checklist

■ I can explain the difference between an image and a container.

■ I can inspect, tag, save, load, pull, import, export, and publish images.

■ I can run, list, inspect, stop, start, restart, pause, exec into, attach to, log, and safely remove containers.

■ I understand port publishing, environment variables, restart policies, health checks, resource limits, and Linux capabilities.

■ I can create, inspect, mount, back up, and safely remove volumes.

■ I can create, inspect, connect, disconnect, and safely remove custom networks.

■ I can build images and explain build context, cache, build arguments, targets, and multi-platform builders.

■ I can validate and operate a Compose project, including logs, exec, one-off runs, builds, pulls, and teardown.

■ I know that contexts can redirect Docker commands to a different host.

■ I can explain Swarm managers, nodes, services, replicas, secrets, configs, rollouts, and stacks.

■ I apply least privilege, avoid exposing secrets, and inspect targets before destructive cleanup.

■ I can use docker help and official docs to confirm flags for my installed version.

Command discovery mini-lab

```
docker --help
docker run --help
docker image --help
docker container --help
docker volume --help
docker network --help
docker compose --help
docker buildx --help
docker swarm --help
docker service --help
```

Documentation: https://docs.docker.com/ · CLI reference: https://docs.docker.com/reference/cli/docker/